



Escuela
Politécnica
Superior

Tokenización de Activos Financieros con Smart Contracts



Máster Universitario en Ciencia de Datos

Trabajo Fin de Máster

Autor:

Nicolás Vives Vicente

Tutor:

Miguel Ángel Teruel Martínez

Junio 2026



Universitat d'Alacant
Universidad de Alicante

Tokenización de Activos Financieros con Smart Contracts

Autor

Nicolás Vives Vicente

Tutor

Miguel Ángel Teruel Martínez

Este trabajo ha sido financiado por el proyecto KOSMOS-UA (PID2024-155363OB-C43), financiado por el Ministerio de Ciencia e Innovación de España, el proyecto BALIDA-AA (CIPROM/2024/13), financiado por la Conselleria de Educación, Cultura, Universidades y Empleo (Generalitat Valenciana), el proyecto IAEAV (INREIA/2024/176), financiado por la Conselleria de Innovación, Industria, Comercio y Turismo (Generalitat Valenciana), y la Cátedra ENIA de Inteligencia Artificial (TSI-100927-2023-6), financiada por el Plan de Recuperación, Transformación y Resiliencia de la Unión Europea Next Generation a través del Ministerio de Transformación Digital y Función Pública.



Máster Universitario en Ciencia de Datos



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, junio 2026

Resumen

El mercado inmobiliario representa uno de los sectores con mayor volumen de capital a nivel global, pero su gestión tradicional depende de sistemas centralizados y procesos de validación manuales que limitan su eficiencia y accesibilidad. Como respuesta a estas carencias, este trabajo explora la tokenización de *Real World Assets*, un proceso que transforma activos físicos en estructuras de datos programables, inmutables y auditables mediante la tecnología *Blockchain*. El objetivo principal es democratizar la inversión en este sector, reduciendo las barreras de entrada y la iliquidez del modelo tradicional mediante el fraccionamiento de la propiedad y su registro en una base de datos distribuida. Para llevar a cabo este desarrollo, el marco teórico establece las bases de la descentralización económica, analizando la evolución desde **Bitcoin** hasta la red **Ethereum**, donde la “*Ethereum Virtual Machine*” (EVM) permite la ejecución imparcial y autónoma de acuerdos a través de *smart contracts*. Se examinan en profundidad los estándares de *tokens* aplicables, prestando especial atención al ERC-20 para la fungibilidad y al ERC-3643 como marco robusto para el cumplimiento normativo de *security tokens*. Adicionalmente, el proyecto se contextualiza bajo la normativa legal vigente, destacando el impacto del Reglamento europeo MiCA y la Ley 6/2023 de los Mercados de Valores en España, fundamentales para dotar a las emisiones tokenizadas de protección al inversor y seguridad jurídica.

En la parte práctica se realiza el diseño y desarrollo de una plataforma descentralizada (*dApp*) basada en un patrón “Factory”, compuesto por un contrato “Mercado”, que gestiona los activos representados por múltiples contratos “Inmueble” (basados en OpenZeppelin ERC-20). Por último se evalúa a la plataforma mediante pruebas unitarias que validan con una tasa de éxito del 100 % el control de acceso basado en roles (RBAC) y la precisión matemática de las transacciones financieras. Paralelamente, el análisis de eficiencia demuestra que los costes operativos (*Gas*) de la compraventa son notablemente inferiores a los gastos registrales y notariales convencionales.

Palabras clave: Tokenización, Bitcoin, Ethereum, *Blockchain* y *Smart Contract*.

Abstract

The real estate market represents one of the sectors with the highest capital volume globally, but its traditional management relies on centralized systems and manual validation processes that limit its efficiency and accessibility. In response to these shortcomings, this work explores the tokenization of *Real World Assets*, a process that transforms physical assets into programmable, immutable, and auditable data structures using *Blockchain* technology. The main objective is to democratize investment in this sector, reducing the barriers to entry and the illiquidity of the traditional model through the fractionalization of ownership and its registration in a distributed database. To carry out this development, the theoretical framework establishes the foundations of economic decentralization, analyzing the evolution from **Bitcoin** to the **Ethereum** network, where the “*Ethereum Virtual Machine*” (EVM) allows the impartial and autonomous execution of agreements through *smart contracts*. The applicable token standards are examined in depth, paying special attention to ERC-20 for fungibility and ERC-3643 as a robust framework for the regulatory compliance of *security tokens*. Additionally, the project is contextualized under current legal regulations, highlighting the impact of the European MiCA Regulation and Law 6/2023 on Securities Markets in Spain, which are essential to provide tokenized issuances with investor protection and legal certainty.

In the practical section, the design and development of a decentralized platform (*dApp*) based on a Factory pattern is carried out, composed of a “Mercado” (market) contract, which manages the assets represented by multiple “Inmueble” (property) contracts (based on OpenZeppelin ERC-20). Finally, the platform is evaluated through unit tests that validate with a 100 % success rate the role-based access control (RBAC) and the mathematical precision of the financial transactions. In parallel, the efficiency analysis demonstrates that the operational costs (*Gas*) of buying and selling are notably lower than conventional registry and notarial fees.

Keywords: Tokenization, Bitcoin, Ethereum, Blockchain, and Smart Contract.

Índice general

1. Introducción	1
1.1. Centralización vs. Descentralización	1
1.1.1. Alcance de la descentralización	2
1.1.2. Beneficios de la descentralización	3
1.2. Motivación: La democratización del sector inmobiliario	3
2. Marco Teórico	5
2.1. Situación económica del siglo XX	5
2.1.1. Impacto del fin del patrón oro en el ciudadano	5
2.1.2. Intentos de vuelta al dinero sólido	6
2.2. Bitcoin	6
2.2.1. Nacimiento de Bitcoin	6
2.2.2. Fundamentos conceptuales de Bitcoin	7
2.2.2.1. Un <i>Ledger</i> público	7
2.2.3. Identidad y seguridad	8
2.2.3.1. Funciones <i>hash</i>	8
2.2.3.2. Transacciones	12
2.2.3.3. Firma digital: Clave pública y privada	12
2.2.3.4. Árbol de Merkle	14
2.2.4. Arquitectura de la red	14
2.2.4.1. Nodos y mineros	15
2.2.5. Bloques	16
2.2.6. Consenso e integridad de la red	19
2.2.6.1. Algoritmo de consenso: <i>Proof-of-Work</i> (PoW)	19
2.2.6.2. Reorganización de la red	20
2.2.6.3. Ataque del 51 %	21
2.2.6.4. <i>Hard Fork</i>	21
2.2.6.5. <i>Soft Fork</i>	22
2.3. Ethereum	23
2.3.1. Nacimiento de Ethereum	23
2.3.2. <i>Smart Contracts</i>	24
2.3.3. El concepto de Gas y el incentivo de cómputo	26
2.3.4. <i>Ethereum Virtual Machine</i>	27
2.3.5. Modelo de cuentas: EOA vs Contratos	29
2.3.5.1. Cuentas de Propiedad Externa (EOA)	29
2.3.5.2. Contract Accounts	29

2.3.6.	<i>The Merge</i> : Del Proof-of-Work al Proof-of-Stake	29
2.3.6.1.	Proof of Stake	30
2.3.7.	Estándares de Tokens: La base de la tokenización	32
2.3.7.1.	ERC-20	32
2.3.7.2.	ERC-721	34
2.3.7.3.	ERC-3643	35
2.3.8.	<i>Utility</i> vs. <i>Security tokens</i>	36
2.4.	Fundamentos de la Tokenización de Activos	37
2.4.1.	Tokenización de activos físicos	38
2.4.2.	Tokenización de inmuebles	39
2.4.3.	Beneficios y riesgos de la tokenización de inmuebles	40
2.5.	El auge de la tokenización RWA	42
2.5.1.	Proyecciones y evolución del mercado en la próxima década	42
2.6.	Modelos de Tokenización Inmobiliaria	43
2.6.1.	Modelos de participación: Directo vs. Indirecto	44
2.6.2.	Rendimientos de la tokenización inmobiliaria	44
2.7.	Marco regulatorio y cumplimiento legal en la tokenización de inmuebles en España	45
2.7.1.	Impacto de MiCA	45
2.7.2.	Ley 6/2023 de los Mercados de Valores	45
2.7.3.	Notaría y Registro de la Propiedad	46
2.8.	Estándar para la tokenización de activos	46
3.	Objetivos	49
4.	Metodología y desarrollo	51
4.1.	Entorno de desarrollo	51
4.2.	Almacenamiento descentralizado de documentos: IPFS y Pinata	52
4.3.	Arquitectura del sistema	53
4.4.	Lógica de los <i>Smart Contracts</i>	56
4.4.1.	El contrato Inmueble	56
4.4.2.	El contrato Mercado: Lógica de <i>Factory</i> y gestión de liquidez	58
4.5.	Desarrollo de la interfaz de usuario (dApp)	60
4.6.	Despliegue y verificación de contratos	64
5.	Pruebas y análisis de resultados	67
5.1.	Validación de funciones (Test unitarios)	67
5.2.	Análisis de eficiencia y costes de Gas	68
5.3.	Evaluación de la experiencia de uso y eficiencia del proceso	70
5.3.1.	Experiencia de usuario en la dApp	70
5.3.2.	Análisis de eficiencia	73
6.	Conclusiones	75

<i>Índice general</i>	IX
-----------------------	----

Bibliografía	77
---------------------	-----------

A. Anexo	83
-----------------	-----------

Índice de figuras

2.1. Ejemplo intuitivo del funcionamiento de una función <i>hash</i> . Adaptado de (Walker, 2015c)	8
2.2. Determinismo en una función <i>hash</i> . Adaptado de (Walker, 2015c).	9
2.3. Salida de longitud constante frente a entradas variables. Adaptado de (Walker, 2015c).	9
2.4. Efecto avalancha ante cambios mínimos. Adaptado de (Walker, 2015c).	9
2.5. Propiedad de irreversibilidad de la función <i>hash</i> . Adaptado de (Walker, 2015c)	10
2.6. Concepto de resistencia a colisiones. Adaptado de (Walker, 2015c).	10
2.7. Inviabilidad de forzar un resultado <i>hash</i> determinado. Adaptado de (Walker, 2015c).	11
2.8. Ejemplo de la estructura de la raíz de Merkle. Tomada de (Walker, 2015c)	15
2.9. Diagrama de la estructura de un bloque. Tomada de (Walker, 2015a).	17
2.10. Diagrama de ejemplo de un <i>smart contract</i> . De elaboración propia.	25
2.11. Diagrama del funcionamiento de Ethereum. Adaptado de (Ethereum Foundation, 2025c).	28
2.12. Diagrama del funcionamiento de <i>Proof of Stake</i> en Ethereum. De elaboración propia.	31
2.13. El ERC-3643 es un conjunto de <i>smart contracts</i> . Tomada de (ERC-3643 Association, 2025).	36
2.14. Crecimiento estimado de la tokenización RWA hasta 2033. Tomada de (Ripple & Boston Consulting Group, 2025).	43
2.15. La necesidad de un marco de tokenización común. Tomada de (ERC-3643 Association, 2025).	47
4.1. Arquitectura del proyecto. De elaboración propia.	55
5.1. Resultados en consola de los <i>test</i> unitarios. De elaboración propia.	68
5.2. Interfaz de la aplicación antes de iniciar sesión. De elaboración propia.	70
5.3. La aplicación mostrando los inmuebles disponibles. De elaboración propia.	71
5.4. Interfaz de la aplicación para la <i>wallet</i> del administrador. De elaboración propia.	72
5.5. Interfaz de la aplicación para inversores. De elaboración propia.	73

Índice de tablas

2.1. Comparativa entre el Contrato Tradicional y el <i>Smart Contract</i>	26
5.1. Perfil de consumo computacional (<i>Gas</i>) de los <i>smart contracts</i>	69
5.2. Comparativa de eficiencia: Modelo Tradicional vs. Plataforma de Tokenización	74

1. Introducción

El mercado inmobiliario representa uno de los sectores con mayor volumen de capital y generación de datos a nivel global, siendo un pilar fundamental en la economía actual. A pesar de su relevancia, la gestión y transferencia de estos activos depende de sistemas de información independientes entre sí y de procesos de validación manuales, limitando así la eficiencia y la transparencia. Con el surgimiento de Bitcoin (Nakamoto, 2008), la tecnología *Blockchain* y la criptografía asociada ofrecen un nuevo paradigma para la representación digital de valor. Este trabajo aborda la tokenización de activos del mundo real (RWA), profundizando concretamente en los inmuebles, por su relevancia, sencillez y capacidad ilustrativa, aunque es aplicable a cualquier activo de este grupo. Se debe destacar que por inmueble se entiende cualquier tipo de edificación: naves industriales, locales, oficinas, hospitales... por lo que el trabajo no busca fomentar la especulación con la vivienda sino dar transparencia, accesibilidad, seguridad y solucionar el problema de la iliquidez inmobiliaria. No se trata exclusivamente de un proceso financiero, sino de un reto de ciencia de datos, ya que se transforman activos físicos en estructuras de información programables, inmutables y auditables, para automatizar procesos que tradicionalmente dependían de la intervención humana, y garantizar así la integridad de los datos.

Para comprender esta revolución tecnológica, es necesario retroceder y analizar las bases sobre las que se ha construido la sociedad moderna, pues la tokenización surge como respuesta directa a carencias e ineficiencias de los modelos tradicionales de organización. La concentración de los datos y de la autoridad en entidades únicas ha supuesto, durante años, serios retos en términos de seguridad, resiliencia y confianza, evidenciando la necesidad de un cambio de sistema.

1.1. Centralización vs. Descentralización

A lo largo de la historia, la centralización ha surgido como solución a problemas fundamentales de confianza. En el ámbito político, el poder se concentró en figuras de autoridad única como los monarcas, cuya función era garantizar el orden. De manera similar, en el campo de la educación y del conocimiento, el acceso estuvo centralizado durante siglos en manos de instituciones y élites privilegiadas, mientras que la mayoría de la población carecía de oportunidades para instruirse.

Esta concentración no ha sido universal ni permanente, a lo largo de la historia han surgido períodos de descentralización. En la antigua Atenas, por ejemplo, se desarrolló un sistema de democracia directa en el que los ciudadanos varones mayores de edad —a partir de dieciocho años— con ciudadanía ateniense participaban activamente en la toma

de decisiones políticas. Estos podían asistir a la Ekklesia (Asamblea), donde se votaban directamente todas las leyes y decisiones importantes sobre la polis. En la actualidad, nos encontramos en un ciclo de descentralización del poder político, o al menos en cierto grado, ya que la ciudadanía delega la toma de decisiones en un número reducido de representantes que, en muchas ocasiones, no refleja completamente los intereses y valores de sus electores. Del mismo modo, el acceso al conocimiento ha experimentado una transformación, mientras que durante la mayor parte de la historia estuvo centralizado en élites educadas, en la actualidad la educación pública y la revolución digital han permitido una mayor democratización del acceso a la información.

Históricamente, debido a limitaciones tecnológicas, el activo más difícil de descentralizar ha sido el dinero. Este ha permanecido centralizado al haber tenido siempre la necesidad de un tercero de confianza para asegurar la validez de la transacción y evitar así pagar a dos partes distintas con el mismo capital; es decir, para no caer en el problema del doble gasto. El dinero en sí mismo no posee apenas valor, el coste de fabricar un billete ronda los escasos céntimos de euro. Un billete de cien euros tiene un valor de cien euros porque existe un tercero de confianza —en el caso de ser euros, el Banco Central Europeo— que certifica que tiene dicho valor. En el caso del dinero digital, empresas como Visa, Mastercard o PayPal son las encargadas de actuar como intermediarios financieros y validar que la transacción entre dos usuarios es posible. Estas empresas, por supuesto, no realizan su trabajo de manera altruista, sino que generan millones de euros cada día gracias a las comisiones (Caballero, 2019). ¿Qué pasaría si se pudiera descentralizar el dinero? ¿Cómo se podría hacer esto? ¿Quién se encargaría de verificar las transacciones? ¿Lo haría de manera altruista y desinteresada?

Todas estas preguntas las responden Bitcoin y su tecnología **Blockchain** (Nakamoto, 2008), la primera moneda que logró la descentralización económica. Analizaremos con gran detalle este fenómeno en el capítulo 2.

1.1.1. Alcance de la descentralización

Resulta natural pensar que si esta tecnología ha logrado solucionar la descentralización del capital —un reto históricamente inalcanzable—, la evolución lógica sea la descentralización de otros activos como empresas, derechos de autor o préstamos. La aparición de redes como Ethereum ha permitido dar este paso más allá logrando la descentralización de los servicios financieros o DeFi (*Decentralized Finance*) (Buterin, 2014).

A diferencia del sistema financiero tradicional, donde las reglas del juego son dictadas y ejecutadas por instituciones centralizadas, en el ecosistema DeFi las reglas se inscriben en código informático autoejecutable: los *smart contracts*. Estos contratos permiten prescindir de un tercero de confianza que valide el cumplimiento de un acuerdo, ya que la propia red garantiza su ejecución imparcial. Esta transformación no es solo técnica, sino social, ya que democratiza el acceso a herramientas de generación de riqueza que antes estaban restringidas a inversores institucionales o grandes capitales debido a las barreras de entrada y los costes operativos de la banca convencional.

1.1.2. Beneficios de la descentralización

La transición hacia sistemas descentralizados no responde únicamente a una inquietud ideológica, sino a la búsqueda de una eficiencia operativa que el sistema tradicional no puede alcanzar. Al eliminar los almacenes de información y los puntos únicos de fallo, la descentralización ofrece tres pilares fundamentales para la gestión de activos:

- **Eficiencia y reducción de costes:** La infraestructura financiera tradicional depende de una red compleja de intermediarios (corresponsales bancarios, cámaras de compensación, auditores) que añaden capas de comisiones y tiempos de liquidación lentos. Se estima que la implementación de tecnologías de registro distribuido implicaría un ahorro que llegaría a los 20.000 millones de dólares anuales en costes de liquidación, regulación y pagos transfronterizos (Fanning & Centers, 2016).
- **Transparencia e inmutabilidad:** En un sistema descentralizado, cada transacción queda registrada de forma verificable y permanente. Esto elimina la asimetría de información, garantizando que todos los participantes tengan acceso a una “única fuente de verdad”, lo cual es crítico en la gestión de activos donde la trazabilidad de la propiedad es clave.
- **Resiliencia y disponibilidad:** Al no depender de un servidor central, las redes descentralizadas operan constantemente sin interrupciones. Esta característica permite que los mercados no cierren, facilitando el intercambio de valor de manera global e instantánea.

1.2. Motivación: La democratización del sector inmobiliario

Tradicionalmente, la inversión en bienes inmuebles ha sido un privilegio reservado a grandes capitales o instituciones. El proceso de tokenización fragmenta la propiedad en unidades digitales mínimas, reduciendo el ticket mínimo de entrada y permitiendo una diversificación del riesgo sin precedentes. Bajo este sistema un inversor sería capaz de repartir su capital en diez propiedades de diferentes ciudades del mundo en lugar de concentrarlo en un solo activo local. Estas fracciones que llamamos *tokens* pueden intercambiarse con mayor agilidad que un inmueble físico, cuya venta se suele demorar meses debido a los numerosos trámites. Aunque la regulación de estos mercados está en plena evolución en el marco europeo, los avances normativos sugieren una integración inminente con los sistemas financieros tradicionales.

La relevancia de este trabajo no solo reside en su potencial teórico, sino en una realidad de mercado que ya moviliza miles de millones de dólares. Partiendo de una base en 2022 de 5 000 millones de USD, en abril de 2026 hay tokenizados 29.300 millones de USD (RWA.xyz, 2026) y las predicciones sitúan el mercado de activos tokenizados entre 12.5 *trillion* (125.000.000 millones) y 23.4 *trillion* (234.000.000 millones) para 2033 (Ripple & Boston Consulting Group, 2025).

Instituciones financieras de primer nivel han validado este modelo. El mayor gestor de activos del mundo, **BlackRock**, lanzó su fondo tokenizado *BUIDL* sobre la red de

Ethereum, alcanzando rápidamente una capitalización superior a los 2.000 millones de dólares (U.S. Securities and Exchange Commission, 2024). En el ámbito inmobiliario específicamente, proyectos como el de **Multibank Group** en Dubái —con un portafolio de 3.000 millones de dólares— demuestran que la tokenización es uno de los negocios del siglo y una herramienta para presente y futuro.

2. Marco Teórico

2.1. Situación económica del siglo XX

Para comprender por qué surge Bitcoin (Nakamoto, 2008) es interesante situarse en un mundo previo a su nacimiento. Hasta la Primera Guerra Mundial, los países utilizaban sus propias monedas respaldadas por el patrón oro. Este sistema funcionaba como mecanismo de control de la oferta monetaria: cada moneda en circulación estaba respaldada por una cantidad de oro equivalente almacenada en los bancos centrales de los países, por lo que la capacidad de los gobiernos para aumentar arbitrariamente la oferta de dinero estaba limitada. Esta limitación inherente prevenía la inflación descontrolada, ya que la cantidad de dinero disponible estaba directamente vinculada a las reservas físicas de oro. Sin embargo, el patrón oro demostró ser insuficiente durante la Primera Guerra Mundial, cuando los gobiernos necesitaban financiar gastos militares masivos que excedían con creces sus reservas de oro. Así, los países abandonaron progresivamente el patrón oro en favor del dinero fiduciario, lo que permitió a los gobiernos emitir la cantidad de capital necesaria para financiar el esfuerzo de guerra (Caballero, 2019). El dinero fiduciario es dinero cuyo valor no se basa en un respaldo físico como el oro, sino en la confianza que deposita la población en la institución emisora, en aquel entonces, el banco central del país. Este cambio de paradigma otorgó a los gobiernos la capacidad de emitir dinero en función de las necesidades políticas y económicas, sustituyendo la escasez física por una gestión basada en la promesa de la estabilidad futura.

2.1.1. Impacto del fin del patrón oro en el ciudadano

A raíz de esta nueva forma de emitir dinero el ciudadano se vuelve cada vez más pobre. Consideremos un ejemplo: en un estado con mil millones de euros en circulación, si poseemos diez mil euros, representamos el 0,001 % del dinero del país. Si el gobierno del país decide emitir cien millones de euros adicionales, habrá aumentado la oferta en un 10 % pero nosotros continuaremos teniendo diez mil euros, que ahora representan aproximadamente el 0,00091 % del dinero total del país. Nuestra fracción del dinero disponible se ha reducido en un 10 % respecto a la que teníamos antes de la emisión. Este fenómeno es el resultado directo del fin del patrón oro: una inflación la cual, aunque a día de hoy se encuentra controlada, empobrece progresivamente a los ciudadanos (Caballero, 2019).

2.1.2. Intentos de vuelta al dinero sólido

Durante la década de los noventa del siglo XX surgieron cuatro relevantes proyectos de dinero digital que posteriormente tendrían un impacto directo en la creación de Bitcoin.

David Chaum comenzó a escribir sobre criptografía desde principios de los ochenta, pero fue en 1989 cuando presentó su proyecto más ambicioso: DigiCash. Se trataba de una plataforma de dinero digital cuya característica distintiva era el anonimato de las transacciones gracias a protocolos criptográficos avanzados (Chaum et al., 1990). Sin embargo, el fracaso de Chaum como gestor empresarial llevó a la compañía a la quiebra en 1998 (Pitta, 1999).

En 1994, Adam Back desarrolló el concepto de prueba de trabajo (PoW), que analizaremos en detalle en la sección 2.2.6.1. Su propuesta consistía en utilizar esta prueba para combatir el spam de correos electrónicos, obligando al ordenador a resolver un pequeño problema criptográfico cada vez que se enviaba un mensaje (Back, 2002).

Nick Szabo contribuyó significativamente a estos desarrollos. En 1994 escribió una introducción a los contratos inteligentes y en 1996 profundizó en sus posibilidades (Szabo, 1996). Szabo propuso un mercado digital sustentado en procesos automáticos y criptográficamente seguros y en 1998 diseñó “bit gold”, un mecanismo para una moneda digital descentralizada. Aunque “bit gold” nunca fue implementado, se ha reconocido como “un precursor directo de la arquitectura de Bitcoin” (Szabo, 2005).

En 1998, Wei Dai publicó “b-money”, un sistema de efectivo electrónico anónimo y distribuido. En su artículo, Dai describió las propiedades fundamentales que posteriormente se implementarían en Bitcoin (Dai, 1998).

2.2. Bitcoin

2.2.1. Nacimiento de Bitcoin

El 31 de octubre de 2008, en plena crisis financiera mundial, Satoshi Nakamoto publicó un *paper* titulado “Bitcoin: A Peer-to-Peer Electronic Cash System” (Nakamoto, 2008). Este era un documento de apenas nueve páginas que presentó, sin cifras ni grandes detalles técnicos, un sistema de dinero digital que pretendía prescindir de terceros de confianza. El *paper* fue tan prometedor que llamó la atención de Adam Back, a quien Nakamoto citó en (Nakamoto, 2008) por su trabajo sobre la *Proof of Work* de 1994 (Back, 2002), y de Hal Finney, quien había colaborado con Back en un proyecto anterior. Juntos y con la contribución de otros desarrolladores iniciaron el trabajo en el nuevo proyecto. El 3 de enero de 2009 se lanzó la red con el bloque génesis y sus desarrolladores comenzaron a minar; a lo largo del trabajo, en las secciones siguientes, se verá qué es un bloque y qué es la minería. Aquel día nació una tecnología que ha cambiado el mundo y todavía no se conoce la identidad de Satoshi Nakamoto, quién se estima que posee más de un millón de bitcoins distribuidos entre todas sus direcciones.

2.2.2. Fundamentos conceptuales de Bitcoin

En esta sección se exponen los fundamentos esenciales para la comprensión del funcionamiento de Bitcoin. El objetivo principal es establecer un marco conceptual preliminar que facilite el análisis detallado de los conceptos que se abordarán en profundidad en las secciones posteriores.

2.2.2.1. Un Ledger público

Definición 2.2.1 *Un libro contable, también conocido como libro de contabilidad, funciona como un registro metódico y transparente donde se registran cronológicamente todos los movimientos económicos, desde los ingresos y gastos diarios hasta las variaciones en el patrimonio personal y las deudas, ya sea para una organización o para un individuo.*

Su función principal es garantizar la trazabilidad de los fondos, facilitando la planificación y el cumplimiento de requisitos bancarios y fiscales. Almacenar esta información de forma precisa, permite demostrar solvencia y facilitar la toma de decisiones financieras.

Una característica crucial de un libro contable es que debe ser confiable y controlado. Si el libro contable fuera abiertamente distribuido y cualquiera pudiera agregar transacciones directamente sin autorización ni verificación, esto presentaría un problema fundamental pues no habría forma de garantizar la integridad de los datos. Cualquier persona podría modificar registros, añadir transacciones fraudulentas o alterar información histórica, lo que haría imposible confiar en la información contenida en el libro. Por ejemplo, David podría inscribirse a sí mismo como acreedor de Raúl por 100 euros sin su consentimiento.

Bitcoin soluciona este problema con el uso de las firmas digitales que veremos en la sección 2.2.3.3 incorporando un libro contable distribuido que puede funcionar como moneda por derecho propio. Al igual que sucede con una transferencia bancaria, el dinero es transferido de una cuenta a otra una vez que se registra la operación en el libro contable aprobado por las entidades. Al tratarse de un libro contable distribuido, la idea principal es que cada nodo de la red mantenga una copia del libro contable y la actualice continuamente incorporando todas las transacciones nuevas que recibe. Sin embargo, surge un problema clave: ¿cuál sería el libro contable válido si cada nodo incorpora las transacciones en un orden diferente? Esta desincronización puede generar situaciones inconsistentes. Por ejemplo, en algunos libros contables podría llegar una transacción que un usuario no puede permitirse, pero que resultaría válida si aún no ha llegado la transacción previa que agotaría sus fondos. Este problema de consenso distribuido es precisamente lo que aborda el *paper* original de Bitcoin, la solución que se propone es adoptar el libro que lleva un mayor trabajo computacional y para ello se introduce el concepto de cadena de bloques, del inglés *Blockchain*. Formalmente se define una cadena de bloques como:

Definición 2.2.2 *Una cadena de bloques, o Blockchain, es una estructura de datos cuya información se agrupa en conjuntos (bloques) a los que se les añade metainformaciones*

relativas a otro bloque para enlazarlos criptográficamente. Cada uno contiene una huella digital del bloque anterior, asegurando una secuencia cronológica inmutable. Estas estructuras son resistentes a la manipulación y a la modificación de datos, dado que cualquier alteración en un bloque invalidaría toda la cadena posterior. Esta propiedad permite que la Blockchain opere como una base de datos distribuida capaz de mantener un historial irrefutable de transacciones sin depender de una autoridad central (Yaga et al., 2018).

Aunque esta definición puede parecer complicada podemos resumir una *blockchain* como un libro contable distribuido junto con un mecanismo de consenso. El objetivo principal es dividir el registro contable descrito anteriormente en bloques individuales y utilizando herramientas criptográficas que veremos más adelante, encadenar estos bloques entre sí creando una estructura inmutable. De este modo, los participantes de la red solo necesitan compartir el último bloque **minado** —término que se definirá en breve— para mantener sus cadenas actualizadas.

2.2.3. Identidad y seguridad

En esta sección se introducen con detalles los elementos que se encargarán de garantizar la identidad y la seguridad de la red Bitcoin.

2.2.3.1. Funciones *hash*

Un algoritmo *hash* toma una entrada, ya sea un texto, un archivo, o cualquier dato digital, y lo transforma en una cadena de caracteres que funciona como una huella digital del contenido original. Para que una, aparentemente simple, cadena de caracteres actúe como tal, debe cumplir una serie de propiedades que veremos en esta sección. En la Figura 2.1 se observa de forma intuitiva cómo actúan esta clase de algoritmos.



Figura 2.1.: Ejemplo intuitivo del funcionamiento de una función *hash*. Adaptado de (Walker, 2015c)

Aunque existe una amplia variedad de funciones *hash*, este trabajo se centra en aquellas implementadas en la red Bitcoin. Para comprender su funcionamiento, es fundamental definir primero las propiedades que caracterizan a las funciones *hash* básicas:

1. **Determinismo:** ante una misma entrada, la función debe generar siempre el mismo valor de salida, garantizando la consistencia del proceso.

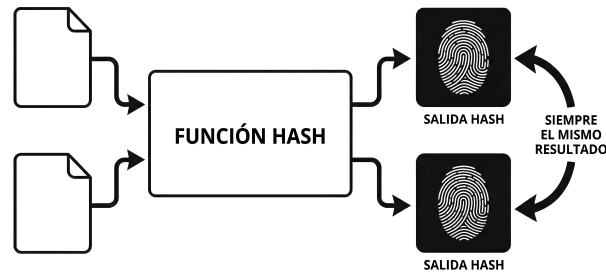


Figura 2.2.: Determinismo en una función *hash*. Adaptado de (Walker, 2015c).

2. **Longitud de salida fija:** independientemente del volumen o tamaño de los datos de entrada, el *hash* resultante debe poseer siempre una extensión constante y predefinida por el algoritmo.

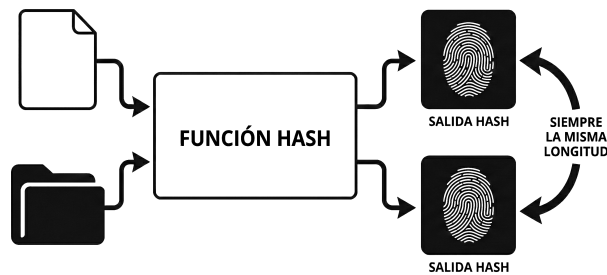


Figura 2.3.: Salida de longitud constante frente a entradas variables. Adaptado de (Walker, 2015c).

3. **Efecto avalancha:** cualquier modificación mínima en la información de entrada debe producir un cambio drástico y total en el *hash* resultante, haciendo que la salida parezca aleatoria respecto a la original.

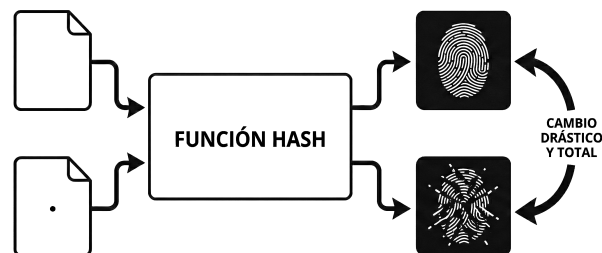


Figura 2.4.: Efecto avalancha ante cambios mínimos. Adaptado de (Walker, 2015c).

No obstante, para aplicaciones de seguridad en entornos distribuidos como Bitcoin, se

requieren las denominadas **funciones hash criptográficas**, las cuales deben cumplir adicionalmente con tres pilares de seguridad:

1. **Unidireccionalidad:** la función debe ser irreversible, lo que implica que resulta computacionalmente inviable obtener el contenido original a partir del *hash*.

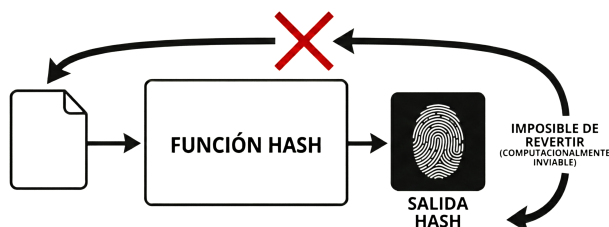


Figura 2.5.: Propiedad de irreversibilidad de la función *hash*. Adaptado de (Walker, 2015c)

2. **Resistencia a colisiones:** debe ser prácticamente imposible encontrar dos entradas distintas que produzcan el mismo valor *hash*. Aunque teóricamente será posible debido a la finita dimensión del conjunto de salida frente a la infinitud de posibles entradas, una función segura garantiza que este suceso sea muy improbable en la práctica.

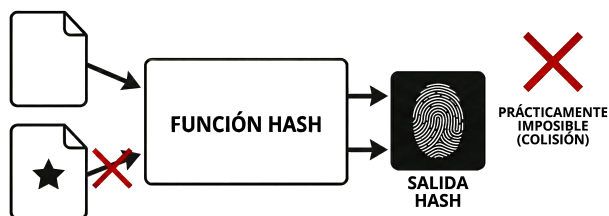


Figura 2.6.: Concepto de resistencia a colisiones. Adaptado de (Walker, 2015c).

3. **Resistencia a la preimagen:** este aspecto es clave y hace referencia a la imposibilidad de manipular la entrada para obtener un resultado *hash* específico. El usuario no puede predecir ni forzar qué datos generarán una salida determinada, obligando a un proceso de prueba y error.

A continuación se describen las funciones Hash más relevantes en Bitcoin:

SHA-256: SHA-256 significa Secure Hash Algorithm 256-bit (Algoritmo de Hash Seguro de 256 bits). La función SHA-256 específicamente produce secuencias de 256 bits, generando un total de 2^{256} posibilidades, un número del orden de 10^{77} , prácticamente

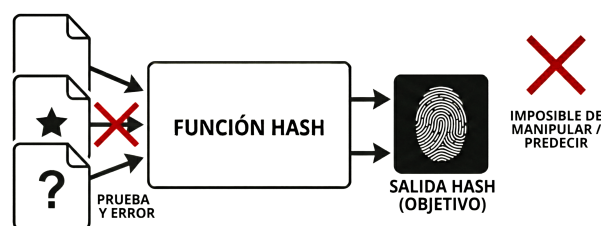


Figura 2.7.: Inviabilidad de forzar un resultado *hash* determinado. Adaptado de (Walker, 2015c).

irreproducible. Este valor se codifica en sistema hexadecimal, resultando en una representación de 64 caracteres compuestos por dígitos del 0 al 9 y letras de la *a* a la *f*. Cada carácter de la cadena representa entonces 4 de los 256 bits originales permitiendo una representación mucho más compacta tal y como se ve en el siguiente ejemplo:

Para la cadena de caracteres **Buenos días**, la función SHA-256 genera el siguiente *hash*:

21aebc48383652046150c3663a57b1cb5bf957b5ece024deb9cbc087478cdf80

Sin embargo, si se modifica un solo carácter, por ejemplo, cambiando la mayúscula por minúscula a **buenos días**, se obtiene un resultado completamente distinto:

5ab4d7f1fc854a344b5f3ed88ef38ede2304d14febc3cd0cab3a07cc923d6b7b

RIPEMD-160: RIPEMD-160 (acrónimo de *RACE Integrity Primitives Evaluation Message Digest*) es una función criptográfica *hash* de 160 bits cuyos *hashes* se representan como 40 caracteres hexadecimales. Es ideal para crear huellas digitales más cortas y todavía seguras.

HASH-256: Se trata del método principal para el *hashing* de datos en Bitcoin. Denominado también como “doble SHA256” o “SHA-256d” consiste en aplicar dos veces la función SHA-256, esto es hacer un *hash* del *hash*.

$$\text{SHA-256}(\text{SHA-256}(\text{data}))$$

Satoshi nunca mencionó por qué decidió usar dos veces la función SHA-256, hoy en día “hashear” dos veces los datos es solo una peculiaridad de Bitcoin y no hay ninguna ventaja en hacerlo, además, las actualizaciones recientes de Bitcoin favorecen el uso de un solo SHA-256 siempre que sea posible (Walker, 2015c).

HASH-160: Este algoritmo funciona procesando los datos mediante SHA-256 y luego aplicando al resultado la función RIPEMD-160. HASH-160 solo se utiliza al construir direcciones heredadas.

2.2.3.2. Transacciones

Una transacción de Bitcoin es esencialmente un conjunto de datos que registra la transferencia de valor entre direcciones, contiene información sobre la cantidad enviada, la cuenta de origen y la cuenta de destino. A diferencia del dinero físico o de las cuentas bancarias tradicionales, Bitcoin opera bajo el modelo UTXO (*Unspent Transaction Output* o Salida de Transacción No Gastada) (Nakamoto, 2008). En este sistema, los fondos no se almacenan como un saldo numérico, sino como “batches” (lotes) de transacciones previas recibidas.

Para realizar un envío, el emisor debe agrupar uno o varios de estos lotes hasta alcanzar o superar la cantidad a enviar y si el valor total de los UTXO seleccionados excede esta cantidad, la red genera automáticamente una dirección de cambio para devolver el excedente al emisor.

Para garantizar la seguridad y propiedad de los fondos, cada lote permanece bloqueado criptográficamente y solo puede ser liberado mediante la **clave privada** correspondiente a la dirección del dueño, proceso que permite validar la transacción y generar nuevos UTXO para el receptor.

2.2.3.3. Firma digital: Clave pública y privada

Un usuario de la red dispone de dos claves criptográficas: una **clave pública** y una **clave privada**. Todos los participantes de la red conocen la clave pública y esta funciona de manera similar al IBAN de una cuenta bancaria, en cambio, la clave privada debe ser conocida exclusivamente por el propio usuario y actúa como una contraseña que solo él posee.

La clave privada es un número aleatorio de 256 bits y la clave pública es otro número de 256 bits que se obtiene a través de la clave privada mediante la función de multiplicación de curvas elípticas (Walker, 2015b). Aunque exista esta relación, no es posible conocer la clave privada a través de la clave pública ya que esta función cumple la propiedad unidireccional vista para funciones *hash* (Ver Figura 2.5).

- **pk: 01011101...** (*public key*)
- **sk: 11000110...** (*secret key*)

A partir de ahora hablaremos de dirección en lugar de clave pública. La dirección de Bitcoin se genera mediante la aplicación sucesiva de funciones *hash* sobre la clave pública, codificando el resultado final en formato Base58. Este sistema de codificación elimina caracteres visualmente ambiguos para reducir errores de transcripción, ofreciendo una representación compacta y legible de la clave original. De forma análoga, la clave privada suele representarse en formato hexadecimal (64 caracteres) para simplificar su visualización frente a la cadena original de 256 bits.

- **pk: 19zpw6...** (*public key* convertida en dirección)
 - **sk: C64D12...** (*secret key* en formato de 64 caracteres hexadecimales)
-

Es fundamental señalar que ambos procesos son estrictamente unidireccionales debido a las propiedades criptográficas de las funciones *hash*, es computacionalmente imposible revertir una dirección para obtener la clave pública original.

Como se ha comentado en la sección 2.2.3.2, para llevar a cabo una transacción es necesario desbloquear los fondos que se pretenden transferir. Este proceso requiere demostrar la posesión de los activos mediante el conocimiento de la clave privada vinculada a la dirección de origen. Sin embargo, incluir directamente dicha clave en los datos públicos de la transacción comprometería la seguridad del usuario, ya que cualquier participante de la red podría visualizarla y tomar control de los fondos.

Como solución a este problema, se emplean las **firmas digitales**. Estas permiten acreditar que el emisor posee la clave privada asociada a una clave pública específica sin necesidad de revelarla en ningún momento. De este modo, la firma actúa como una prueba criptográfica de autorización que garantiza la integridad y la autenticidad del pago sin exponer la información sensible del propietario. Cuando un usuario quiere firmar una transacción, utiliza su clave privada para generar una firma digital única que depende tanto del mensaje (información de la transacción) como de la clave privada:

$$\text{Signature}(\text{Message}, \text{sk}) = \text{Signature}$$

Al firmar el mensaje con su clave privada, el emisor genera una firma exclusiva e irrepetible para esa transacción específica.

Para verificar la validez de la firma se introduce una función de verificación:

$$\text{Verify}(\text{Message}, \text{Signature}, \text{pk}) = \text{T/F}$$

Esta función simplemente devuelve TRUE si la clave pública corresponde a la clave privada utilizada para generar la firma y FALSE en caso contrario.

Aunque cualquiera conoce las claves públicas nadie más puede crear firmas válidas porque no tienen acceso a su clave privada. Esto garantiza que:

- La transacción fue **realmente autorizada** por el emisor.
- La transacción **no ha sido alterada** después de ser firmada.
- El emisor **no puede negar** haber firmado la transacción.

Aunque podríamos pensar que alguien podría replicar una clave privada, la realidad es que se trata de una secuencia de 256 bits, esto hace un total de 2^{256} posibilidades, un número extremadamente grande, que hace prácticamente imposible identificarla mediante prueba y error (lo conocido como fuerza bruta).

Para evitar que una transacción se replique indefinidamente, lo cual sería posible pues al no cambiar ninguna información continuarían siendo válidas las claves privadas y públicas, lo que se hace es asignar a cada transacción un identificador único e irrepetible.

Para verificar que un individuo realmente dispone de fondos suficientes para asumir todas las transacciones que ha firmado en la red, es necesario mantener un histórico

completo de transacciones. De esta manera, el sistema podrá impedir que se ejecuten transacciones cuando no hay fondos disponibles.

Antes de finalizar esta sección se introducen las siguientes definiciones que serán necesarias para comprender adecuadamente capítulos posteriores.

Definición 2.2.3 *Un **wallet** (en español, cartera) es un software que permite a los usuarios gestionar sus claves criptográficas, actuando como una herramienta de firma que utiliza las claves para autorizar transacciones. Funciona como un puente entre el usuario y sus activos en la red blockchain.*

Definición 2.2.4 *Un **exchange** es un mercado digital donde los usuarios pueden comprar, vender o intercambiar activos. A diferencia de los mercados bursátiles tradicionales, que son entidades centralizadas, estos pueden ser centralizados (CEX), gestionados por una empresa, o descentralizados (DEX).*

2.2.3.4. Árbol de Merkle

Un árbol de Merkle es una manera de organizar y estructurar grandes cantidades de datos para que sean más sencillos de procesar. En el caso de Bitcoin y *blockchain*, el árbol de Merkle se usa para estructurar los datos de las transacciones de manera tal que su procesamiento requiera menos recursos (Merkle, 1980).

Recordamos que una transacción de Bitcoin no es un simple texto que diga “David envía 1 BTC a Raúl”. Es un archivo de datos con una estructura técnica muy estricta. La idea es aplicar HASH-256 a cada una de las transacciones y cada dos hashes creados volver a aplicar HASH-256 formando una estructura de árbol (ver Figura 2.8) que sella por completo todas las transacciones contenidas en el bloque.

2.2.4. Arquitectura de la red

La arquitectura de la red Bitcoin se define como una infraestructura descentralizada de tipo **Peer-to-Peer** o **P2P** (en español, punto a punto), donde todos los participantes mantienen una jerarquía horizontal, eliminando la necesidad de servidores centrales o autoridades intermediarias. El acceso a esta red se consigue mediante la ejecución de una implementación del protocolo Bitcoin (comúnmente denominada “cliente”), la cual transforma al equipo informático en un nodo operativo capaz de interactuar con el ecosistema.

El mantenimiento y la operatividad de la red dependen de la comunicación constante entre estos nodos a través del **Protocolo Bitcoin**, el cual estandariza el intercambio de mensajes y datos. Esta interacción permite la transmisión de datos, como nuevas transacciones, garantizando la sincronización y actualización permanente de todos los integrantes de la red de forma eficiente y en tiempo real.

Como se ha mencionado en la sección 2.2.2, Bitcoin se define por su naturaleza abierta y descentralizada, por lo cual cualquier usuario con acceso a internet puede integrarse en la red únicamente mediante la instalación del software cliente para actuar como un nodo activo.

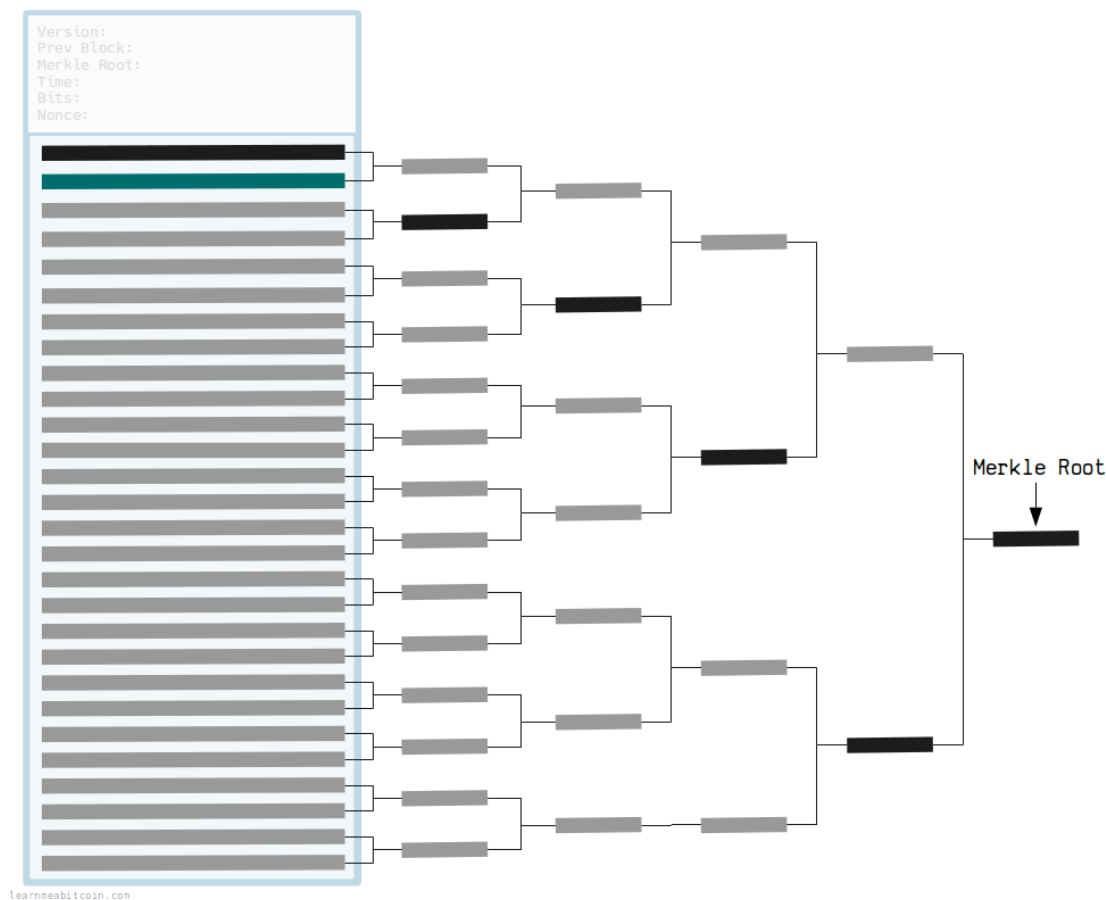


Figura 2.8.: Ejemplo de la estructura de la raíz de Merkle. Tomada de (Walker, 2015c)

2.2.4.1. Nodos y mineros

Un nodo de Bitcoin se define como una unidad de cómputo (cualquier ordenador) que ejecuta el software cliente de la red y mantiene una copia íntegra y actualizada de la cadena de bloques. Su función primordial reside en la interconexión con otros nodos y la verificación de cada transacción y cada bloque recibido según las reglas del protocolo. Este proceso de validación incluye, entre otros, la verificación de firmas criptográficas, la prevención del doble gasto o el cumplimiento del programa de emisión monetaria. Si un minero encuentra un bloque que viola una regla (como crear más bitcoins de los permitidos), los nodos lo rechazarán automáticamente, invalidando el trabajo del minero. En definitiva son los que otorgan soberanía al usuario, ya que no dependen de terceros para validar el estado de sus fondos. El trabajo de los nodos permite crear una infraestructura de red distribuida garantizando el cumplimiento de las reglas de consenso.

Cada nodo de la red Bitcoin dispone de un espacio de almacenamiento temporal en memoria, denominado técnicamente *mempool*, donde se conservan las nuevas transaccio-

nes recibidas antes de ser procesadas. La actividad esencial de un nodo consiste en el intercambio de información, centrada fundamentalmente en dos categorías de transacciones:

- **Transacciones recientes:** Aquellas operaciones que acaban de ser emitidas y se propagan por la red de manera individual.
- **Transacciones confirmadas:** Operaciones que han sido validadas e integradas en bloques que ya han sido añadidos a la cadena. A diferencia de las anteriores, estas se comparten de forma agrupada al compartirse todo el bloque.

El nodo es responsable del almacenamiento de estos bloques confirmados, cada transacción recibida es verificada de forma autónoma y solo se retransmite al resto de la red si cumple estrictamente con los criterios de integridad del protocolo. En caso de detectar irregularidades, la transacción es descartada, garantizando así la seguridad del sistema.

Mientras que un nodo convencional se limita a validar y propagar información, un **minero** representa una categoría especializada de nodo cuya función primordial es la ordenación cronológica de transacciones y la proposición de nuevos bloques. El trabajo del minero se conoce como **minería de bloques** y consiste en la resolución de un problema criptográfico de carácter aleatorio.

1. Un minero selecciona un conjunto de transacciones de su *mempool* y las agrupa junto a un valor variable denominado *nonce* y a otros campos.
2. A este grupo le aplica una función HASH-256, que producirá un resultado como los vistos en la sección 2.2.3.1.
3. El objetivo es variar el *nonce* hasta obtener un resultado que cumpla con un requisito específico: el *hash* resultante debe ser inferior a un umbral determinado, lo que visualmente se traduce en un número concreto de ceros iniciales.

A lo largo de las siguientes secciones se explicará en mayor detalle el proceso de minería pero conviene tener una idea previa de este mecanismo de prueba y error que garantiza la seguridad y la inmutabilidad de la red.

2.2.5. Bloques

Una vez conocida la idea, podemos ver en detalle cómo funciona una cadena de bloques. Para ello, se describe a continuación la estructura que presenta un bloque individual formado por un minero.

Un bloque se define fundamentalmente como una unidad de datos que actúa como contenedor de un conjunto de transacciones. Estructuralmente, cada bloque se compone de un listado de transacciones (el cuerpo del bloque) y un encabezado, denominado *block header*. Este encabezado cumple una función crítica pues un aspecto clave es que los mineros aplican la función *hash* a este encabezado de bloque para intentar minarlo y no al propio bloque completo, por lo que debe contener toda la información esencial.

- **Cabecera del bloque (Block Header):**

- Versión
- *Hash* del bloque anterior
- Raíz de Merkle
- Marca de tiempo (*Timestamp*)
- Dificultad
- Nonce

- **Transacciones:**

- Contador de transacciones
- *Coinbase Transaction*
- Transacciones regulares

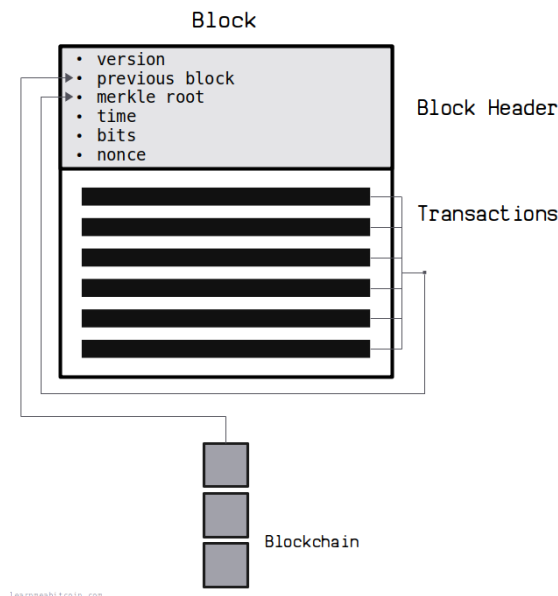


Figura 2.9.: Diagrama de la estructura de un bloque. Tomada de (Walker, 2015a).

Cabecera del bloque (Block Header): Ocupa unos 80 bytes y contiene un resumen de todo el contenido del bloque.

- **Versión:** Es un metadato de 4 bytes que se utiliza para indicar actualizaciones de la red. Originalmente era un simple entero que marcaba una actualización de la estructura del bloque tras un *soft fork* (sección 2.2.6.5), pero ahora los mineros lo utilizan para votar por actualizaciones del software (Walker, 2015a).

- **Hash del bloque anterior:** Este campo contiene el *hash* del último bloque añadido a la cadena. Todos los mineros buscan extender la cadena de bloques más larga puesto que es la que todos los nodos adoptan como la cadena de bloques “correcta”. Si construyéramos sobre un bloque más abajo en la cadena, nuestro bloque no formaría parte de la cadena más larga, por lo que no sería añadido a la *blockchain* y nuestros esfuerzos por minar el bloque serían en vano. Si este *hash* no estuviera incorporado en la cabecera del bloque no existiría la estructura de cadena pues no habría ningún elemento enlazando los bloques entre sí. De esta manera se puede dividir el registro contable en bloques ordenados.
- **Raíz de Merkle (Merkle Root):** Este campo contiene una huella digital para todos los datos de las transacciones incluidas en el bloque. Esta huella digital se obtiene siguiendo el procedimiento descrito en la sección 2.2.3.4.

Para ser más eficiente la red Bitcoin no mina transacciones, mina cabeceras de bloques (que solo ocupan 80 bytes) por lo tanto si la raíz de Merkle no estuviera en la cabecera, un minero podría cambiar una transacción en el cuerpo del bloque sin que el *hash* del bloque cambiara. Al poner la raíz en la cabecera, se encadena el contenido al sello de seguridad. Si se modifica un solo céntimo en una transacción, la raíz cambia, la cabecera cambia y el bloque entero se vuelve inválido instantáneamente.
- **La Marca de tiempo (Timestamp):** Es un registro de 4 bytes que establece el momento aproximado de creación del bloque, expresado en segundos desde la época Unix. Para determinar un instante preciso, el sistema Unix utiliza como referencia la cantidad de segundos transcurridos desde su época de origen, fijada el 1 de enero de 1970. Es un sistema de conteo lineal que ignora las correcciones por segundos intercalares.
- **La dificultad o Bits:** Es una representación compacta del objetivo en el momento de la minería del bloque. Cada bloque debe estar por debajo de un valor objetivo específico para que se considere válido (para que se agregue a la cadena de bloques). Sin embargo, en lugar de almacenar el valor objetivo completo de 32 bytes en la cabecera del bloque, utilizamos una codificación compacta, *bits* de 4 bytes (Walker, 2015a).
- **Nonce:** Este campo es la abreviatura de “*number used once*” (número usado una vez). Se trata simplemente de un campo libre en el encabezado del bloque cuyo valor varía para obtener diferentes resultados a la hora de aplicar *hash* a toda la cabecera.

Cada bloque de la cadena de bloques muestra el valor *nonce* que produjo un *hash* en el bloque anterior que cumplía el objetivo. Encontrar el *nonce* correcto no requiere habilidad, simplemente se trata de probar diferentes valores lo más rápido posible y confiar en tener suerte. Actualmente, la red de bitcoin es capaz de probar (tiene un *hashrate*) de más de 1000 millones de TH/s (Terahashes/segundo) (<https://www.blockchain.com/explorer/charts/hash-rate>).

Transacciones: Con el lanzamiento, esta parte estaba limitada a 1 MB. Sin embargo, tras la actualización *Segregated Witness* (SegWit) (Lombrozo et al., 2015), el límite máximo se establece en 4 millones de Unidades de Peso (WU), lo que permite que el tamaño de los bloques supere el límite original, alcanzando habitualmente entre 1.5 MB y 2 MB de información real en disco.

- **Contador de transacciones:** El primer dato después del encabezado del bloque es un recuento de transacciones que indica el número de transacciones futuras en el bloque. Es un campo compacto, por lo que suele tener un tamaño de 1 o 3 bytes (dependiendo del número de transacciones en el bloque) (Walker, 2015a).
- **Coinbase Transaction:** La primera transacción de cada bloque es la transacción *Coinbase*. Se trata de una transacción especial que los mineros colocan dentro del bloque para obtener la recompensa de haberlo minado (subsidio del bloque más las comisiones de transacción), se explica más detalladamente el funcionamiento de estas recompensas en la sección 2.2.6.1. Es importante tener en cuenta que esta transacción es un requisito para cada bloque, sin ella el bloque sería inválido.
- **Transacciones regulares:** Tras la transacción *coinbase*, tenemos todas las transacciones “normales”. Estas operaciones son seleccionadas por el minero directamente desde el *mempool* (área de espera para nuevas transacciones) durante la fase de construcción del bloque. Si bien el minero dispone de autonomía para determinar el volumen de transacciones a incluir (dentro de los límites de tamaño impuestos por el protocolo), el incentivo económico ofrecido por el sistema contribuye a maximizar dicha ocupación. Al incluir un mayor número de transacciones, el minero incrementa sus beneficios en forma de comisiones de red (*fees*).

2.2.6. Consenso e integridad de la red

2.2.6.1. Algoritmo de consenso: *Proof-of-Work* (PoW)

Como hemos mencionado previamente en 1994, Adam Back desarrolló el concepto de prueba de trabajo (PoW) (Back, 2002) que posteriormente en 2009 sería implementado en Bitcoin. La prueba de trabajo no es más que la resolución de un problema criptográfico de carácter aleatorio. Un nodo selecciona un conjunto de transacciones de su *mempool* y siguiendo el procedimiento de almacenamiento en bloques visto en la sección 2.2.5 a la cabecera del bloque se le aplica una función HASH-256. El objetivo es probar distintos valores *nonce* en la cabecera hasta obtener un resultado que cumpla con un requisito específico: el *hash* resultante debe ser inferior a un umbral determinado, lo que visualmente se traduce en un número concreto de ceros iniciales. Como se ha visto a lo largo del trabajo las funciones *hash* son impredecibles, por lo que encontrar el *nonce* correcto es una cuestión de prueba y error, que es lo que garantiza la seguridad y la inmutabilidad de la red.

La dificultad de este desafío se ajusta dinámicamente en función de la potencia de cómputo global (*hashrate*) conectada a la red. Este ajuste asegura que el tiempo promedio de generación de un bloque se mantenga constante, aproximadamente cada diez minutos.

Para asegurar la participación de los nodos en la validación de la red, el protocolo establece un sistema de incentivos económicos. Cada bloque nuevo incluye una transacción especial denominada *Coinbase Transaction* que hemos visto en la sección 2.2.5. Esta es la primera transacción de cada bloque y es la que otorga al minero una recompensa por su trabajo. Esta recompensa se compone de dos elementos:

- **Subsidio de bloque:** La creación de nuevas unidades de bitcoin. De acuerdo con el diseño deflacionario de Satoshi Nakamoto, el subsidio de bloque no es constante. Se rige por un proceso conocido como *Halving*, la emisión de nuevos bitcoins se reduce a la mitad cada 210,000 bloques (aproximadamente cada cuatro años). Tras el lanzamiento de la red por cada bloque minado se emitían 50 BTC nuevos y actualmente (2026) esta cifra es de 3.125 BTC, este mecanismo garantiza un límite asintótico en el suministro total de 21 millones de unidades.
- **Comisiones de red:** Cada transacción lleva asociada unas comisiones que se llevará el minero que incluya la transacción en un bloque válido, de esta manera cuanto más altas sean las comisiones más incentivados estarán los mineros a validar una transacción. Esta parte de la recompensa recoge la suma de todas las comisiones de las transacciones incorporadas en el bloque.

En otras *Blockchains* se emplean otros protocolos de consenso existentes como *Proof of Stake (PoS)*, *Proof of Importance (PoI)*, *Delegated Proof-of-Stake (DPoS)* ... Veremos uno de ellos en la sección 2.3.6.

2.2.6.2. Reorganización de la red

Dada la naturaleza descentralizada de la red y la latencia en la propagación de datos, es posible que dos mineros encuentren un bloque válido de manera casi simultánea. Este fenómeno genera una bifurcación accidental o *fork*, donde diferentes nodos de la red reciben bloques distintos pero igualmente válidos, provocando un desacuerdo temporal sobre cuál es el último eslabón real de la cadena.

Para resolver este conflicto, Bitcoin sigue la **Regla de la Cadena con Mayor Trabajo Acumulado**, comúnmente simplificada como la “cadena más larga”. Técnicamente el sistema prioriza la rama que demuestre haber requerido una mayor cantidad de esfuerzo computacional para ser construida. El protocolo dicta que:

- **Mantenimiento de ramas paralelas:** Ante una bifurcación, los nodos almacenan ambas ramas en su memoria de manera temporal, manteniendo una como principal y la otra como reserva.
 - **Diferenciación por trabajo:** El conflicto se resuelve en el momento en que se mina un nuevo bloque sobre una de las dos ramas. Al añadirse este nuevo eslabón, la rama elegida se convierte automáticamente en la cadena con mayor trabajo acumulado.
-

- **Mecanismo de Reorganización (*Reorg*):** Cuando los nodos reciben el bloque que prolonga la cadena alternativa, se activa una reorganización. El nodo desactiva los bloques de su antigua cadena en favor de los nuevos. Los bloques que quedan fuera de la cadena principal se denominan *stale blocks* (o bloques huérfanos) y sus transacciones, si no han sido incluidas en la rama ganadora, vuelven al *mempool* para ser procesadas nuevamente.

2.2.6.3. Ataque del 51 %

Un ataque del 51 % hace referencia a construir intencionadamente una nueva cadena de bloques más larga para reemplazar los bloques ya existentes en la *blockchain* original. Este tipo de ataque es más fácil de ejecutar cuando se posee la mayoría de la potencia de minería, por lo que se le conoce como “ataque mayoritario” o “ataque del 51 %”. Como se ha visto, los nodos de la red siempre aceptan la cadena de bloques más larga (con mayor trabajo computacional), por lo tanto, si quisiéramos deshacer una transacción de la cadena de bloques, solo necesitaríamos construir una nueva cadena de bloques más larga sin esa transacción. Al construir una nueva cadena más larga para reemplazar una existente, se reescribiría la cadena de bloques y se crearía un nuevo historial de transacciones que cada nodo de la red adoptaría. Como resultado, se revertirían transacciones que antes se consideraban permanentes en la cadena de bloques. Hay que tener en cuenta que cada minero tiene incentivos para construir sobre la cadena de bloques más larga, por lo tanto, si la potencia minera combinada de todos los demás mineros de la red es mayor que la del atacante, será increíblemente difícil superar a los demás mineros para construir una cadena más larga y reemplazar la existente (Nakamoto, 2008).

Si se plantea de forma técnica, realizar el ataque del 51 % es posible, pero cuando se pone en práctica, es totalmente inviable. Para que el atacante logre adquirir más potencia minera que todos los demás mineros juntos, debería superar el *hashrate* de la red, que actualmente en 2026 es de más de 1000 millones de TH/s (Terahashes/segundo) (<https://www.blockchain.com/explorer/charts/hash-rate>). Siendo muy generosos, asumiendo que un equipo medio aporta un *hashrate* de 100 MH/s, el atacante necesitaría 1000 billones (millones de millones) de equipos para realizar el ataque, lo que supone una cantidad de dinero astronómica y una cantidad de equipos imposible de fabricar. Suponiendo que se consigue, hay que tener en cuenta que los mineros invertirán también más dinero en defender la cadena principal por lo que la inversión del atacante aumentará según el tiempo que tarde en reunir tal cantidad de equipos. De cualquier modo, de lograr el ataque a la red, el resto de nodos no haría absolutamente nada pues no tendrían ningún interés en actualizar la cadena, mantendrían la cadena original donde sus Bitcoins tienen valor y todo el esfuerzo del atacante por acabar con la red no habría servido para nada (Caballero, 2019).

2.2.6.4. Hard Fork

Un *hard fork* se define como una actualización del software de la red que introduce cambios en las reglas de consenso incompatibles con versiones anteriores. Este fenómeno

ocurre cuando se modifican los parámetros del protocolo para que bloques o transacciones que anteriormente se consideraban inválidos pasen a ser válidos bajo la nueva normativa.

Para que un *hard fork* del software tenga éxito, es necesario que todos los nodos de la red acepten los cambios y actualicen su software. Si una parte de la red adopta el nuevo software y otra mantiene la versión original, se produce una división permanente del registro contable en dos direcciones divergentes. Dado que los nodos antiguos no poseen la capacidad técnica para validar los nuevos bloques, ambas cadenas coexisten de forma paralela e independiente, dando lugar a menudo a la creación de una nueva criptomoneda (como ocurrió en el caso histórico de Bitcoin Cash) (Walker, 2025a).

Para ilustrar este concepto, se puede considerar un cambio en el suministro máximo de Bitcoin. Si la regla vigente establece un máximo de 21 millones de BTC y una actualización propone ampliar dicho límite a 50 millones, la bifurcación real en la *blockchain* se produce en el instante en que se mina el primer bloque con un nuevo sistema de recompensas que garantice la producción de 50 millones. Por un lado los mineros actualizados comenzarán a construir sobre este nuevo bloque y los mineros y nodos antiguos rechazarán dicho bloque por infringir las reglas previas y continuarán extendiendo la cadena actual.

Cualquier alteración que vulnere las reglas de consenso existentes derivará necesariamente en un *hard fork*. Entre los cambios más significativos se incluyen:

- **Modificaciones en la estructura de bloques:** Incrementos en la capacidad de almacenamiento de datos como puede ser pasar de bloques de 1 MB a otros tamaños.
- **Alteraciones en la criptografía:** Cambios en los algoritmos de firma digital.
- **Cambios en la política monetaria:** Aumentar el suministro máximo de 21 millones de unidades o ajustar manualmente el objetivo de dificultad.

2.2.6.5. *Soft Fork*

Un *soft fork* representa una actualización del protocolo que es compatible con las versiones anteriores del software. A diferencia de un *hard fork*, que expande el conjunto de reglas, un *soft fork* se caracteriza por restringir las reglas de consenso, invalidando tipos de transacciones o bloques que anteriormente eran aceptados. El principio fundamental es que los nuevos bloques, al cumplir con reglas más estrictas, siguen siendo válidos para los nodos que no han actualizado su software. Por lo tanto, los nodos antiguos pueden seguir aceptando los bloques generados por los nodos actualizados y mantenerse sincronizados con la cadena de bloques principal sin necesidad de modificar su código.

Para ilustrar este concepto, se puede considerar un escenario hipotético sobre el tamaño del bloque. Si la regla vigente permite bloques de hasta 1 MB, un *soft fork* podría restringir el límite a 0,5 MB. Los bloques de 0,5 MB son perfectamente válidos según la regla antigua (que acepta cualquier valor ≤ 1 MB). Por consiguiente, los nodos que no han actualizado verán estos bloques más pequeños como legítimos y los añadirán a su registro sin interrupción (Walker, 2025b).

El éxito de un *soft fork* depende de que la mayoría del *hashrate* de la red adopte el nuevo software. Bajo la **Regla de la Cadena con Mayor Trabajo Acumulado**, si la mayoría de los mineros migran a la nueva versión, construirán la cadena con mayor trabajo utilizando las reglas restringidas. Los nodos antiguos, al seguir siempre la cadena con más trabajo, adoptarán la cadena actualizada de forma natural. Aunque una minoría de mineros antiguos intente seguir las reglas anteriores, no podrán competir en velocidad y sus bloques serán eventualmente desplazados por una reorganización de la cadena.

El principal riesgo de un *soft fork* es que la mayoría de los mineros no actualice. La red correría el riesgo de dividirse pues los nodos actualizados rechazarían los bloques de los antiguos mineros mientras que los nodos antiguos aceptarían ambos. Sin embargo, a diferencia de un *hard fork*, donde las dos cadenas nunca pueden converger, esta división de la cadena se puede resolver añadiendo más potencia de minería a la nueva cadena. Si los mineros actualizados logran construir la cadena más larga, los nodos antiguos realizarán una reorganización de la cadena para adoptar la *blockchain* compuesta por los nuevos bloques.

A pesar de ser el método preferido para actualizar Bitcoin, ya que no obliga a todos los usuarios a una actualización inmediata y reduce el riesgo de fragmentación permanente, los *soft forks* presentan una desventaja significativa como es el incremento de la complejidad técnica. Debido a la necesidad de mantener la compatibilidad con nodos antiguos, los desarrolladores deben implementar soluciones más restringidas, lo que hace que el software sea más complejo que si se tuviera la libertad de realizar cambios incompatibles con los nodos antiguos.

2.3. Ethereum

Hasta ahora se ha visto el funcionamiento de la primera y más importante *blockchain* hasta la fecha. A diferencia del modelo de transferencia de valor puro que define a Bitcoin, Ethereum es una *blockchain* que nace con la idea de utilizar esta tecnología para ejecutar lógica programable de manera que se garantice la ejecución de acuerdos sin intermediarios. Su innovación reside en la implementación de la *Ethereum Virtual Machine* (EVM), se trata de una capa de abstracción que convierte a la *blockchain* en una máquina de estados global y Turing-completa (capaz de realizar cualquier cálculo o programa posible). Bajo este paradigma, los nodos de la red no se limitan a la validación de transacciones, sino que actúan como unidades de procesamiento de código. Este concepto se materializa a través de los *smart contracts*, protocolos autónomos que garantizan que acuerdos se ejecuten inmutablemente sin intermediarios, facilitando así el despliegue de activos digitales complejos y aplicaciones descentralizadas (dApps) con disponibilidad global (Buterin, 2014).

2.3.1. Nacimiento de Ethereum

El 31 de enero de 1994 nació en Kolomna, Rusia, Vitalik Buterin, cofundador y creador de Ethereum. Vitalik emigró a Canadá junto a su familia en el año 2000 y desde peque-

ño destacó en la escuela, fue ubicado en una clase para niños especialmente dotados, y mostraba grandes capacidades en matemáticas, programación y economía. Cuando tenía 17 años, su padre, que de profesión era informático, le enseñó Bitcoin, ya que este quería involucrarse en la red, pero no tenía capacidad para minar bitcoins ni dinero para comprarlos. A Vitalik le fascinó la idea ya que, como desveló posteriormente, fue un jugador activo de World of Warcraft entre 2007 y 2010, pero abandonó el juego cuando Blizzard, la compañía dueña del mismo, realizó un cambio en uno de sus personajes favoritos, que le hizo darse cuenta de los peligros de la centralización (Caballero, 2019).

A partir de entonces Vitalik se adentró en el mundo de las criptomonedas. Contactó a través del chat de un foro sobre Bitcoin con una persona que quería comenzar un blog sobre noticias de la moneda. Vitalik escribió para el blog cobrando 5 bitcoins (alrededor de 3,5 dólares) por artículo, hasta que poco después el blog cerró por la falta de interés de los medios (Bitcorati Inc., 2014).

Mientras estaba en la universidad, Vitalik dedicaba la mayor parte de su tiempo a las criptomonedas, por lo que decidió abandonar los estudios y pasar seis meses viajando por el mundo, conociendo a otras personas del mundo cripto y participando en comunidades de Bitcoin. Durante esta etapa descubrió el posible uso de la tecnología *blockchain* más allá de una moneda digital.

En noviembre de 2013, Vitalik Buterin publicó el *whitepaper* (Buterin, 2014) que fundaba Ethereum iniciando su fase de desarrollo formal en diciembre de ese mismo año junto a un equipo multidisciplinar de cofundadores. El 30 de julio de 2015 se lanzó una primera versión de la *Blockchain* de Ethereum, “Frontier”, que permitía a los usuarios y mineros experimentar con los **contratos inteligentes**. Posteriormente, en marzo de 2016, la red transitó hacia su segunda fase principal, denominada “Homestead”, lo que supuso una mayor estabilidad del protocolo y la apertura formal del comercio de Ether (ETH), la divisa utilizada en la red (Ethereum Foundation, 2025d).

2.3.2. *Smart Contracts*

El concepto de *Smart Contract*, propuesto originalmente por Nick Szabo en 1996 (Szabo, 1996) como un mecanismo para digitalizar acuerdos contractuales, encuentra su máxima expresión en la red Ethereum. Estos no son más que programas informáticos que logran que los términos de un acuerdo se cumplan de manera autónoma. De esta manera cualquier participante del mismo puede conocer inmediatamente el resultado prescindiendo por completo de la validación por parte de terceros. Además, permiten automatizar flujos de trabajo y activar una acción cuando se cumplen unas condiciones establecidas.

El número de condiciones de un *smart contract* no tiene límite. Como mínimo, la configuración exige determinar cómo se van a codificar las transacciones, el acuerdo de sus reglas y la evaluación de las excepciones o “vacíos legales”. Asimismo, debe quedar establecido un marco normativo para la resolución de disputas entre las partes.

Para ilustrar cómo los *smart contracts* transforman una interacción cotidiana en un proceso autónomo y programable, utilizaremos el ejemplo de Miguel (inquilino) y Daniel (propietario). Se puede observar gráficamente este ejemplo en la Figura 2.10.

1. Antes de activar el contrato, Miguel y Daniel deben definir cómo se traduce su acuerdo en código. Ambos utilizan sus direcciones de Ethereum (claves públicas) como firma legal, el pago se define en Ether (la criptomoneda de Ethereum) y el contrato se conecta a **Oracles** externos (servicios que proveen datos externos).
2. Las reglas podrían codificarse de la siguiente manera: SI (Fecha == Día 1 del mes) Y (Saldo en cuenta de Miguel $\geq$ 500€), ENTONCES transferir 500€ a la cuenta de Daniel.
3. Un *smart contract* robusto debe prever qué ocurre si algo falla. En el contrato se incluyen variables cómo: ¿qué pasaría si en la cuenta de Miguel no hay fondos? Por ejemplo, si el día 1 Miguel no tiene saldo, el contrato envía una notificación automática a su móvil y reintenta el cobro el día 2 con un recargo del 5 % (penalización por demora). También se podrían añadir cláusulas: Miguel puede retener el 15 % del alquiler si un sensor inteligente (IoT) detecta que la calefacción no ha funcionado durante más de 48 horas en invierno.
4. Al inicio del contrato, Miguel deposita una fianza en el *smart contract*, si al finalizar el mes hay una disputa sobre daños en la vivienda, los fondos quedan bloqueados hasta que una tercera clave autorice la liberación hacia Daniel o la devolución a Miguel.

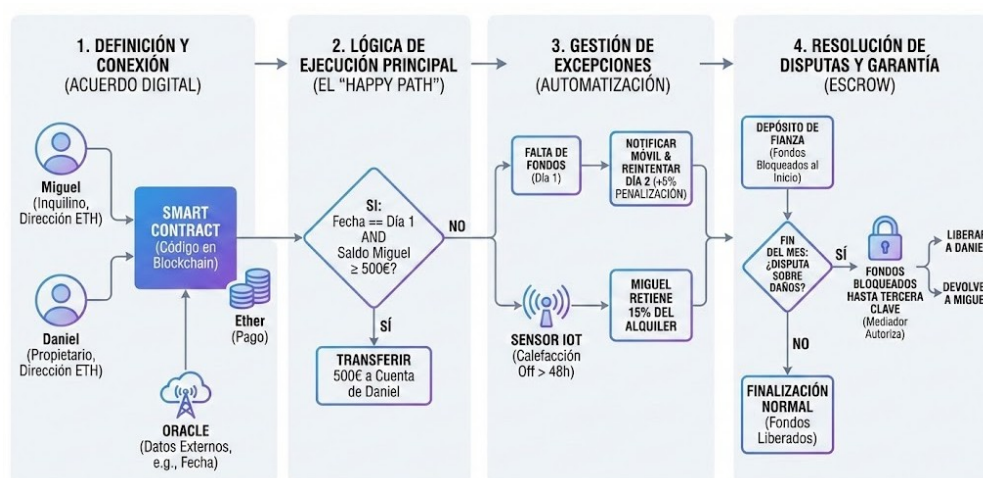


Figura 2.10.: Diagrama de ejemplo de un *smart contract*. De elaboración propia.

Los *smart contracts* tienen grandes beneficios, al ser digitales y automatizados eliminan la necesidad de tramitar papeleo y dedicar tiempo a corregir los errores que se pueden producir al rellenar documentos de forma manual. Dado que no hay terceros involucrados, no es necesario comprobar si la información ha sido modificada para beneficiar a alguna de las partes, aumentando así la confianza y la transparencia. Además debemos recordar que todo se escribe en una *blockchain* pública, por lo que cualquier persona puede realizar

un seguimiento instantáneo, y se beneficia de la seguridad de la red que hemos visto en la sección 2.2.6.

Un *smart contract* se escribe en el lenguaje de programación **Solidity**, y luego se traduce a **bytecode**, que es el lenguaje que realmente entiende la **Ethereum Virtual Machine (EVM)**, que veremos en la sección 2.3.4. A continuación se muestra la Tabla 2.1 donde podemos ver las diferencias entre un contrato tradicional y un contrato inteligente.

Característica	Contrato Tradicional	<i>Smart Contract</i>
Lenguaje	Natural (Legal, a veces ambiguo)	Código (Lógico, determinista)
Ejecución	Manual o mediante terceros	Autoejecutable (Algorítmico)
Intermediarios	Notarios, jueces, abogados	Protocolo <i>Blockchain</i> (Consenso)
Modificabilidad	Por acuerdo mutuo entre partes	Inmutable tras su despliegue
Coste	Elevado (Tasas e intermediarios)	Reducido (Comisiones o Gas)
Cumplimiento	Basado en la buena fe o ley	Garantizado por criptografía
Resolución	Vía judicial o arbitraje	Resolución preprogramada

Tabla 2.1.: Comparativa entre el Contrato Tradicional y el *Smart Contract*

Se dan ahora las condiciones para introducir la siguiente definición:

Definición 2.3.1 *Una aplicación descentralizada (dApp) es un programa informático que se ejecuta sobre una blockchain o una red distribuida. A diferencia de las aplicaciones tradicionales, utilizan smart contracts para operar de forma autónoma, eliminando intermediarios y permitiendo una mayor seguridad, resistencia a la censura y transparencia.*

2.3.3. El concepto de Gas y el incentivo de cómputo

Como mecanismo de seguridad para evitar los bucles infinitos y que un código se ejecute infinitamente provocando un colapso de la red se introduce el concepto de **Gas**. No se trata de dinero en sí mismo, sino una forma de medir cuánto esfuerzo le toma a la red procesar una solicitud. Sumar dos números es sencillo (cuesta poco Gas), pero guardar información nueva en la base de datos de la *blockchain* es “caro” (cuesta mucho Gas). Cada *byte* de información que se envía en una transacción cuesta 5 unidades de Gas para evitar que los usuarios llenen la red con código innecesario (Buterin, 2014).

Además del destinatario, una firma identificadora del remitente y el Ether por transferir de uno a otro, las transacciones de Ethereum contienen un campo de datos opcional y dos campos **Startgas** y **Gasprice** relativos al control de la transacción mediante

gas. El *startgas* (Límite de Gas) es el máximo número de pasos computacionales que se permite realizar a la transacción. Una buena analogía sería echar al depósito del coche combustible para 100 kilómetros, si no llega al destino con esta cantidad, se detendrá. El *gasprice* (Precio del Gas) indica cuánto se está dispuesto a pagar en Ether por cada unidad de gas. El total de gas que se paga va directamente a quien procesa la transacción, si es compleja y utiliza muchos recursos de computación (CPU), ancho de banda (red) y almacenamiento (disco duro); se paga más, recompensando de forma justa a quien mantiene la red (Buterin, 2014).

Una vez se han visto cómo funcionan los *smart contracts* y el Gas, estamos en condiciones de describir el funcionamiento de la **Ethereum Virtual Machine (EVM)**.

2.3.4. **Ethereum Virtual Machine**

Hemos utilizado el concepto de libro contable distribuido para explicar la *blockchain* de Bitcoin. Este es un ejemplo perfecto si estamos hablando de una *blockchain* que almacena únicamente transacciones. Sin embargo, aunque Ethereum tenga su propia moneda (Ether), la característica que la diferencia de otras *blockchains* como Bitcoin es el despliegue de *smart contracts*. La *Ethereum Virtual Machine* es un ecosistema virtual descentralizado que ejecuta estos *smart contracts* de forma consistente y segura en todos los nodos encargados de desplegar la *blockchain*.

Para explicar la arquitectura de Ethereum, es preciso abandonar la analogía del libro contable y adoptar la de máquina de estado distribuida. El estado de la red es una gran estructura de datos, que no solo sostiene todas las cuentas y saldos, sino que también incluye el **estado de la máquina**. La transición de este estado se produce de forma secuencial en cada bloque según un conjunto de reglas establecidas por la EVM.

Para entender cómo funciona Ethereum cuando procesa una transacción o ejecuta un contrato inteligente se puede observar la Figura 2.11.

El **Estado Global de Ethereum** es la base de datos que contiene todas las cuentas, los balances y el código de los contratos. Se indica que es “persistente” ya que se guarda en la *blockchain* para siempre o hasta ser modificada en el siguiente bloque. Como se ve en la Figura 2.11 el estado global se divide en tres partes:

- **Estado de la máquina:** Entorno de ejecución de una transacción específica:
 - **Estado de la máquina (volátil):** Mientras el código se ejecuta se tienen en cuenta los siguientes factores:
 - **Program Counter (PC):** Indica qué instrucción de código debe ejecutar la máquina a continuación.
 - **Gas disponible:** El gas restante, si llega a cero antes de terminar, la ejecución se detiene y se revierten los cambios.
 - **Stack (Pila):** Es una estructura tipo **LIFO** (lo último en entrar es lo primero en salir) donde la EVM realiza los cálculos. Tiene un límite de 1024 elementos de 256 bits cada uno.

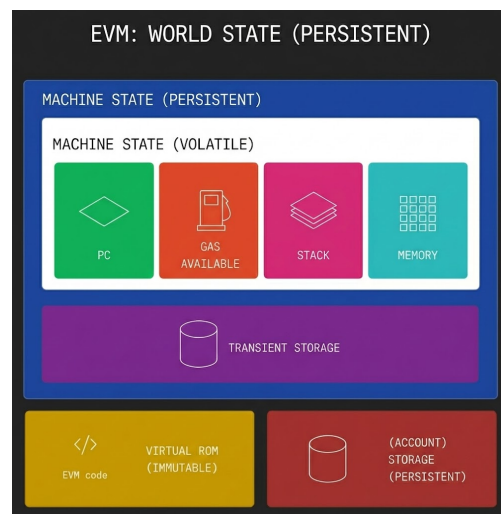


Figura 2.11.: Diagrama del funcionamiento de Ethereum. Adaptado de (Ethereum Foundation, 2025c).

- **Memoria:** Un espacio de almacenamiento temporal y volátil (equivalente a una memoria RAM). Se usa para pasar datos entre funciones, es más económica que el almacenamiento en disco, pero se borra al terminar la ejecución.
- **Almacenamiento transitorio:** Es una adición reciente que permite guardar datos que persisten durante toda la transacción (incluso si hay varias llamadas entre contratos), pero que no se almacenan finalmente en la *blockchain*.
- **Virtual ROM (Immutable):** Aquí es donde se aloja el EVM Code (*bytecode* del contrato). Se considera una “ROM” porque el código de un contrato inteligente es inmutable, una vez desplegado no se pueden cambiar sus instrucciones.
- **Account storage (Persistent):** Se trata del “disco duro” del contrato. Aquí es donde se guardan los datos permanentes como el saldo de cada usuario. Es la parte más cara de usar en términos de gas porque requiere que todos los nodos de la red guarden esa información para siempre.

La EVM tiene grandes ventajas como la programabilidad, que nos permite ir mucho más allá de los pagos simples y diseñar aplicaciones complejas y personalizadas que se ajustan a cualquier necesidad. La interoperabilidad es una característica clave que permite que diferentes *smart contracts* interactúen entre sí, y a todo esto hay que sumar la seguridad que ofrece la propia naturaleza de la *blockchain*, que garantiza que cada movimiento sea transparente, rastreable y prácticamente imposible de manipular.

2.3.5. Modelo de cuentas: EOA vs Contratos

En contraste con el esquema UTXO de Bitcoin detallado en la sección 2.2.3.2, la red Ethereum apuesta por una arquitectura de cuentas. En este sistema, cada cuenta se identifica mediante una dirección de 20 bytes y opera a través de transiciones de estado, las cuales consisten en el intercambio directo de activos y metadatos. Estructuralmente, una cuenta de Ethereum se compone de los siguientes cuatro elementos:

- **Nonce:** Es un contador, indica cuántas transacciones se han enviado desde la cuenta impidiendo que alguien procese la misma transferencia dos veces.
- **Saldo (Balance):** La cantidad de Ether (ETH) que posee la cuenta.
- **Código de contrato:** Las instrucciones programadas. Si es una cuenta personal, este espacio está vacío.
- **Almacenamiento (Storage):** Es la memoria a largo plazo de la cuenta, donde se guardan datos permanentes .

2.3.5.1. Cuentas de Propiedad Externa (EOA)

Una cuenta de propiedad externa, en inglés *Externally Owned Account*, es una cuenta gestionada por una persona con una clave privada, que no posee código programable y desde la cual se pueden iniciar transacciones.

2.3.5.2. Contract Accounts

A diferencia de las cuentas de usuario, las de contrato albergan código ejecutable que se ejecuta con la recepción de una instrucción o mensaje. Esta activación les otorga autonomía para gestionar su propio almacenamiento de datos, emitir comunicaciones hacia otras cuentas o, incluso, instanciar nuevos contratos dentro de la red.

2.3.6. The Merge: Del Proof-of-Work al Proof-of-Stake

Como bien se ha explicado en la sección 2.2.6.1, bitcoin implementa la *Proof of Work* (PoW) como mecanismo de consenso. Ethereum nació utilizando este sistema, pero en su código incluyó la famosa *Difficulty Bomb* (bomba de dificultad), un mecanismo diseñado para aumentar exponencialmente en el tiempo la dificultad de minado de bloques, forzando así a una transición a un nuevo mecanismo de consenso, la *Proof of Stake* (PoS), en español “prueba de participación”, que fue ideada por Vitalik Buterin y Virgil Griffith en 2017 con el protocolo **Casper** (Buterin & Griffith, 2017). La PoS no se implementó inmediatamente tras su planteamiento porque la red ya era demasiado grande y realizar un cambio de estas dimensiones requería años de pruebas de seguridad.

En 2020 Vitalik Buterin y un grupo de investigadores publicaron el artículo “Combining GHOST and Casper” (Buterin et al., 2020) en el que presentaban **Gasper**, un protocolo de consenso basado en prueba de participación (PoS). El protocolo combinaba

Casper con LMD GHOST —una regla de elección de bifurcación— y demostró seguridad y viabilidad para la incorporación a Ethereum.

Con el objetivo de mostrar que la prueba de participación era completamente segura y además, más sostenible, el 1 de diciembre de 2020 se lanzó la *Beacon Chain*. Al tratarse de una *blockchain* alternativa se podían hacer comprobaciones sin habilitarla en la *mainnet* de Ethereum. El 15 de septiembre de 2022 se produjo *The Merge*, en español, “La fusión”. La implementación de la *Beacon Chain* supuso el fin de la minería como método para validar bloques, transfiriendo ese poder a los nodos validadores del sistema PoS. A partir de este punto, estos nodos asumieron la tarea de proponer bloques y verificar cada transacción. A pesar de la magnitud de migrar de un modelo de prueba de trabajo a uno de participación, la integridad de la cadena de bloques desde su bloque génesis se mantuvo intacta, sin aparentemente cambiar nada para los usuarios que interactúan con la red.

2.3.6.1. Proof of Stake

Con la implementación de la Prueba de Participación en Ethereum, los validadores han reemplazado por completo a los mineros. Estos nuevos protagonistas deben comprometer sus fondos en Ether dentro de un contrato inteligente específico para asegurar la red. El protocolo establece un umbral de entrada de 32 ETH y la gestión simultánea de tres herramientas de software: los clientes de ejecución, consenso y validación. Además, para preservar la integridad del sistema ante variaciones bruscas en el número de nodos, los aspirantes pasan por una fase de espera controlada que regula la incorporación de nuevos validadores, haciendo así que el ingreso no sea inmediato. Una vez habilitados, los validadores reciben nuevos bloques enviados por otros validadores. Las transacciones de ese bloque se procesan de nuevo para comprobar que el validador que envió el bloque no cometió ninguna irregularidad. Además de las transacciones, se verifica la firma criptográfica, asegurando que el bloque realmente provenga del validador que tenía el turno de proponerlo y que no ha sido manipulado por un tercero. Si todo es correcto (las transacciones son válidas y la firma es auténtica), este nodo emite una certificación (*attestation*) digital afirmando que se ha revisado el bloque y es legítimo. Cuando la gran mayoría de los validadores envían su certificación para el mismo bloque, este se considera consolidado y se añade permanentemente a la cadena de bloques (Ethereum Foundation, 2025e).

Mientras que en el modelo de Prueba de Trabajo (PoW) la generación de bloques depende de la variación en la dificultad de minado, la Prueba de Participación (PoS) de Ethereum opera según intervalos fijos. Se trata de un sistema que organiza el tiempo en *slots* o ranuras de 12 segundos, agrupados en unidades mayores, los *epochs* (épocas), de 32 ranuras cada una. Durante cada intervalo de 12 segundos, el protocolo designa aleatoriamente a un validador encargado de proponer un bloque: compilar las transacciones pendientes, construir el bloque y difundirlo al resto de la red. Por otro lado, cada ranura cuenta con un comité de validadores elegidos al azar que actúan como testigos, validando o no la propuesta de bloque. De no existir este sistema, la emisión de votos por parte de todos los nodos cada 12 segundos generaría una sobrecarga insostenible

para la infraestructura (Ethereum Foundation, 2025e)

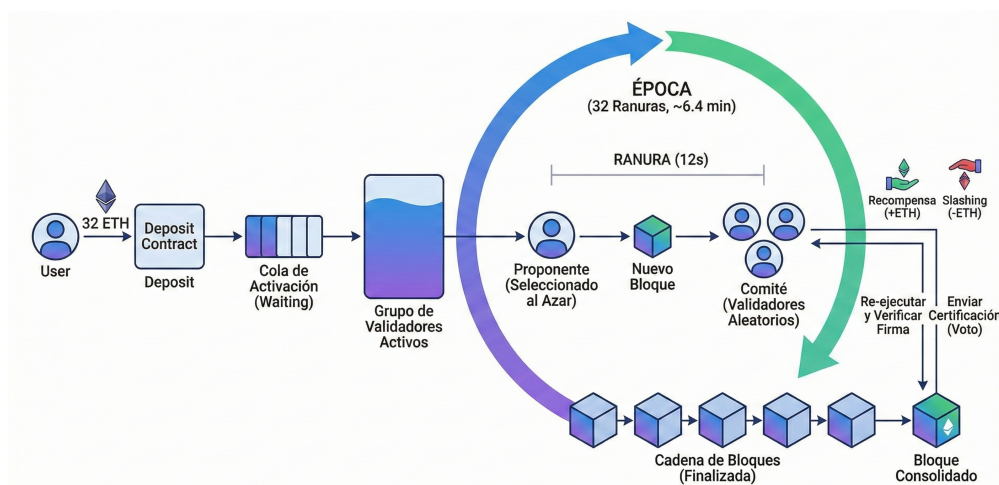


Figura 2.12.: Diagrama del funcionamiento de *Proof of Stake* en Ethereum. De elaboración propia.

En la arquitectura de Ethereum *Proof of Stake* (PoS) de PoS, un ataque del 51 % no requiere potencia computacional, sino el control de la mayoría del suministro de ETH depositado en el contrato de participación. Con este umbral de control, un actor malicioso podría influir en la regla de elección de bifurcación utilizando sus certificaciones para dotar de mayor “peso” a una bifurcación específica.

Mientras que con la *Proof of Work* un atacante que fracasa conserva su infraestructura física, con la *Proof of Stake* el capital del atacante está directamente expuesto al protocolo, que incluye mecanismos automatizados para detectar comportamientos contradictorios (como firmar dos bloques en la misma ranura). Al detectarse el ataque, el sistema ejecuta un proceso de *slashing*, destruyendo una parte significativa del ETH depositado y expulsando al validador de la red (Ethereum Foundation, 2025e).

Adquirir el 51 % del ETH en circulación para un ataque no solo requiere una inversión descomunal de capital, sino que la propia presión de compra dispararía el precio del ETH, incrementando el coste de adquisición de forma exponencial a medida que el atacante intenta acumular fondos.

En el caso de que un atacante logre superar las defensas, la comunidad mantiene una última línea pudiendo votar por eliminar específicamente las participaciones del atacante, neutralizando así su poder económico de forma permanente. También, los nodos honestos pueden optar por ignorar la cadena controlada por el atacante y continuar construyendo sobre una bifurcación minoritaria. Además se requiere la coordinación de los proveedores de infraestructura (exchanges, *stablecoins* como USDC/USDT, y protocolos de liquidez), quienes decidirían reconocer el valor económico únicamente en la cadena legítima.

“La Fusión” significó el final de la prueba de trabajo para Ethereum y dio inicio a la era de una *blockchain* más sostenible y ecológica, reduciendo el consumo de energía de la red en un 99.988 % (Kaneko, 2025).

2.3.7. Estándares de Tokens: La base de la tokenización

Tras analizar Bitcoin y Ether como los activos más representativos de la tecnología *Blockchain*, se introduce el concepto de **token** para comprender el alcance de este nuevo paradigma económico.

Definición 2.3.2 *Un token (término que en inglés significa “ficha”) es una unidad de valor basada en criptografía que es emitida por una entidad privada dentro de una red blockchain representando un activo o una utilidad específica.*

Cabe distinguir que, si bien todas las criptomonedas pueden categorizarse como *tokens*, no todos los *tokens* funcionan como criptomonedas. Mientras que el dinero en efectivo, el BTC o el ETH son activos intercambiables entre sí, existen otros elementos (como títulos de propiedad o entradas a eventos) cuya naturaleza es única y no intercambiable por una versión idéntica. Esta distinción reside en la propiedad de la **fungibilidad**.

Como se detalló en la sección 2.3.2, la red Ethereum permite que los *smart contracts* automaticen las reglas de emisión, gestión y transferencia de diferentes *tokens* según las instrucciones del código programado. Para que los *tokens* puedan interactuar con *wallets*, *exchanges* y otras *dApps*, es necesario que este conjunto de reglas sea común.

Definición 2.3.3 *Un estándar de token es un conjunto de reglas, protocolos y especificaciones técnicas que definen cómo se crean, emiten y gestionan los activos digitales en una blockchain. Fundamenta unas reglas de comportamiento que un smart contract debe implementar para garantizar la compatibilidad e interoperabilidad dentro de un ecosistema blockchain específico.*

Las diversas redes *blockchain* evolucionan gracias a las aportaciones de su comunidad, realizadas a través de *improvement proposals* y *request for proposals*. Cuando estas sugerencias logran una aceptación generalizada, pasan a definir los estándares del protocolo (Caballero, 2019). Este trabajo se centra principalmente en la red Ethereum, por ser la red más extendida en este aspecto y con una mayor variedad.

En el contexto de Ethereum, estos estándares se conocen como **ERC** (*Ethereum Request for Comments*). Sin estos estándares, cada desarrollador crearía una lógica de transferencia o consulta de saldo distinta, lo que obligaría a los ecosistemas (como Uniswap o MetaMask) a realizar integraciones personalizadas para cada nuevo activo, haciendo el sistema ineficiente e inescalable.

Para crear el estándar más ampliamente utilizado, aceptado y adaptado del mundo, pasaron 19 propuestas antes de que la número 20 convenciera a la mayor parte de la comunidad.

2.3.7.1. ERC-20

Propuesto originalmente por Fabian Vogelsteller en Noviembre 2015, el estándar **ERC-20** (*Ethereum Request for Comments 20*) nació con el objetivo de unificar la interfaz de los activos digitales dentro de la red Ethereum. Este protocolo establece un marco

normativo que permite que diversos *tokens* funcionen de manera predecible y compatible entre diferentes *wallets*, *exchanges* y *smart contracts* (Ethereum Foundation, 2025a).

La característica distintiva del ERC-20 es la **fungibilidad**. Bajo este principio, cada unidad del activo es idéntica en valor y naturaleza a cualquier otra, lo que permite su intercambio recíproco sin distinción, de manera análoga al funcionamiento de las divisas tradicionales o el propio Ether (ETH).

Entre las funcionalidades que proporciona el estándar ERC-20, destacan:

- Envíos de *tokens* entre distintas direcciones.
- Consulta del balance disponible en un monedero específico.
- Verificación del volumen total de activos emitidos y circulantes en la red.
- Validación de permisos de una entidad externa para gestionar fondos específicos.

Para que un contrato inteligente sea categorizado formalmente como ERC-20, debe implementar obligatoriamente una serie de métodos y eventos detallados a continuación.

Métodos

```
1 function name() public view returns (string)
2 function symbol() public view returns (string)
3 function decimals() public view returns (uint8)
4 function totalSupply() public view returns (uint256)
5 function balanceOf(address _owner) public view returns (uint256 balance)
6 function transfer(address _to, uint256 _value) public returns (bool success)
7 function transferFrom(address _from, address _to, uint256 _value) public
  returns (bool success)
8 function approve(address _spender, uint256 _value) public returns (bool
  success)
9 function allowance(address _owner, address _spender) public view returns (
  uint256 remaining)
```

Código 2.1: Interfaz de métodos del estándar ERC-20. Extraído de (Ethereum Foundation, 2025a).

Eventos

```
1 event Transfer(address indexed _from, address indexed _to, uint256 _value)
2 event Approval(address indexed _owner, address indexed _spender, uint256
  _value)
```

Código 2.2: Interfaz de eventos del estándar ERC-20. Extraído de (Ethereum Foundation, 2025a).

2.3.7.2. ERC-721

Bajo la denominación de **NFT** (*Non Fungible Tokens*), encontramos activos digitales que se distinguen por ser únicos e indivisibles. Su arquitectura es especialmente útil para representar elementos que requieren exclusividad, como pases numerados para eventos, artículos de coleccionismo o títulos de propiedad digital. Debido al enorme potencial de estas aplicaciones, se desarrolló el estándar **ERC-721** como el marco normativo de referencia.

Introducido a principios de 2018 por William Entriken, Dieter Shirley, Jacob Evans y Nastassia Sachs, el **ERC-721** (*Ethereum Request for Comments 721*) define una estructura donde cada unidad es única, permitiendo variar su valor según las características del activo. A nivel de código, la distinción se garantiza mediante un `tokenId` (`uint256`). De este modo, la pareja formada por la dirección del contrato y dicho identificador asegura una unicidad universal para cada activo (Ethereum Foundation, 2025b)

Entre las funcionalidades que proporciona el estándar ERC-721, destacan:

- Envíos de *tokens* entre distintas direcciones.
- Consulta de balance disponible en un monedero específico.
- Consulta de la dirección de la cuenta del dueño de un *token* específico
- Verificación del volumen total de activos emitidos y circulantes en la red.
- Validación de permisos de una entidad externa para gestionar fondos específicos.

Para que un contrato inteligente sea categorizado formalmente como ERC-721, debe implementar obligatoriamente una serie de métodos y eventos que detallaremos a continuación.

Métodos

```
1  function balanceOf(address _owner) external view returns (uint256)
2  function ownerOf(uint256 _tokenId) external view returns (address)
3  function safeTransferFrom(address _from, address _to, uint256 _tokenId,
    bytes data) external payable
4  function safeTransferFrom(address _from, address _to, uint256 _tokenId)
    external payable
5  function transferFrom(address _from, address _to, uint256 _tokenId)
    external payable
6  function approve(address _approved, uint256 _tokenId) external payable
7  function setApprovalForAll(address _operator, bool _approved) external
8  function getApproved(uint256 _tokenId) external view returns (address)
9  function isApprovedForAll(address _owner, address _operator) external
    view returns (bool)
```

Código 2.3: Interfaz de métodos del estándar ERC-721. Extraído de (Ethereum Foundation, 2025b).

Eventos

```
1  event Transfer(address indexed _from, address indexed _to, uint256
    indexed _tokenId)
2  event Approval(address indexed _owner, address indexed _approved, uint256
    indexed _tokenId)
3  event ApprovalForAll(address indexed _owner, address indexed _operator,
    bool _approved)
```

Código 2.4: Interfaz de eventos del estándar ERC-721. Extraído de (Ethereum Foundation, 2025b).

2.3.7.3. ERC-3643

Introducimos en esta sección el **ERC-3643**, estándar de la tokenización de RWA (Real World Assets) (concepto que se definirá en la sección 2.4.1) y que conviene conocer para los siguientes capítulos.

Propuesto originalmente por el equipo de Tokeny Solutions (Joachim Lebrun, Tony Malghem y otros) (Tokeny Solutions, 2023) y basado en el protocolo de código abierto **T-REX** (*Token for Regulated EXchanges*), el **ERC-3643** es un estándar de *tokens* regulados que añade una capa de gestión de identidad institucional a la red Ethereum. A diferencia de otros estándares, el ERC-3643 utiliza un sistema de registros de identidad de código abierto, ONCHAINID. En este sistema, la identidad de cada participante es verificada por las partes autorizadas y almacenada en la cadena de bloques, lo que permite una gestión de identidad fluida y segura. El objetivo de este sistema es garantizar que las transferencias solo se ejecuten tanto si el emisor como el receptor cumplen con las reglas de cumplimiento programadas en el contrato inteligente (ERC-3643 Association, 2025).

Entre las funcionalidades que proporciona el estándar ERC-3643, destacan:

- Únicamente personas o entidades verificadas pueden interactuar con los *tokens* a través de un proceso de *Know Your Customer* (KYC).
- Se pueden establecer reglas sobre la transferencia de tokens, en qué lugares y bajo qué marcos regulatorios. Veremos el marco legal bajo el que se encuentran en la sección 2.7.
- Capacidad de congelar y descongelar *tokens* por orden judicial o pérdida de claves.
- Recuperación de activos para usuarios que han perdido el acceso a su *wallet* privada.
- A pesar de sus características mejoradas de seguridad y cumplimiento, los tokens ERC-3643 mantienen la compatibilidad con las plataformas y herramientas ERC-20 existentes. Esto significa que pueden integrarse en los ecosistemas *blockchain* existentes con modificaciones mínimas.

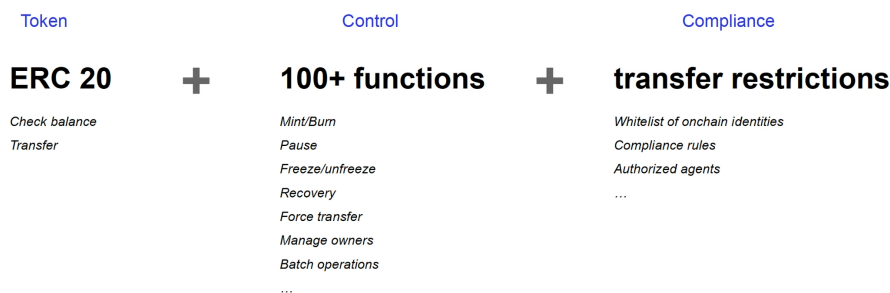


Figura 2.13.: El ERC-3643 es un conjunto de *smart contracts*. Tomada de (ERC-3643 Association, 2025).

La Figura 2.13 muestra cómo se construye un *token* regulado mediante tres capas. Es necesario que tenga las funcionalidades de un *token* como el ERC-20, una capa de gestión que está compuesta por más de 100 funciones y una capa legal con reglas a verificar por el contrato.

Complementaremos esta sección con la sección 2.8, donde veremos por qué se añaden estas funcionalidades y por qué es relevante el estándar ERC-3643.

2.3.8. *Utility vs. Security tokens*

En la sección anterior se han visto varios estándares, sin embargo, la principal clasificación de *tokens* se realiza con respecto a su representación implícita, donde existen dos clases distintas:

- ***Utility tokens***: Frecuentemente mal denominados como tokens de utilidad, estos activos funcionan como llaves digitales que habilitan el uso de las funcionalidades de una plataforma o servicio específico.
- ***Security tokens***: Se traducen habitualmente de forma imprecisa como tokens de seguridad. Se trata de activos que actúan como certificados digitales, otorgando derechos sobre participaciones societarias o títulos de deuda.

Cuando un *token* es considerado *Utility* carece de regulación financiera, legalmente se comporta de la misma manera que una prenda o un objeto decorativo. Sin embargo, cuando un *token* es considerado *Security* pasa a estar bajo el marco de la normativa financiera, rigiéndose en territorio español por la Ley del Mercado de Valores (LMV). Esta regulación implica que no se puedan realizar transacciones con estos *tokens* como si fueran precisamente prendas de ropa u objetos decorativos, si no que su tenencia se reserva a perfiles de inversión profesional y, en consecuencia, su puesta en circulación debe cumplir con las directrices de los organismos de supervisión. En el ámbito nacional, dicha competencia recae sobre la Comisión Nacional del Mercado de Valores (CNMV), la

cual que realiza esta tarea tratando de proteger al ciudadano de instrumentos financieros no regulados o directamente de estafas (Caballero, 2019).

Clasificar correctamente un *token* de forma jurídica es una de las decisiones más importantes en la fase de diseño de un proyecto. El valor de un *utility token* suele fluctuar en función de la oferta y demanda vinculadas al mercado al que pertenece. Si la revalorización del *token* se produce exclusivamente debido a estas leyes de mercado el *token* podría encajar en esta categoría. Por el contrario, si el incentivo principal de adquisición es la expectativa de beneficio derivada de factores externos como por ejemplo el reparto de dividendos o intereses prefijados, el instrumento debe clasificarse como un *security token*.

Los organismos reguladores —con la SEC en Estados Unidos y la CNMV en el marco europeo— han aumentado la supervisión de las denominadas “securities camufladas”. El lanzamiento de un activo financiero bajo la apariencia de un *utility token* conlleva riesgos legales significativos y sanciones administrativas severas.

En este sentido, la elección del modelo de *security token* no debe percibirse como un obstáculo burocrático, asumir el rigor regulatorio permite construir una arquitectura de *tokenomics* robusta y transparente, lo que incrementa la confianza en el proyecto y lo dota de la seguridad jurídica necesaria para atraer capitales a largo plazo (Caballero, 2019).

En adelante se comenzarán a introducir conceptos y secciones relacionadas con la tokenización inmobiliaria, que como se verá en el capítulo 3, constituirá uno de los principales objetivos de este trabajo.

2.4. Fundamentos de la Tokenización de Activos

Definición 2.4.1 *La tokenización de un activo consiste en el proceso de representar digitalmente la propiedad o los derechos subyacentes de un activo en una blockchain. A través de esta técnica, el activo —ya sea tangible (como materias primas o bienes raíces) o intangible (como patentes, instrumentos de deuda o derechos de autor)— se fragmenta en tokens, que funcionan como títulos representativos de una fracción del valor o de los derechos de explotación del activo original.*

En el ámbito de la tokenización, el término “activo” se emplea en un sentido amplio, ya que técnicamente es posible tokenizar también pasivos. La naturaleza de esta distinción depende estrictamente de la posición del sujeto en el balance: por ejemplo, una obligación de deuda constituye un pasivo para la entidad que busca financiación, pero representa un activo (un derecho de cobro) para el inversor o acreedor. En consecuencia, cualquier instrumento que represente valor económico y sea capaz de generar rendimientos, ya se trate de instrumentos de deuda (renta fija) o de capital (*equity*), es susceptible de ser representado mediante tokens (Caballero, 2019).

Este proceso es posible gracias a la tecnología *blockchain*, que garantiza la inmutabilidad del registro, la seguridad criptográfica y la transferencia eficiente de los activos. Un activo tokenizado se emite sobre una cadena de bloques, permaneciendo inicialmente

bajo la custodia del emisor, el propietario del mismo. La venta de estos activos puede gestionarse públicamente —sujeta al cumplimiento normativo de la Comisión Nacional del Mercado de Valores (CNMV)— o de forma privada. Esta innovación ha democratizado un mercado antes restringido a altos patrimonios, permitiendo que inversores con menor capacidad adquisitiva accedan a la propiedad fraccionada y a sus rendimientos derivados con barreras de entrada mínimas (Caballero, 2019).

2.4.1. Tokenización de activos físicos

Definición 2.4.2 *Se definen como **Real World Assets (RWA)** aquellos activos que poseen una existencia física o legal fuera del ecosistema blockchain, pero cuyos derechos de propiedad, explotación o flujos económicos son representados mediante tokens en una red de registro distribuido (DLT). El valor de estos activos digitales está vinculado directamente al rendimiento y la valoración de su activo subyacente en el mundo real.*

La principal complejidad de tokenizar un activo físico se encuentra en la sincronización constante entre su estado en el mundo real y su representación digital. Para que la lógica programable de los *smart contracts* sea efectiva y responda a la realidad del activo, es necesario el uso de oráculos.

Definición 2.4.3 *Un **oráculo** es un puente que conecta una blockchain con fuentes de datos o sistemas externos. Las funciones programadas en un smart contract a veces requieren de datos que no necesariamente se encuentran en la blockchain. Los oráculos se encargan de hacer llegar esta información y que así los contratos puedan ejecutarse correctamente.*

Por ejemplo, al adquirir un inmueble, el comprador debe asegurarse de que la titularidad del vendedor es legítima y que la vivienda se encuentra al corriente de sus obligaciones tributarias, sin deudas frente a acreedores de cualquier clase. Para ello, es necesario contar con una certificación registral vigente, complementada con los documentos bancarios o fiscales pertinentes. Los oráculos realizan esta labor de puente, vinculando el ecosistema *blockchain* con los sistemas de información tradicionales y proporcionando al *smart contract* los datos requeridos para su correcta ejecución.

Existen varios tipos de oráculos, a continuación se describen algunos de los más relevantes:

- **Oráculos de software:** Son los más comunes y se encargan de suministrar información proveniente de fuentes digitales externas. Integran datos procedentes de APIs web, bases de datos, cotizaciones de mercados financieros en tiempo real o cualquier información disponible en la red que sea necesaria para ejecutar las condiciones del contrato.
- **Oráculos de hardware:** Permiten que el *smart contract* interactúe con el mundo físico. Se basan en la infraestructura del Internet de las Cosas (IoT), tales como

sensores de temperatura, lectores de radiofrecuencia (RFID), medidores inteligentes de suministro o dispositivos de control de acceso. Son críticos en la logística y gestión de activos tangibles para validar estados físicos de manera automatizada.

- **Oráculos humanos:** Consisten en individuos o entidades con conocimientos especializados que validan la ocurrencia de un evento que no puede ser captado fácilmente por sensores digitales. En el ámbito inmobiliario, esta figura suele recaer en profesionales como notarios, registradores o peritos tasadores.

2.4.2. Tokenización de inmuebles

Este trabajo se centrará concretamente en inmuebles como RWA, ya que es un campo muy ilustrativo y no requiere la introducción de conceptos avanzados como sí lo hacen otros activos. Cabe destacar que tokenizar un inmueble es darle la capacidad de divisibilidad, transparencia y liquidez a una propiedad. Es fundamental precisar que el *token* no pretende sustituir la función del Registro de la Propiedad, sino que actúa como un certificado digital, programable e inmutable que representa y automatiza la ejecución de los derechos sobre el activo.

La tokenización de activos inmobiliarios debe entenderse como un proceso que se compone de ingeniería legal, económica y tecnológica. En realidad la tokenización no se ejecuta de forma directa sobre el bien inmueble físico, sino que se realiza sobre un Vehículo de Propósito Especial (**SPV**, por sus siglas en inglés). Esta entidad legal actúa como titular legal y única del inmueble, mientras que el proceso de emisión digital fracciona y representa los derechos económicos o las participaciones societarias de dicha entidad en forma de *tokens*. Esta estructura ahorra la burocracia que supondría tener una enorme cantidad de *token holders* en el Registro de la Propiedad y es la que da validez real al activo.

Para tokenizar un inmueble, la plataforma Tutellus (Tutellus, 2025a) propone seguir los siguientes pasos:

1. **Constitución del SPV:** El proceso se inicia con la selección del activo y la construcción del SPV, que actuará como titular en el registro del inmueble. En esta etapa, es recomendable que expertos, taseen el activo y auditen su situación legal.
2. **Diseño de la arquitectura financiera (*Tokenomics*) y dictamen jurídico:** Se define la naturaleza del *token* (valor nominal, cantidad y derechos vinculados). Es crucial determinar si el activo otorga derechos de propiedad, flujos de caja derivados del alquiler o derechos de gobernanza. Este diseño debe acompañarse de una *legal opinion* que clasifique el instrumento (generalmente como *security token*) para asegurar el cumplimiento con la Ley del Mercado de Valores.
3. **Selección de la infraestructura DLT:** Se elige la red de registro distribuido (*blockchain*) más adecuada según criterios de seguridad, escalabilidad y ecosistema. Redes como Ethereum o Polygon son las opciones predominantes debido a su robustez y compatibilidad con estándares financieros.

4. **Desarrollo y auditoría del *smart contract*:** El contrato inteligente actúa como el protocolo de ejecución inmutable que automatiza el reparto de rentas y las reglas de transferencia. Dado que el código es ley (De Filippi & Hassan, 2016) cuando el contrato está desplegado, es fundamental someter los contratos a una auditoría externa para reducir riesgos de errores en el código.
5. **Implementación de mecanismos de *compliance*:** Para operar dentro del marco legal, se integran protocolos de *Know Your Customer* (KYC) y *Anti-Money Laundering* (AML). Estas herramientas validan la identidad de los inversores y aseguran que la emisión cumpla con las normativas del regulador correspondiente. En España, veremos en la sección 2.7 que se debe verificar y confirmar que el *whitewater* (documento oficial) describe a la perfección el proyecto de tokenización a través de una ESI (Empresa de Servicios de Inversión) en función de la normativa de la CNMV (Comisión Nacional del Mercado de Valores).
6. **Gestión operativa mediante oráculos:** Una vez desplegado el proyecto, se requiere la integración de oráculos para conectar la información del mundo real (ingresos por alquiler, gastos de mantenimiento) con el *smart contract*. Esto garantiza que la distribución de beneficios sea precisa y responda a la actividad económica real del inmueble.
7. **Estrategia de liquidez y mercados secundarios:** Se lanza la oferta al mercado y se establecen alianzas con sistemas de negociación alternativos o mercados secundarios. El objetivo es permitir que el inversor pueda liquidar su posición sin las dificultades y retrasos propios del mercado inmobiliario tradicional.

2.4.3. Beneficios y riesgos de la tokenización de inmuebles

La transición del modelo inmobiliario tradicional, caracterizado por su iliquidez y altas barreras de entrada, hacia un ecosistema tokenizado, ofrece las siguientes ventajas competitivas (Tutellus, 2025a; 2025b; 2025c):

- **Optimización de la liquidez y fraccionamiento del activo:** Históricamente, el sector inmobiliario se ha considerado un mercado ilíquido debido a la cantidad de trámites y de tiempo necesario para el traspaso del inmueble. La tokenización permite el fraccionamiento de la propiedad, transformando un activo indivisible en múltiples unidades digitales. Esto facilita la salida de inversores a través de mercados secundarios, reduciendo drásticamente los tiempos de venta con respecto a los del mercado tradicional.
 - **Democratización del capital y alcance global:** Mediante la reducción de los umbrales mínimos de inversión (fijados en ocasiones desde los 100 euros), se expande la base de inversores potenciales de un ámbito local a una escala global, eliminando fronteras geográficas y permitiendo que inversores minoristas accedan a proyectos de alta rentabilidad que anteriormente estaban reservados a grandes capitales o instituciones.
-

- **Transparencia inmutable y trazabilidad:** Gracias a la naturaleza de la tecnología *blockchain*, cada transacción, derecho de propiedad y reparto de beneficios queda registrado de forma inmutable. Esta trazabilidad elimina la opacidad histórica del sector, permitiendo que cualquier tenedor de *tokens* realice una auditoría en tiempo real de los flujos económicos, garantizando la integridad de la información sin depender de terceros.
- **Eficiencia operativa y reducción de costes transaccionales:** La implementación de *smart contracts* permite la automatización de procesos críticos, como el reparto de dividendos o la ejecución de derechos de voto. Esta desintermediación reduce la dependencia de gestores, notarios y otros intermediarios tradicionales, lo que se traduce en una reducción significativa de los costes operativos y de transacción, optimizando así el rendimiento neto de la inversión.
- **Reducción de burocracia:** La digitalización de los derechos sobre el activo simplifica los trámites administrativos. Al estar las reglas de cumplimiento codificadas directamente en el protocolo, se simplifica la transferencia de activos, aportando una agilidad transaccional propia de los mercados financieros digitales.

A pesar de las ventajas analizadas, al tratarse de una tecnología vanguardista, la tokenización de activos inmobiliarios no está exenta de riesgos (Tutellus, 2025a; 2025b; 2025c):

- **Incertidumbre regulatoria:** Actualmente, el marco legal para los *security tokens* se encuentra en una fase de evolución y adaptación. En la Unión Europea, aunque existen avances significativos, la clasificación jurídica y el tratamiento fiscal de estos activos pueden variar entre jurisdicciones. Esta falta de armonización internacional genera una barrera de entrada para la inversión transfronteriza y una inseguridad jurídica hasta que se consoliden marcos definitivos.
 - **Riesgo legal:** Uno de los mayores riesgos reside en la desconexión entre el *token* y el activo subyacente. Si la ingeniería legal que vincula el *token* con el SPV no es sólida, el *token* pierde su validez como título representativo de valor. Una arquitectura jurídica deficiente puede invalidar los derechos económicos o societarios ante un tribunal, convirtiendo al activo digital en un instrumento sin respaldo legal real.
 - **Riesgo de iliquidez en el mercado secundario:** Si bien la tecnología facilita la transferencia instantánea, la liquidez real depende de la profundidad del mercado secundario. La existencia de un *token* no garantiza la presencia de compradores. Si el proyecto inmobiliario carece de atractivo o el volumen de negociación es reducido, el inversor podría enfrentar dificultades en la transacción de *tokens*, replicando la iliquidez característica del mercado tradicional.
 - **Proyectos sin respaldo real:** El denominado *hype* tecnológico puede impulsar proyectos con una validación insuficiente de sus activos. La *blockchain* aporta trans-
-

parencia y eficiencia en la gestión, pero no puede compensar las carencias de un proyecto inmobiliario mal fundamentado o con valoraciones fuera de mercado.

- **Riesgos de código:** Errores de programación en el código del *smart contract* o vulnerabilidades en el protocolo pueden dar lugar a la pérdida irreversible de fondos o derechos. La inmutabilidad de la cadena, si bien es una ventaja para la seguridad, se convierte en un riesgo si la lógica del contrato no ha sido verificada rigurosamente por expertos.

2.5. El auge de la tokenización RWA

El concepto de Real World Assets (RWA) visto en la sección 2.4.1 se ha consolidado como el puente definitivo entre las finanzas tradicionales (TradFi) y las finanzas descentralizadas (DeFi). A diferencia de los activos nativos digitales, cuyo valor reside exclusivamente en su dinámica programada dentro de la red, los RWA representan derechos sobre activos que existen fuera de la cadena de bloques, de este modo, adquieren las ventajas de la tecnología *blockchain* sin que el activo pierda su vínculo legal y físico con el mundo real.

Durante los últimos años, entidades de relevancia global como **BlackRock**, a través de su fondo tokenizado **BUIDL** (U.S. Securities and Exchange Commission, 2024), o **J.P. Morgan**, mediante su plataforma *kinexys* (J.P. Morgan, 2024), han respaldado el uso de *blockchains* para la gestión de estos activos. Asimismo, **Ondo Finance** (Ondo Finance, 2025) se ha posicionado como un actor clave en el sector de los RWA al tokenizar instrumentos financieros de bajo riesgo como bonos del tesoro estadounidense y productos de renta fija.

En el contexto nacional, **Brickken** (Brickken, 2025) se consolidó durante 2025 como un referente gracias a ofrecer una infraestructura modular para la emisión de activos digitales. Su propuesta permite la tokenización de una amplia gama de activos, abarcando desde el capital social de empresas emergentes (*Equity Tokenization*) hasta derechos de propiedad intelectual.

2.5.1. Proyecciones y evolución del mercado en la próxima década

De acuerdo con un informe conjunto elaborado por *Boston Consulting Group* (BCG) y la firma de infraestructura digital *Ripple* (Ripple & Boston Consulting Group, 2025), se estima que la tokenización de activos reales en el escenario intermedio crecerá aproximadamente de 0.6 billones de dólares en 2025 a 18.9 billones (*trillions* americanos) de dólares en 2033. Esto representa una tasa de crecimiento anual compuesta del 53 %. Se puede observar este crecimiento estimado de la tokenización RWA en la Figura 2.14.

El 53 % de CAGR (tasa de crecimiento anual compuesta) indica que no es un crecimiento lineal, sino una adopción acelerada a medida que la infraestructura tecnológica y regulatoria madura. El estudio tiene en cuenta una varianza de escenarios, donde una situación optimista que asume una claridad regulatoria global rápida y una migración masiva de activos tradicionales proyecta hasta 23.4 billones, mientras que un escenario

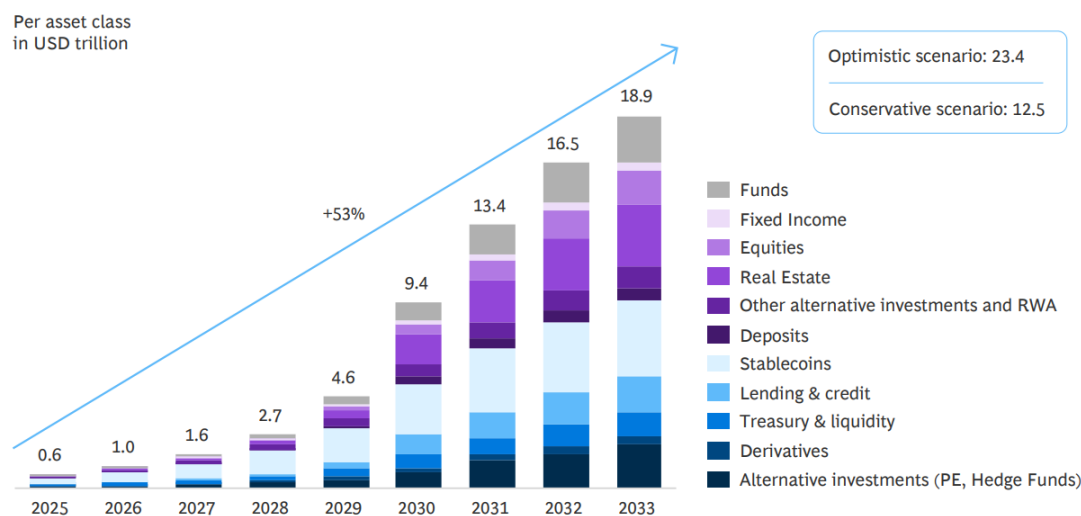


Figura 2.14.: Crecimiento estimado de la tokenización RWA hasta 2033. Tomada de (Ripple & Boston Consulting Group, 2025).

más conservador teniendo en cuenta posibles frenos por fricciones legales o lentitud en la adopción institucional asume como mínimo unos 12.5 billones.

El gráfico de barras apiladas revela los sectores que impulsarán este volumen. Los inmuebles (*Real Estate*) representados en el bloque morado central se consolidan como uno de los pilares de volumen físico tokenizado. Los bloques azul claro y azul oscuro muestran que la base del ecosistema son los activos líquidos, que sirven como “combustible” o garantía para otras transacciones. Es crucial notar que el gráfico no incluye criptoactivos nativos (como Bitcoin o Ether), centrándose exclusivamente en activos que tienen representación en la economía tradicional, fuera de la cadena. Estas cifras subrayan que la tokenización no constituye únicamente una optimización técnica, sino una reconfiguración estructural de los mercados de capitales globales. La adopción de estos modelos permitirá transformar activos históricamente ilíquidos en instrumentos financieros dinámicos, accesibles y plenamente integrados en el ecosistema digital.

Dentro de este marco económico, la tokenización inmobiliaria se posiciona como uno de los sectores con mayor potencial de crecimiento debido a su naturaleza y a su tradicional falta de liquidez. Mientras que en 2023 la tokenización se centraba mayoritariamente en deuda pública (T-Bills), el escenario actual de 2026 muestra un desplazamiento hacia activos más complejos, donde las propiedades e inmuebles fraccionados permiten capturar valor tanto por la revalorización del activo como por la generación de rentas pasivas.

2.6. Modelos de Tokenización Inmobiliaria

La aplicación de la tecnología *blockchain* al sector inmobiliario ha dado lugar a diversos modelos operativos que buscan resolver el dilema de la inversión en ladrillo: liquidez,

accesibilidad y seguridad jurídica. A continuación, se analizan los dos enfoques predominantes en el mercado actual.

2.6.1. Modelos de participación: Directo vs. Indirecto

Uno de los mayores retos del estado del arte actual es el “vínculo” entre el *token* y el Registro de la Propiedad. Actualmente se puede realizar de las siguientes formas:

- **Modelo de Propiedad Indirecta (SPV):** Se trata del modelo estándar en España y Europa. Como se comenta en la sección 2.4.2, consiste en la creación de una Sociedad de Propósito Especial (SPV) que adquiere el inmueble. Lo que se tokeniza no es la escritura del piso, sino las acciones o participaciones de dicha sociedad, o bien un préstamo participativo cuyos rendimientos están ligados al alquiler y venta del activo. Este modelo es el más extendido ya que permite cumplir con la normativa de mercados de valores (como la Ley 6/2023 (Jefatura del Estado, 2023) en España).
- **Modelo de Propiedad Directa:** Aunque es el ideal tecnológico, su implementación es limitada y solo se puede aplicar bajo legislaciones muy avanzadas como en algunos estados de EE.UU. o microestados con legislaciones *ad hoc*. En este modelo, el *token* representa directamente un porcentaje de la escritura de propiedad.

2.6.2. Rendimientos de la tokenización inmobiliaria

La arquitectura de un *smart contract* permite programar una amplia variedad de derechos y obligaciones. En la práctica actual del sector, y siguiendo la clasificación propuesta por referentes educativos y operativos como Tutellus (Tutellus, 2025b), los *tokens* inmobiliarios se agrupan en tres categorías principales en función del rendimiento que ofrecen al inversor:

- **Token de propiedad fraccionada (*Equity Token*):** Esta clase de tokens corresponden directamente a una participación en el capital de la entidad propietaria del activo. Al adquirir este tipo de activo, el inversor se convierte en copropietario indirecto del inmueble. Según el informe de la OECD (2020) (OECD, 2020), este tipo de *token* otorga derechos proporcionales a los dividendos generados por la explotación del activo y a la plusvalía obtenida en una futura venta.

Ejemplo práctico: La inversión de 5.000 € en un activo tokenizado puede representar el 0,5 % de las participaciones de la sociedad dueña de un edificio residencial, otorgando derechos políticos y económicos sobre esa fracción.

- **Token de flujo de caja o rendimiento (*Yield Token*):** A diferencia de los modelos de participación directa, este instrumento no transmite la titularidad del capital social. En su lugar, garantiza el derecho a percibir una fracción de los ingresos operativos o rentas derivadas del arrendamiento, sin que el tenedor ostente la copropiedad del bien inmueble.
-

Ejemplo práctico: Un activo digital que habilita la recepción del 0,1 % de los beneficios netos mensuales generados por un local de negocio, eximiendo al inversor de las responsabilidades legales ligadas a la propiedad.

- **Token híbrido:** Presenta una naturaleza dual al combinar propiedad y derecho de gobernanza.

Ejemplo práctico: Un activo que acredita la posesión del 0,1 % del activo y, a su vez, otorga capacidad de voto para decidir sobre la gestión de mantenimientos o la selección de los arrendatarios.

2.7. Marco regulatorio y cumplimiento legal en la tokenización de inmuebles en España

Para llevar a cabo la tokenización inmobiliaria hay que ajustarse al marco normativo que garantiza la protección del inversor y la integridad del mercado. En el escenario europeo actual, la regulación que se aplica es la siguiente:

2.7.1. Impacto de MiCA

El Reglamento MiCA (Markets in Crypto-Assets) (Parlamento Europeo y Consejo de la Unión Europea, 2023) establece un marco legal uniforme para los criptoactivos y los servicios relacionados en la Unión Europea, el cual entró en vigor el 30 de diciembre de 2024. Aunque se centra en activos que no se califican como instrumentos financieros (no regula directamente los *security tokens* inmobiliarios), su implementación ha establecido un estándar de cumplimiento que influye en todo el ecosistema.

Para la tokenización de vivienda, el impacto de MiCA se manifiesta principalmente en la emisión, obligando a que los proveedores de servicios de criptoactivos operen bajo estándares de custodia y segregación de activos más estrictos. Asimismo, MiCA facilita la comercialización transfronteriza al establecer ciertas reglas en la Unión Europea, permitiendo que proyectos inmobiliarios con base en España puedan atraer capital europeo con mayor seguridad jurídica. Por último, el reglamento eleva la transparencia, imponiendo requisitos a los emisores de *security tokens*.

2.7.2. Ley 6/2023 de los Mercados de Valores

A nivel nacional, el escenario regulatorio se ha consolidado mediante la Ley 6/2023, de los Mercados de Valores y de los Servicios de Inversión (Jefatura del Estado, 2023). Esta norma no solo amplía el marco previsto en los reglamentos europeos, sino que deja constancia de la voluntad del estado español de aceptar e incorporar la tecnología de registro distribuido en el sistema financiero.

El artículo 6 de dicha ley representa un hito jurídico al reconocer expresamente la legalidad de los instrumentos financieros representados en sistemas basados en tecnología

blockchain. No obstante, la aplicación de esta ley depende estrictamente de la naturaleza del derecho tokenizado. Si el activo tokenizado se diseña como un valor negociable (orientado a la inversión), será de plena aplicación la legislación de mercados de valores. Esto implica la supervisión de la CNMV, la obligación de emitir un folleto informativo (o documento equivalente) y la intervención de figuras reguladas como Empresas de Servicios de Inversión (ESI), Depositarios Centrales de Valores o Entidades de Contrapartida Central.

Cuando los *tokens* representan derechos económicos directos o indirectos sobre un inmueble —como la participación en los rendimientos del alquiler o la plusvalía por su venta—, adquieren la condición jurídica de instrumentos financieros bajo el marco de la Ley 6/2023 (Jefatura del Estado, 2023). En este escenario, la emisión y comercialización quedan sujetas a la supervisión directa de la CNMV, lo que garantiza que el proyecto no opere en un vacío legal. Esta regulación exige la intervención de ESIs autorizadas, encargadas de validar el *whitepaper* donde se detalla el *token*, que deriva en un marco de protección al inversor. Este conjunto de reglas incluye la obligación de realizar evaluaciones de idoneidad y conveniencia, la clasificación de clientes según su perfil de riesgo y la transparencia total en la información de costes y riesgos asociados, elevando la tokenización inmobiliaria al mismo estándar de rigor que cualquier otro producto financiero del mercado tradicional.

2.7.3. Notaría y Registro de la Propiedad

En España, cualquier *token* que pretenda representar derechos sobre un inmueble debe, por imperativo legal y ante notario, formalizarlo en escritura pública e inscribirse en el Registro de la Propiedad. De nuevo se debe mencionar que tokenizar un derecho sobre un inmueble no aporta ninguna ventaja si por cada transacción los titulares deben llevar a cabo todos los trámites tradicionales de notaría y registro. Esta redundancia elimina las ventajas competitivas de la *blockchain* como la inmediatez y la reducción de costes. Es por ello que la industria ha optado por alternativas como la tokenización de vehículos como el SPV, modelos que permiten la agilidad del mundo digital cumpliendo con la rigidez del sistema tradicional.

2.8. Estándar para la tokenización de activos

La tokenización de activos requiere del uso de estándares específicos, ya que por ejemplo no tendría sentido representar el capital social de una empresa con un *token* bajo el estándar ERC-20. Como sabemos, si perdemos la clave de la *wallet* que da el acceso a los *tokens*, estos son irrecuperables y estaríamos perdiendo el capital social de la empresa. Esto carece de sentido jurídico, pues no pueden desaparecer las participaciones de una empresa, siempre han de tener un poseedor con derechos sobre ellas. Para solucionar este tipo de problemas son necesarias funciones extra, como un sistema de recuperación de claves. Estándares como el ERC-3643, visto en la sección 2.3.7.3, implementan estas funcionalidades entre otras, convirtiéndose en un estándar idóneo para la tokenización

de activos (Caballero, 2019).

Las criptomonedas y los *utility tokens* han demostrado el potencial de la tecnología *blockchain* para transferir activos de forma eficiente y descentralizada, pero a menudo también presentan limitaciones en el cumplimiento normativo. Los *security tokens*, al tener una expectativa de beneficio, requieren un estricto cumplimiento de las regulaciones financieras. En este sentido, el ERC-3643 proporciona el marco robusto necesario para satisfacer estas exigencias, facilitando la adopción institucional de la tecnología *blockchain* en los mercados financieros regulados (ERC-3643 Association, 2025).

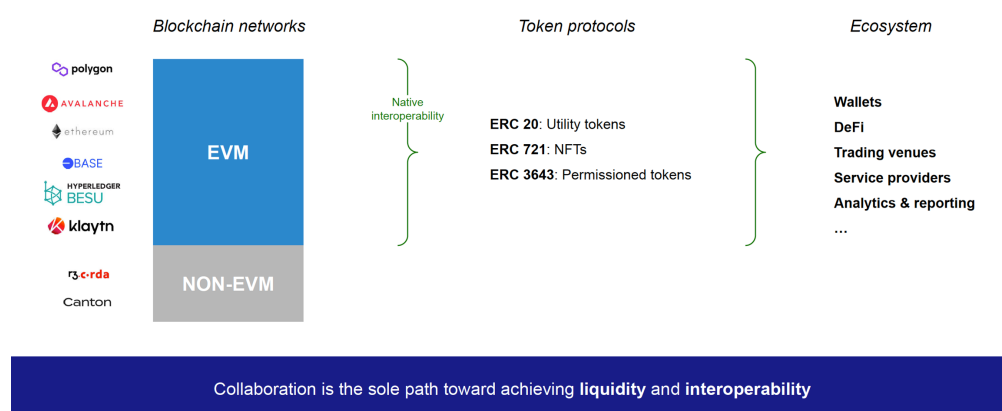


Figura 2.15.: La necesidad de un marco de tokenización común. Tomada de (ERC-3643 Association, 2025).

En la Figura 2.15 se puede observar cómo existe una gran variedad de redes, desde las públicas compatibles con EVM (como Ethereum o Polygon) hasta redes privadas o No-EVM (como R3 Corda). Sin un marco común, cada una de estas redes funciona de manera aislada, si se emite un activo en una red, es extremadamente difícil y costoso moverlo a otra si no comparten reglas técnicas básicas. En el centro de la imagen aparecen los estándares ERC-20, ERC-721 y ERC-3643 vistos en la sección 2.3.7. Estos protocolos funcionan como un acoplamiento entre las diferentes redes, al usar un estándar, se asegura la interacción del activo con distintas redes. A la derecha, la columna de *Ecosystem* muestra las plataformas que interactúan con los *tokens*, como las *wallets* o las DeFi entre otras, y es que si cada banco o empresa creara su propio estándar, una *wallet* tendría que desarrollar una integración distinta para cada activo.

Como dice la Figura 2.15, “La colaboración es el único camino para lograr liquidez e interoperabilidad”. Para que se de la liquidez es necesaria la presencia de muchos compradores y vendedores en un mismo lugar o en lugares conectados, si el mercado se encontrara fragmentado en 50 estándares distintos, la liquidez se vería reducida en 50 partes. Un marco común permite que cualquier plataforma “entienda” el *token* de forma automática, reduciendo costes, acelerando la adopción y aumentando la liquidez.

3. Objetivos

Este trabajo tiene como objetivo principal el diseño y desarrollo de una plataforma descentralizada para la tokenización de activos, concretamente de bienes inmuebles. Dada la naturaleza del proyecto en el contexto de la ciencia de datos, la fiabilidad de un activo digital depende de la veracidad de la información que lo respalda. Por ello, se debe establecer un control estricto sobre la transferencia de información del mundo físico a la *blockchain*. Se debe implementar una arquitectura basada en roles donde la ingesta y validación inicial de los datos de cada inmueble quede reservada exclusivamente a los administradores, evitando así que se pueda registrar información fraudulenta o de baja calidad en la red. Asimismo, los administradores deben ser capaces de modificar el precio de cualquier vivienda tokenizada, ya que al igual que ocurre en el mercado real, existen fluctuaciones en el precio de las mismas. Finalmente, para los inversores, la plataforma debe contar con una interfaz accesible que permita comprar o vender *tokens*, garantizando que cada movimiento genere un registro de datos inmutable, auditable y transparente.

Para alcanzar este propósito general, se definen los siguientes objetivos específicos:

- **Diseño de estructuras de datos inmutables:** Implementar la lógica de negocio mediante *Smart Contracts* (basados en el estándar ERC-20), garantizando que las estructuras de datos de cada inmueble sean precisas, inmutables y trazables desde el instante de su emisión.
- **Integridad documental descentralizada:** Integrar el protocolo IPFS (*Inter-Planetary File System*) para garantizar la disponibilidad permanente, la validez y la inmutabilidad de las escrituras y documentos oficiales, anclando el dato físico al activo digital mediante identificadores de contenido (CID).
- **Sincronización con datos de mercado:** Desarrollar un mecanismo seguro de actualización de precios que permita a los administradores alinear el valor del activo en la *blockchain* con la fluctuación del mercado inmobiliario del mundo real.
- **Consumo de oráculos *Off-chain*:** Implementar la extracción de datos financieros externos mediante APIs para dotar a la plataforma de conversión de divisas en tiempo real (EUR/ETH), mejorando la legibilidad de la información para el usuario final.
- **Abstracción y visualización de la información:** Desarrollar una interfaz de usuario (dApp) reactiva que acerque la tokenización al público general de manera visualmente clara e intuitiva.

4. Metodología y desarrollo

En este capítulo se describen con detalle todos los procesos realizados para cumplir con los objetivos expuestos en la sección anterior. Se explicará principalmente el flujo que seguirá la plataforma, abarcando desde la definición de las estructuras inmutables de datos (*smart contracts*) hasta su almacenamiento permanente en la *blockchain* y la presentación de los mismos mediante la dApp.

4.1. Entorno de desarrollo

A continuación, se introducen los componentes y las tecnologías utilizadas para realizar cada una de las partes del proyecto.

Smart Contracts (Backend on-chain): Se ha utilizado el lenguaje de programación **Solidity** para la redacción de los *smart contracts* del proyecto. Como se ha comentado a lo largo del trabajo se trata del estándar para el desarrollo de contratos y actúa como el lenguaje de definición de esquemas, estructurando cómo se almacena y procesa la información en la *Ethereum Virtual Machine* (EVM) (Wood, 2014).

Para la fase de escritura, compilación y depuración se eligió **Remix IDE** por su agilidad y su integración directa con proveedores Web3. Además, se ha integrado la biblioteca **OpenZeppelin** para la implementación del estándar ERC-20 (Ethereum Foundation, 2025a), garantizando que los datos financieros de las propiedades inmobiliarias cumplan con los estándares de seguridad auditados de la industria (OpenZeppelin, 2023), evitando vulnerabilidades para la manipulación de saldos.

Entorno de pruebas y almacenamiento de documentos: Para el despliegue y validación de los *smart contracts* se ha optado por **Sepolia (Sepolia Testnet)** (Ethereum Foundation, 2024b), una red de pruebas *Proof of Stake* (PoS) de Ethereum (Ethereum Foundation, 2025e). Esta decisión permite simular un entorno de producción idéntico a la red principal (*Mainnet*), garantizando que la trazabilidad de las operaciones y el coste computacional (*Gas*) operen de forma realista, pero sin incurrir en costes económicos reales durante la fase de pruebas.

Para tener la capacidad de almacenamiento de archivos pesados (como los PDFs de las escrituras) sin consumir demasiados recursos en la *blockchain* se ha integrado el protocolo **IPFS** (Benet, 2014) que se describe en la sección 4.2.

Capa de visualización y lógica de cliente (Frontend off-chain): La interfaz de usuario se ha construido utilizando la biblioteca de JavaScript **React** (Meta Platforms, Inc.,

2023). Su enfoque basado en componentes reutilizables y su eficiente gestión del estado (a través de *Hooks* como `useState` y `useEffect`) la hacen una herramienta idónea para consumir y visualizar los datos alojados en la *blockchain* en tiempo real a través de una *Single Page Application* (SPA).

Para establecer el puente de datos (*pipeline*) entre la interfaz web y los nodos de Ethereum, se ha implementado la librería **Ethers.js** (Moore, 2023). Esta herramienta facilita la serialización de transacciones, la firma criptográfica y la traducción de las estructuras de datos de la EVM (como los enteros de 256 bits en formato Wei) a métricas comprensibles para los usuarios.

La capa visual se ha desarrollado combinando hojas de estilo en cascada (CSS) para la configuración global, y estilos integrados dinámicamente en JavaScript (CSS-in-JS) para la arquitectura de componentes, implementando diseños como el *Glassmorphism* para conseguir una experiencia pulida.

Infraestructura y oráculos externos: Como proveedor de identidad y firma se ha utilizado **MetaMask** (ConsenSys, 2024). Se trata de una *wallet* diseñada para gestionar criptomonedas e interactuar con *dApps* en la red Ethereum y redes compatibles (BNB Chain, Polygon, etc.), permitiendo a los usuarios autenticarse de forma descentralizada y firmar transacciones de manera segura.

Para dotar a la plataforma de precios en divisas tradicionales como el Euro, se ha implementado la **API pública de CoinGecko** (CoinGecko, 2024). Esta actúa como un oráculo de datos, proporcionando la tasa de cambio en tiempo real ETH/EUR en el momento de la consulta.

Integración y despliegue: Por último, el despliegue del entorno *frontend* se ha realizado de forma descentralizada con **Vercel** (Vercel, 2024), una plataforma en la nube (PaaS) optimizada para aplicaciones React. Garantiza que el acceso a la información sea global, escalable y seguro, gracias a su red de entrega de contenido (CDN) y la provisión automática de certificados SSL (HTTPS) que cifran la transmisión de datos entre el cliente y el servidor.

4.2. Almacenamiento descentralizado de documentos: IPFS y Pinata

Para almacenar las escrituras de los inmuebles (en formato PDF) de forma segura y transparente, se ha utilizado el protocolo IPFS (*InterPlanetary File System*) (Benet, 2014). En la web tradicional (HTTP), los archivos se buscan por su ubicación en un servidor centralizado. Si este servidor falla, o si el administrador elimina o reubica el archivo, el enlace se rompe (el conocido error 404). Esto supone un riesgo de pérdida de datos inaceptable para una arquitectura *Web3*, ya que el *Smart Contract* perdería la referencia al documento legal que respalda el activo tokenizado.

IPFS resuelve este problema: en lugar de buscar por ubicación, el sistema busca por contenido (*Content Addressing*). Al subir el PDF de la escritura a la red descentralizada de IPFS, el protocolo procesa el archivo y genera un identificador único de forma criptográfica basado en su contenido, similar al *hash* estudiado en la sección 2.2.3.1 pero en este ecosistema se denomina **CID** (*Content Identifier*) (Protocol Labs, 2024).

Sin embargo, la arquitectura colaborativa de IPFS implica que los archivos no persisten en la red de forma indefinida a menos que un nodo asigne explícitamente recursos para su mantenimiento (proceso conocido como *pinning*). Para garantizar la alta disponibilidad y persistencia ininterrumpida de los documentos legales de las viviendas, se ha utilizado el servicio de **Pinata** (Pinata Technologies, 2024). Como proveedor de infraestructura IPFS, Pinata actúa como un nodo que asegura los archivos en la red, garantizando que el documento esté accesible globalmente las 24 horas sin depender de un único punto de fallo.

El **CID** generado tras esta fijación en Pinata forma parte de la dirección exacta mediante la cual se puede acceder al archivo, [https://harlequin-immense-llama-284.mypinata.cloud/ipfs/\[CID\]](https://harlequin-immense-llama-284.mypinata.cloud/ipfs/[CID]). En este proyecto se introduce directamente por simplicidad esta URL que está estructurada a través de un *gateway* en el *smart contract*, es el dato exacto que se registra en la *blockchain* tanto para las escrituras como para las imágenes de la vivienda. Esta arquitectura garantiza la integridad de los archivos de la propiedad, que entran en la categoría de datos no estructurados, y en el caso de que un actor malicioso intentase alterar un solo carácter del PDF original o un píxel de la imagen, el protocolo IPFS generaría un CID completamente distinto. Al no coincidir con el identificador integrado en la URL registrada en la *blockchain*, cualquier intento de fraude o manipulación de documentos sería detectado y rechazado al instante por la red. Finalmente, la accesibilidad pública de los archivos originales se consigue directamente a través de este *gateway* privado de Pinata que traduce el protocolo nativo a HTTP, permitiendo su lectura y visualización en cualquier navegador estándar y evitando los fallos de disponibilidad que presentan otros *gateways* públicos genéricos.

4.3. Arquitectura del sistema

El proyecto de tokenización inmobiliaria desarrollado tiene una arquitectura descentralizada de tres capas, diseñada para garantizar la inmutabilidad de los registros, la transparencia en las transacciones y la simplificación del proceso de interacción del usuario con la *blockchain*.

Capa lógica: Se han desarrollado dos *smart contracts* que interactúan entre sí mediante un patrón de diseño *Factory* (Zheng et al., 2021).

- **El Mercado:** Se trata del contrato principal y se encarga de gestionar los activos, tanto de la creación de los nuevos contratos correspondientes a cada inmueble como de la compraventa de los mismos.

- **El Inmueble:** Es un contrato heredado del estándar ERC-20 (Ethereum Foundation, 2025a). Por cada nueva propiedad física que se registra en el sistema, el **Mercado** genera un contrato **Inmueble** único e independiente que representa el suministro total de *tokens* de esa propiedad concreta.

Capa de almacenamiento (IPFS) y oráculos: Como se detalló en la sección 4.2, IPFS garantiza la disponibilidad permanente de los documentos legales de cada inmueble tokenizado mediante identificadores de contenido (CID) fijados a través de Pinata. Se trata de un sistema de archivos externo que se incorpora ya directamente en los contratos de forma estructurada a través de un *gateway*, para poder así acceder fácilmente desde la información del propio *smart contract*.

Como se ha comentado en la Sección 4.1 utilizamos la **API pública de CoinGecko** (CoinGecko, 2024) como un oráculo de datos para obtener la tasa de cambio en tiempo real ETH/EUR e incorporar las cantidades según la divisa de interés en la capa que se detalla a continuación.

Capa de presentación: Desarrollada mediante React (Meta Platforms, Inc., 2023) y Ethers.js (Moore, 2023), actúa como el punto de entrada para los distintos perfiles de usuario (administrador e inversor). Esta capa es la encargada de *parsear* los datos crudos de la red y ejecutar las funciones del contrato de mercado sin acceder directamente a Remix IDE, sino a través de una aplicación Web3 más accesible y sencilla para usuarios que decidan comprar o vender *tokens* de alguna vivienda.

En la Figura 4.1 se puede observar la arquitectura y comunicación de las diferentes partes del proyecto.

A continuación se detalla la arquitectura de las diferentes operaciones que permite realizar la plataforma.

Flujo 1: Proceso de tokenización

1. El Administrador autenticado introduce los datos físicos y económicos de la propiedad en el *frontend*, incluyendo las direcciones que contienen tanto el CID del documento legal como el de una imagen de la vivienda.
 2. Se invoca la función de creación en el contrato del Mercado, el cual despliega un nuevo Contrato Inmueble asociado a esa propiedad.
 3. El Contrato Inmueble acuña la totalidad de los *tokens* establecidos y los transfiere automáticamente al contrato del Mercado, que almacenará todos los *tokens* de todas las propiedades registradas.
 4. Por último, el Mercado registra la dirección del nuevo contrato y su precio unitario en su base de datos interna y emite un evento de éxito para actualizar el *frontend* de forma asíncrona (Ethereum Foundation, 2024e).
-

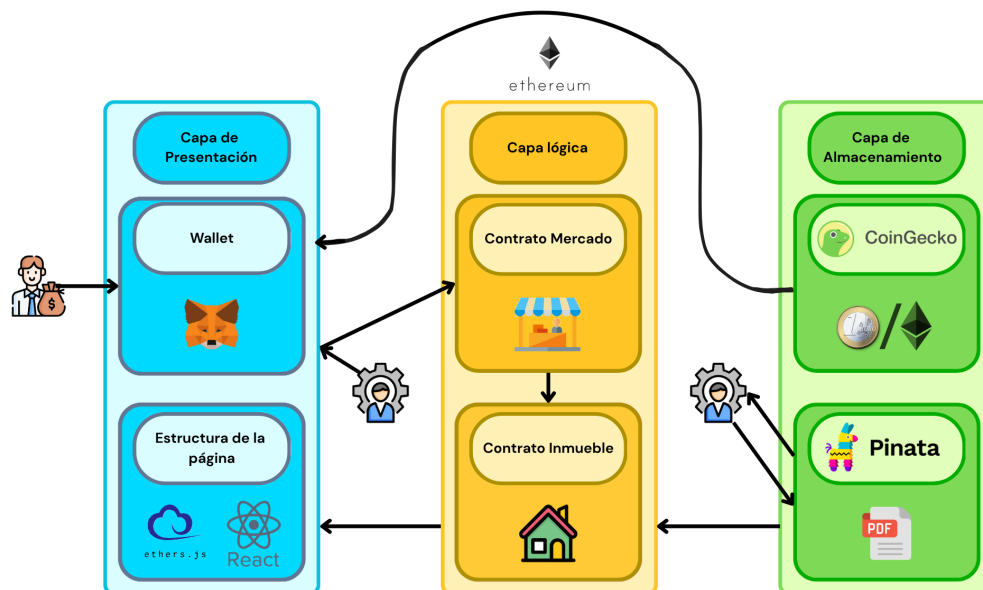


Figura 4.1.: Arquitectura del proyecto. De elaboración propia.

Flujo 2: Proceso de inversión: Compra de *tokens*

1. En la capa de presentación cualquier cuenta sin privilegios de administrador puede acceder únicamente al panel de inversión, donde puede consultar el catálogo, seleccionar una propiedad y definir la cantidad de participaciones a adquirir o a vender.
2. La transacción se emite al contrato del Mercado, el cual realiza de forma transparente la conversión del valor en Euros que se muestran en la interfaz a la cantidad exacta en la moneda nativa de la red (Ether/Wei).
3. El Mercado verifica el inventario disponible y una vez aprobada la transacción, transfiere la cantidad concreta de *tokens* de la vivienda correspondiente a la *wallet* del inversor.

Flujo 3: Proceso de venta

1. Cuando un inversor decide vender sus participaciones, al tratarse de *tokens* alojados en su propia *wallet*, el estándar de seguridad ERC-20 exige una operación estructurada en dos fases.

2. **Fase de aprobación:** El inversor firma una transacción invocando la función *approve* en el Contrato Inmueble, otorgando permisos temporales al Mercado para manipular la cantidad exacta de *tokens* a vender.
3. **Fase de liquidación:** El inversor invoca la función de **venta** en el Mercado. Este retira los *tokens* de la *wallet* del usuario utilizando el permiso previo (*transferFrom*) y los reincorpora a su inventario de *tokens* disponible.
4. Por último y de forma simultánea, el Mercado desbloquea la cantidad equivalente en Ether de sus reservas y la transfiere a la cuenta del inversor, completando así el ciclo.

4.4. Lógica de los *Smart Contracts*

En esta sección se describirán las funciones críticas y los patrones de diseño más relevantes de los dos *smart contracts* que actúan como la base del proyecto.

4.4.1. El contrato Inmueble

El contrato `ContratoInmueble` actúa como la representación digital y fraccionada de la propiedad física. Está diseñado para priorizar la seguridad y la inmutabilidad de los datos, y delega la gestión al contrato del mercado. A continuación, se detallan las decisiones clave de su implementación:

Herencia y estándares de seguridad: Se implementa el estándar de la industria **ERC-20** (Ethereum Foundation, 2025a) mediante la importación de las librerías auditadas de *OpenZeppelin* (OpenZeppelin, 2023). Esta decisión de arquitectura evita la programación manual de funciones clave (como `transfer` o `balanceOf`), garantizando que los *tokens* estén libres de vulnerabilidades y sean reconocidos de forma nativa por cualquier *wallet* de la red Ethereum.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 // Importamos el estándar ERC-20 de OpenZeppelin
5 import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
6
7 contract ContratoInmueble is ERC20 {
```

Código 4.1: Herencia y Estándares de Seguridad

Estado inmutable y trazabilidad: El contrato define variables de estado públicas (`direccionInmueble`, `ipfsDocumentoUrl`) e `ipfsImagenUrl` que vinculan el activo digital con el mundo real. Almacenar esta información en el contrato permite su auditoría y consulta pública ya que queda guardada de forma permanente y transparente en la *blockchain*.

Inicialización y despliegue: La configuración del activo se parametriza a través del constructor, ejecutado únicamente en el momento del despliegue. Para optimizar los costes de *Gas*, las cadenas de texto (**string**) recibidas como parámetros se asignan temporalmente a la memoria volátil de la máquina virtual (**memory**) antes de ser consolidadas en el almacenamiento permanente del contrato.

```

1 // Variables públicas para guardar los datos del mundo real
2 string public direccionInmueble;
3 string public ipfsDocumentoUrl; // URL con el CID del PDF de la escritura
4 string public ipfsImagenUrl; // URL con el CID de la imagen del inmueble
5
6 constructor(
7     string memory _nombre,           // Ej: "Piso Elche Altabix"
8     string memory _simbolo,          // Ej: "EAX"
9     uint256 _cantidadTotalTokens,    // Ej: 100000 (partes en las que
    dividimos la casa)
10    string memory _direccion,         // Ej: "Calle Sixto Marco 1, Número
    2, Planta 3, Puerta 4"
11    string memory _ipfsUrl,           // Ej: "ipfs://QmTuY..." (El hash del
    PDF)
12    string memory _ipfsImagenUrl,     // Ej: "ipfs://QmTuY..." (El hash de
    la imagen)
13    address _mercado                  // La dirección del contrato general
    que gestionará las ventas
14 ) ERC20(_nombre, _simbolo) {
15
16     direccionInmueble = _direccion;
17     ipfsDocumentoUrl = _ipfsUrl;
18     ipfsImagenUrl = _ipfsImagenUrl;

```

Código 4.2: Inicialización y Despliegue

Emisión de activos: En el instante de su creación, el contrato invoca la función interna `_mint` (OpenZeppelin, 2024b) para generar el suministro total. Estos *tokens* no son custodiados por el contrato del inmueble, sino que se emiten directamente a la dirección (`address`) del contrato del Mercado, almacenándose en las reservas de este y habilitando inmediatamente su comercialización. Dado que la Máquina Virtual de Ethereum (EVM) no procesa números de coma flotante, el contrato hereda la función `decimals()` (establecida en 18 por defecto). De este modo, al emitir, por ejemplo, 1.000 participaciones, el sistema inscribe criptográficamente la emisión de 1000×10^{18} unidades atómicas. De esta forma se garantiza la máxima precisión matemática en la división del activo, previniendo errores de redondeo durante la transferencia de participaciones. Se puede consultar el código completo del contrato en el Anexo A.

```

1 // Creamos todos los tokens y se los damos al contrato general (
    Mercado)
2 // Se multiplica por 10**decimals() porque blockchain no entiende de
    decimales reales
3 _mint(_mercado, _cantidadTotalTokens * 10 ** decimals());

```

Código 4.3: *Minting* de los *tokens*

4.4.2. El contrato Mercado: Lógica de *Factory* y gestión de liquidez

El contrato `MercadoInmobiliario` es el núcleo del sistema de tokenización y el único punto de interacción de los inversores con el mercado. Actúa simultáneamente como fábrica de nuevos activos, entidad bancaria de *tokens* y además ejecuta las transacciones. A continuación, se detalla el diseño y las decisiones lógicas implementadas:

Patrón *factory* e interfaces: Para optimizar el despliegue de activos, el mercado actúa como una “fábrica” (Zheng et al., 2021). Al importar el archivo `ContratoInmueble.sol`, el contrato adquiere la capacidad de instanciar nuevos contratos subordinados desde su propio código. Además, se importa la interfaz estándar `IERC20`, lo que permite al mercado interactuar financieramente con los *tokens* (enviar, recibir o aprobar) sin necesidad de cargar en memoria la implementación completa del estándar, optimizando el consumo de recursos computacionales.

Estructuras de datos y trazabilidad: Para gestionar el catálogo de inmuebles de forma eficiente, se ha diseñado una arquitectura de datos basada en estructuras personalizadas e identificadores únicos.

- **Agrupación lógica (`struct`):** Se define la estructura `DetallesInmueble` que contiene la dirección del contrato (`direccionContrato`), el precio en unidades atómicas o *Wei* (`precioPorToken`) y el inventario restante (`tokensDisponibles`).
- **Almacenamiento indexado:** Se implementa una tabla *hash* o *mapping* (Ethereum Foundation, 2024h) que relaciona la dirección única de cada contrato (*key*) con su ficha comercial (*value*), permitiendo así búsquedas y actualizaciones. Adicionalmente, un vector (`array`) almacena el registro ordenado de las direcciones para facilitar la iteración desde la aplicación cliente.

```

1 struct DetallesInmueble {
2     address direccionContrato;
3     uint256 precioPorToken;    // Valor en Wei
4     uint256 tokensDisponibles; // Inventario restante
5 }
6
7 mapping(address => DetallesInmueble) public inmuebles;
8 address[] public listaInmuebles;
```

Código 4.4: Definición de estructuras y almacenamiento del Mercado

Control de acceso y creación de activos: El control de acceso se establece en el constructor, asignando la variable global `msg.sender` (la dirección que despliega el contrato) como administrador único. Durante la función `crearInmueble`, una barrera de seguridad (`require`) valida esta identidad (Ethereum Foundation, 2024g) antes de invocar la instrucción `new`, la cual ordena a la *blockchain* desplegar el nuevo contrato.

Finalmente, se extrae el identificador del nuevo activo (**address**) para registrarlo en la base de datos y se emite un **evento** para notificar asíncronamente a la interfaz web (Ethereum Foundation, 2024e).

Gestión de la compra de *tokens*: Esta función incorpora el modificador **payable** (Ethereum Foundation, 2024f), habilitando la recepción de pagos con la criptomoneda nativa de la red (Ether). Para garantizar la inmutabilidad y exactitud de los registros, se aplican las siguientes lógicas:

- **Modificación permanente:** Mediante **DetallesInmueble storage**, se indica a la máquina virtual que las modificaciones posteriores afectarán directamente al estado permanente de la *blockchain*, y no a una copia temporal en memoria.
- **Barreras matemáticas:** Se aplican múltiples validaciones (**require**) (Ethereum Foundation, 2024g) para asegurar que la propiedad existe, que hay inventario suficiente (escalando la solicitud a los 18 decimales del ERC-20) y que los fondos enviados en la transacción (**msg.value**) cubren de forma estricta el coste total calculado.

```
1  function comprarTokens(address _tokenInmueble, uint256 _cantidadTokens)
2      public payable {
3      DetallesInmueble storage inmueble = inmuebles[_tokenInmueble];
4      require(inmueble.direccionContrato != address(0), "El inmueble no
5          existe");
6
7      uint256 cantidadTokensDec = _cantidadTokens * 10 ** 18;
8      require(inmueble.tokensDisponibles >= cantidadTokensDec, "Inventario
9          insuficiente");
10
11     uint256 costeTotal = inmueble.precioPorToken * _cantidadTokens;
12     require(msg.value >= costeTotal, "Fondos insuficientes");
13
14     inmueble.tokensDisponibles -= cantidadTokensDec;
15     IERC20(_tokenInmueble).transfer(msg.sender, cantidadTokensDec);
16 }
```

Código 4.5: Lógica de validación y compra de tokens

Gestión de la venta de *tokens*: La venta prioriza la seguridad del usuario y la integridad financiera del *smart contract*.

Para retirar los activos de la *wallet* del inversor, se invoca **transferFrom**. Esta función exige que el usuario haya firmado previamente una transacción de aprobación (**approve**), evitando retiradas no autorizadas. Tras actualizar el inventario y calcular el reembolso, se ejecuta la transferencia mediante una llamada de bajo nivel.

Al envolver **msg.sender** en **payable()**, se autoriza a la dirección a recibir fondos. Mediante **.call{value: pagoTotal}()** se envía la cantidad exacta de Ether sin ejecutar funciones adicionales. Esta operación devuelve un valor *booleano* (exito) que es

estrictamente evaluado mediante un `require` y si la transferencia falla por cualquier problema en la red, la transacción entera se revierte (*revert*), anulando el retiro de los *tokens* y protegiendo el patrimonio del usuario.

```

1      function venderTokens(address _tokenInmueble, uint256 _cantidadTokens)
2          public {
3              // [Validaciones y cálculo de decimales omitidos por brevedad]
4
5              // Retiro de tokens autorizado previamente por el usuario
6              IERC20(_tokenInmueble).transferFrom(msg.sender, address(this),
7                  cantidadTokensDec);
8              inmueble.tokensDisponibles += cantidadTokensDec;
9
10             uint256 pagoTotal = inmueble.precioPorToken * _cantidadTokens;
11
12             // Llamada de bajo nivel para la transferencia de Ether
13             (bool exito, ) = payable(msg.sender).call{value: pagoTotal}("");
14             require(exito, "Fallo al devolver el ETH");
15
16             emit TokensVendidos(msg.sender, _tokenInmueble, _cantidadTokens);
17         }

```

Código 4.6: Código de la función venderTokens

Actualización de precios: Por último, para adaptarse a las fluctuaciones del mercado inmobiliario real, el contrato implementa una función de revalorización (`modificarPrecio`). Mediante un modificador de acceso (`require`), el sistema garantiza que única y exclusivamente el administrador de la plataforma pueda alterar el precio unitario (`precioPorToken`) en el almacenamiento inmutable (`storage`), evitando así posibles ataques externos.

Al igual que con Contrato Inmueble, el código completo de `MercadoInmobiliario.sol` se puede consultar en el Anexo A.

4.5. Desarrollo de la interfaz de usuario (dApp)

La capa de presentación o *frontend* se ha desarrollado utilizando la biblioteca React (Meta Platforms, Inc., 2023). Actúa como el puente entre los inversores y los *smart contracts* desplegados en la *blockchain*. Su arquitectura se centra en la optimización de llamadas RPC (*Remote Procedure Call*) y la abstracción de los procesos relacionados con la *blockchain* mediante la librería `Ethers.js` (Moore, 2023). A continuación, se detallan los elementos más relevantes.

Gestión de autenticación y enrutamiento de red: Para el acceso a la plataforma se requiere que el usuario conecte una identidad digital (MetaMask) (ConsenSys, 2024). La función `conectarWallet` no solo solicita el acceso a las cuentas, sino que implementa el enrutamiento forzado para garantizar la seguridad, obligando al proveedor Web3 a cambiar a la red de pruebas Sepolia (`chainId: '0xaa36a7'`) (Ethereum Foundation,

2024b) para no utilizar fondos reales. Una vez establecida la conexión, se captura la dirección pública del usuario y se invoca la función de control de acceso (`verificarRol`) para determinar si se debe renderizar el panel de administración.

```

1  const conectarWallet = async () => {
2    setError('');
3    try {
4      if (window.ethereum) {
5        // Solicitud de acceso a la wallet del usuario
6        const cuentas = await window.ethereum.request({ method: '
          eth_requestAccounts' });
7
8        // Medida de seguridad: Forzar enrutamiento a Sepolia Testnet
9        try {
10         await window.ethereum.request({
11           method: 'wallet_switchEthereumChain',
12           params: [{ chainId: '0xaa36a7' }]
13         });
14       } catch (e) { /* Omisión silenciosa si la red ya es correcta */ }
15
16       setCuenta(cuentas[0]);
17       verificarRol(cuentas[0]); // Verificación de privilegios (RBAC)
18     } else {
19       setError('Instala MetaMask.');

```

Código 4.7: Inyección del proveedor y enrutamiento a Sepolia

Sincronización del estado *on-chain* y concurrencia: Para mantener el catálogo de inmuebles actualizado, se ha diseñado la función asíncrona `cargarEscaparate`. Dado que las llamadas RPC al *getter* público del *array* en Solidity requieren consultas individuales por índice, el sistema itera el registro del *smart contract* de forma secuencial hasta provocar una excepción (*out-of-bounds*). Este error se captura mediante una estructura `try...catch` para determinar dinámicamente el fin de la lista.

Para optimizar el tiempo de carga y no saturar el nodo RPC con peticiones secuenciales, se utiliza `Promise.all` (Mozilla Developer Network, 2024). De esta manera se ejecutan múltiples lecturas de la *blockchain* de forma concurrente, recopilando el nombre, símbolo, dirección y archivos IPFS simultáneamente antes de inyectarlos en el estado visual de React.

```

1  while (sigueBuscando) {
2    try {
3      // 1. Lectura del registro principal: Obtiene la dirección del
        contrato derivado (ERC-20).
4      const direccionInmueble = await contratoMercado.listaInmuebles(
        index);
5      // 2. Lectura del mapping de detalles comerciales (precio y stock
        disponible).

```

```

6      const detalles = await contratoMercado.inmuebles(direccionInmueble)
7      ;
8      // 3. Instanciación del contrato ERC-20 específico del inmueble
9      // para extraer sus metadatos.
10     const contratoCasa = new ethers.Contract(direccionInmueble,
11         abiInmueble, provider);
12
13     // Ejecución asíncrona concurrente (Promise.all) para optimizar los
14     // tiempos de respuesta (RPC calls).
15     const [n, s, d, docUrl, imgUrl] = await Promise.all([
16         contratoCasa.name(),
17         contratoCasa.symbol(),
18         contratoCasa.direccionInmueble(),
19         contratoCasa.ipfsDocumentoUrl(),
20         contratoCasa.ipfsImagenUrl()
21     ]);
22
23     // Mapeo y formateo de los datos crudos de la EVM al modelo de
24     // datos del frontend.
25     listaCasas.push({
26         direccion: direccionInmueble,
27         nombre: n,
28         simbolo: s,
29         direccionFisica: d,
30         ipfsDocumentoUrl: docUrl,
31         ipfsImagenUrl: imgUrl,
32         precioUnitarioWei: detalles.precioPorToken,
33         // Normalización de enteros de precisión arbitraria (BigInt)
34         // eliminando los 18 decimales nativos.
35         tokensDisponibles: detalles.tokensDisponibles / 10n ** 18n
36     });
37     index++;
38 } catch (err) {
39     sigueBuscando = false;
40 }
41 }

```

Código 4.8: Lectura de datos concurrentes desde la *blockchain*

Capa de abstracción de transacciones: La aplicación delega todas las operaciones de escritura (crear activos, modificar precios, comprar o vender) a una Función de Orden Superior (*Higher-Order Function*) denominada `ejecutarTx`.

Este patrón de diseño centraliza la lógica repetitiva del ciclo de vida de una transacción en Web3 (Ethereum Foundation, 2024a). Evita la recarga del navegador, inyecta el **Signer** (la firma criptográfica del usuario) en el *callback* proporcionado, maneja los estados visuales de carga (*“Ejecutando orden...”*) y, al finalizar, ordena una recarga automática del estado global (`cargarEscaparate`) para que la interfaz refleje inmediatamente los cambios de la red.

```

1      const ejecutarTx = async (e, callbackAction, mensajeCarga, mensajeExito) =>
2      {

```

```

2   if (e && e.preventDefault) e.preventDefault();
3   setEstadoTx(mensajeCarga); setError('');
4
5   try {
6     // Petición del Signer a MetaMask
7     const provider = new ethers.BrowserProvider(window.ethereum);
8     const signer = await provider.getSigner();
9
10    // Ejecución de la función específica (Comprar, Vender, Tokenizar...)
11    await callbackAction(signer);
12
13    setEstadoTx(mensajeExito);
14    setTimeout(() => setEstadoTx(''), 4000);
15    cargarEscaparate();
16  } catch (error) {
17    setError(traducirErrorWeb3(error));
18    setEstadoTx('');
19  }
20 };

```

Código 4.9: Función de enrutamiento y abstracción de transacciones

Experiencia de usuario y parseo de errores: La Máquina Virtual de Ethereum (EVM) devuelve excepciones muy técnicas y con codificaciones complejas en formato JSON. Mostrar estos mensajes directamente al usuario final afectaría negativamente a la experiencia de uso. Para solventarlo, se ha implementado la función `traducirErrorWeb3`, que intercepta los rechazos de firma o los errores lógicos del contrato (como insuficiencia de fondos o fallos de `require`) y los traduce a advertencias comprensibles en la interfaz.

Renderizado condicional de la interfaz: El *Document Object Model* (DOM) de la aplicación muta en tiempo real dependiendo de la información extraída de la *blockchain*. En la función de renderizado (`return`), se aplica un patrón de diseño condicional estricto:

- **Control de sesión (!cuenta):** Si la variable de estado `cuenta` es nula, se oculta toda la plataforma y se renderiza exclusivamente la pasarela de conexión.
- **Renderizado iterativo:** Se utiliza la función `.map()` sobre el *array* `escaparate` para transformar dinámicamente los datos puros en componentes visuales (tarjetas de propiedades), aplicando conversiones en tiempo real del precio en *Wei* a un valor aproximado en Euros gracias al oráculo de precios.

```

1   return (
2     <div style={S.body}>
3       /* Si no hay cuenta conectada mostramos el botón de conexión */
4       {!cuenta ? (
5         <button onClick={conectarWallet} style={S.btnPrimary}>
6           Conectar MetaMask
7         </button>

```

```

8      ) : (
9          /* Si hay cuenta, renderizamos la logica de la DApp */
10         <div>
11             <div style={S.badge}>Wallet: {cuenta}</div>
12
13             {/* Zona exclusiva de Administrador */}
14             {esAdmin && (
15                 <h2 style={S.sectionTitle}>Panel de Gestion de Activos</h2>
16                 /* Formularios de creacion y precios ... */
17             )}
18
19             {/* Zona de inversores accesible para cualquier cuenta... */}
20         </div>
21     )}
22 </div>
23 );

```

Código 4.10: Renderizado condicional del DOM según estado de sesión

4.6. Despliegue y verificación de contratos

La última fase del desarrollo consiste en migrar cada componente desde el entorno local a una infraestructura pública. Este proceso es el que termina la arquitectura de la DApp, garantizando que el sistema sea accesible, auditable y operativo para los usuarios finales.

Compilación y despliegue *On-Chain*: El despliegue de los *smart contracts* se ha gestionado a través de **Remix IDE**. Tras optimizar el código mediante el compilador de Solidity (reduciendo así los costes futuros de *Gas* asociados a la escritura en la base de datos distribuida), se utilizó la inyección del proveedor Web3 (*Injected Provider*) para enlazar el entorno de desarrollo con la *wallet* del administrador en MetaMask.

El despliegue inicial del contrato **MercadoInmobiliario** se firmó y transmitió a **Sepolia**, que permite simular condiciones de producción reales como los tiempos de minado o consumo de *Gas* utilizando la criptomoneda de prueba Sepolia ETH, ya que es una red de pruebas (*testnet*) de Ethereum.

Verificación y transparencia (Etherscan): Una vez confirmada la creación del contrato en la red, se validó públicamente el código mediante el explorador de bloques **Sepolia Etherscan** (Etherscan, 2024). Este proceso compara el código original con el *bytecode* alojado inmutablemente en la *blockchain*.

- **Auditoría pública:** Al lograr una coincidencia exacta, Etherscan certifica el contrato, permitiendo a cualquier inversor o entidad auditora leer la lógica de la plataforma y garantizar la ausencia de vulnerabilidades que puedan llevar a un uso malicioso del sistema. Se puede comprobar en: <https://sepolia.etherscan.io/address/0x3A679093fDf1d9F1dDCca168021297AD42d1CBa8>.

- **Extracción del ABI:** La verificación publica automáticamente el ABI (*Application Binary Interface*) del contrato (Ethereum Foundation, 2024d), lo que permite a aplicaciones de terceros interactuar con la plataforma y proporciona interoperabilidad al contrato.

Despliegue de la Aplicación Cliente (Vercel): Para completar la arquitectura y permitir el acceso a los inversores, la capa de presentación desarrollada en React (*frontend*) fue compilada y desplegada en la infraestructura en la nube de **Vercel**.

Como Plataforma como Servicio (PaaS), Vercel se encarga de empaquetar el código fuente, optimizar los recursos estáticos y distribuirlos a través de una Red de Entrega de Contenido (CDN) global. El resultado es el alojamiento de la interfaz en una URL pública (<https://<nombre-del-proyecto>.vercel.app>).

Mediante este despliegue, la plataforma adquiere alta disponibilidad, enrutamiento seguro bajo el protocolo criptográfico HTTPS, y permite a cualquier usuario interactuar con el mercado inmobiliario descentralizado conectando su *wallet* desde cualquier navegador compatible. Veremos en la sección 5.3 cómo es la experiencia de uso al utilizar la aplicación. La plataforma se encuentra disponible en: <https://web-tokenizacion.vercel.app/>

5. Pruebas y análisis de resultados

En este capítulo se evalúa empíricamente la solidez, seguridad y eficiencia de la arquitectura de datos desarrollada. Tras la fase de diseño e implementación descrita en el capítulo anterior, es conveniente someter la plataforma a un proceso de validación.

Para ello, el análisis se ha estructurado en tres partes: la validación de la lógica de negocio mediante pruebas unitarias, el análisis del coste computacional (*Gas*) asociado a la escritura en la *blockchain*, y la evaluación de la interfaz orientada al inversor final.

5.1. Validación de funciones (Test unitarios)

La inmutabilidad de la *blockchain* exige que el código esté libre de errores, ya que una vez instanciado, no puede ser modificado de forma tradicional. Para garantizar la integridad de la plataforma, se han diseñado pruebas unitarias con el objetivo de validar las transiciones de estado de los *smart contracts* y la reversión de transacciones (Ethereum Foundation, 2024i).

Las pruebas se han ejecutado aislando los componentes y simulando interacciones desde múltiples *wallets* en un entorno de desarrollo local haciendo uso del *framework* **Hardhat** (Nomic Foundation, 2024). Los *test* desarrollados se han clasificado en tres categorías:

Control de acceso basado en roles (RBAC): El objetivo de estas pruebas es garantizar que la ingesta de nuevos datos en la *blockchain*, es decir, el registro de nuevas viviendas, está completamente limitada al administrador (OpenZeppelin, 2024a).

- **Caso de éxito:** Se simula la invocación de la función `crearInmueble` desde la cuenta del administrador. El sistema valida la firma, genera el contrato derivado y actualiza el diccionario (*mapping*) correctamente.
- **Caso de fallo (Rechazo esperado):** Se invoca la misma función, así como la función `modificarPrecio`, desde una cuenta inversora estándar. La Máquina Virtual de Ethereum (EVM) intercepta la llamada y revierte la transacción instantáneamente, protegiendo al mercado de manipulaciones no autorizadas.

Integridad matemática y financiera: Se deben validar también la precisión de las operaciones de compra y de venta de *tokens*.

- **Precisión de decimales:** Se verifica que la adquisición de *tokens* escale correctamente a los 18 decimales nativos del estándar ERC-20 sin errores de redondeo.

- **Bloqueo por fondos insuficientes:** Se simula una compra enviando un valor en *Wei* (`msg.value`) estrictamente inferior al coste total calculado. El sistema evalúa la condición de límite y niega el cambio del estado, impidiendo la emisión de *tokens* sin la cantidad económica necesaria.

Cambio de estado y lógica de inventario: Estas pruebas certifican que tras una transacción los datos reflejan la realidad y todo tiene sentido lógico.

- **Sincronización bidireccional:** Al ejecutar la función `venderTokens`, se valida simultáneamente que el saldo de *tokens* del usuario disminuya, que el inventario del mercado aumente en la misma proporción exacta, y que la reserva del contrato libere la cantidad correspondiente (Ether) hacia la cuenta del inversor. Si cualquiera de estas tres operaciones falla, la transacción se revierte por completo.

Tras la ejecución de las pruebas, se obtuvo una tasa de éxito del 100 %. En la Figura 5.1 se puede comprobar la salida por consola del entorno de simulación que certifica la validación del sistema.

```
Pruebas Unitarias y de Seguridad: Mercado Inmobiliario
Control de acceso basado en roles
✓ Caso de éxito: El administrador crea un inmueble
✓ Caso de fallo: Un inversor estándar NO puede crear inmuebles
✓ Caso de fallo: Un inversor estándar NO puede modificar el precio
Integridad matemática y financiera
✓ Bloqueo por fondos insuficientes en la compra
✓ Precisión de decimales: Compra exitosa y escalado a 18 decimales
Cambio de estado y lógica de inventario
✓ Sincronización bidireccional (Tokens, Inventario y Liquidez ETH) en la venta (46ms)

6 passing (648ms)
```

Figura 5.1.: Resultados en consola de los *test* unitarios. De elaboración propia.

5.2. Análisis de eficiencia y costes de Gas

En la red Ethereum, el almacenamiento y el cálculo son recursos cotizados. Cada instrucción ejecutada por la EVM consume una cantidad de *Gas* proporcional a su complejidad. En esta sección se analiza la eficiencia de los contratos desarrollados, diferenciando entre costes de despliegue y costes operativos.

Para comprender el impacto económico de la plataforma, es necesario recordar cómo se calcula la tarifa de transacción (conocida como *Gas Fee*). El coste total en Ether (ETH) de cualquier operación viene dado por la siguiente ecuación (Ethereum Foundation, 2024c):

$$\text{Coste Total (ETH)} = \text{Gas Usado} \times \text{Precio del Gas (Gwei)} \quad (5.1)$$

El **Gas Usado** es una constante técnica, ejecutar la función `venderTokens` siempre consumirá el mismo volumen de operaciones computacionales, independientemente del

estado del mundo. Sin embargo, el **Precio del Gas**, medido en *Gwei* (10^{-9} ETH), es una variable dinámica determinada por el libre mercado y la congestión de la red (Ethereum Foundation, 2024c).

Dado que como se ha descrito en el Capítulo 2 el espacio dentro de cada bloque de Ethereum es limitado, la red opera bajo un modelo de subasta. Durante periodos de alto tráfico, por ejemplo, cuando hay picos de volatilidad en el mercado o lanzamientos en masa de nuevos *tokens*, los usuarios compiten entre sí ofreciendo un **Precio del Gas** más alto para incentivar a los validadores a incluir sus transacciones primero. Por el contrario, en momentos de baja actividad, el Precio del *Gas* desciende a su tarifa base.

Para auditar el consumo de cada función se ha utilizado la herramienta **Hardhat Gas Reporter** (Cgewecke, 2025). Las mediciones se han realizado sobre la versión *0.8.20* del compilador de *Solidity* con la optimización activada. En la Tabla 5.1 se muestran los resultados obtenidos:

Contrato / Método	Tipo de Operación	Consumo Promedio (<i>Gas</i>)
MercadoInmobiliario	Despliegue (Deploy)	2.925.111
crearInmueble	Creación de Activo	1.129.413
comprarTokens	Transacción (Inversor)	68.428
venderTokens	Transacción (Inversor)	59.496
approve	Permiso (ERC-20)	46.952

Tabla 5.1.: Perfil de consumo computacional (*Gas*) de los *smart contracts*.

El despliegue de la infraestructura inicial representa el mayor gasto individual del sistema. El contrato **MercadoInmobiliario** actúa como un registrador/fábrica de propiedades, requiriendo 2.925.111 unidades de *Gas* para su instanciación en la red. Asimismo, la creación de un nuevo activo (**crearInmueble**) consume 1.129.413 unidades de *Gas*, un coste asumible al tratarse de una operación que solo se realiza una vez por inmueble.

El análisis de las funciones transaccionales permite estudiar la viabilidad comercial y la escalabilidad del proyecto. Las operaciones de compra (**comprarTokens**) y venta (**venderTokens**) consumen en promedio 68.428 y 59.496 unidades de *Gas* respectivamente. Se trata de una cantidad bastante baja que nos indica lo bien optimizadas que están las funciones. Esta eficiencia se debe a que las operaciones de compraventa no requieren crear nuevos datos en la red, sino únicamente actualizar los balances del contrato ERC-20 ya existente, lo cual representa una carga computacional mínima.

Estimación de costes en moneda fiat: Para poner en contexto sobre el impacto económico real, se ha realizado una estimación utilizando un precio promedio de red de 20 *Gwei* y una cotización de 3.000 EUR/ETH. Bajo estos parámetros, un inversor puede liquidar sus participaciones (**venderTokens**) asumiendo un coste de red aproximado de 3,57 EUR. Este coste extremadamente inferior a los gastos notariales, registrales y de intermediación bancaria propios de la transmisión de bienes raíces de forma tradicional, respaldando la eficiencia financiera de la tokenización.

5.3. Evaluación de la experiencia de uso y eficiencia del proceso

Tras comprobar la solidez de los *smart contracts* y analizar la viabilidad económica en términos de consumo de *Gas*, esta sección se centra en evaluar la aplicación desarrollada. El objetivo es determinar en qué medida el sistema propuesto optimiza la experiencia del inversor y mejora la eficiencia del mercado inmobiliario tradicional.

5.3.1. Experiencia de usuario en la dApp

La interfaz de usuario desarrollada actúa como una capa de abstracción que oculta la complejidad técnica de la *blockchain*, permitiendo una interacción fluida. Cuando cualquier usuario se conecta a la plataforma el primer paso a realizar es la conexión de su cuenta de Metamask.



Figura 5.2.: Interfaz de la aplicación antes de iniciar sesión. De elaboración propia.

Una vez se inicia sesión con Metamask se tiene acceso a toda la lista de inmuebles tokenizados disponibles para operar con ellos, tal y como se ve en la Figura 5.3.

Mercado de Inmuebles Tokenizados

Opera con fracciones de Inmuebles Registrados en la Blockchain

Comunidad: Administrador | Wallet: 0xc964...f9d1

Propiedades Disponibles



Piso Elche Zona Estadio

Avenida de Alonso Número 14, Piso 3,
Puerta 3

[Ver Título de Propiedad](#)

2.38 €

100000 / ELC1

Seleccionar



Piso Gran Vía

Calle de la Gran Vía, Número 32, Piso
5, Puerta A

[Ver Título de Propiedad](#)

7.57 €

100000 / GV-1

Seleccionar



Casa Rural Los Olivos, Albacete

Paraje "Los Olivos", Camino de la
Ribera s/n, Polígono 4, Parcela 12.

[Ver Título de Propiedad](#)

3.79 €

200000 / CRA1

Seleccionar



Local en Alicante Explanada

Explanada de España, Número 10,
Planta 2ª, de Alicante

[Ver Título de Propiedad](#)

4.74 €

100000 / L-AL1

Seleccionar



Nave industrial Parque Empresarial Elche

Elche Parque Empresarial, Calle
Severo Ochoa, Número 15.

[Ver Título de Propiedad](#)

4.58 €

100000 / N-EL1

Seleccionar



Local de Fiesta en Elche Zona Universidad

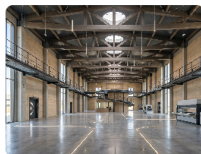
Avenida de la Universidad, Número
50, Planta Baja.

[Ver Título de Propiedad](#)

3.55 €

100000 / L-EL1

Seleccionar



Nave industrial de lujo en Elche

Elche Parque Empresarial, Bulevar del
Calzado, Número 10.

[Ver Título de Propiedad](#)

7.74 €

100000 / N-EL2

Seleccionar

Figura 5.3.: La aplicación mostrando los inmuebles disponibles. De elaboración propia.

Existe la opción de seleccionar cualquier inmueble para ver su imagen ampliada simplemente haciendo click sobre la misma. Además, si se selecciona “Ver título de propiedad”, se accede directamente a la escritura del inmueble correspondiente que se encuentra en IPFS.

Centrándose en la operación con las propiedades, cuando se selecciona un inmueble, se copia la dirección del contrato automáticamente en el panel de inversión, donde se permite seleccionar la cantidad a comprar o vender y efectuar la operación. Se debe distinguir que si el acceso a la aplicación se hace mediante la cuenta de administrador aparecerá también el panel de gestión de inmuebles, tal y como se ve en la Figura 5.4.

The image displays two side-by-side panels from a web application. The left panel, titled 'Panel de Gestión', contains a section for 'Nueva Tokenización' with input fields for 'Nombre (Piso Gran Vía)', 'Símbolo (GV-1)', 'Tokens', 'Dirección (Avenida...)', 'URL de Escritura (IPFS)', 'URL de Imagen (IPFS)', and 'Valor Total (€)' (with a small '≈ 0 ETH' indicator). A 'Crear Activo' button is at the bottom. Below this is a 'Revalorización' section with fields for 'Dirección Inmueble (0x...)' and 'Nuevo Valor Total (€)', followed by an 'Actualizar Precio' button. The right panel, titled 'Panel de Inversión', has the subtitle 'Adquiere o vende fracciones de propiedad en tiempo real.' It features a text input field containing a hexadecimal address, a 'Cantidad de Participaciones' input field, and two buttons: 'Comprar' (highlighted in black) and 'Vender'.

Figura 5.4.: Interfaz de la aplicación para la *wallet* del administrador. De elaboración propia.

Si por el contrario el acceso se realizara con cualquier cuenta genérica, este panel no aparecería, disponiendo únicamente de la posibilidad de comprar o vender tokens del inmueble seleccionado.

Figura 5.5.: Interfaz de la aplicación para inversores. De elaboración propia.

Para la experiencia de uso existen tres factores muy importantes a tener en cuenta:

- **Accesibilidad y autogestión:** Mediante la integración de *wallets* como Meta-mask, el usuario tiene el control total de sus activos sin depender de credenciales centralizadas. Además, el proceso de “Login” se simplifica a una firma digital, garantizando la seguridad y la privacidad.
- **Transparencia inmediata:** La *dApp* permite consultar de forma directa la dirección del contrato de cada inmueble y su documentación legal asociada en IPFS. Una clara ventaja con respecto al sector inmobiliario tradicional donde todo es mucho más opaco.
- **Interacción en tiempo real:** Las funciones de compra y venta se pueden ejecutar en cualquier momento en unos pocos segundos, haciendo que la transferencia de valor sea casi instantánea tras la confirmación del bloque.

Cabe destacar que, durante la ejecución de operaciones de escritura en la *blockchain* (como la compra o venta de *tokens*), la *dApp* delega la firma criptográfica al proveedor Web3. Esto genera una ventana emergente donde el inversor puede auditar los datos del contrato y visualizar el coste exacto de (*Gas fee*) antes de confirmar la operación.

5.3.2. Análisis de eficiencia

Para cuantificar la mejora que supone esta tecnología, se presenta una comparativa entre el modelo de inversión inmobiliaria convencional y la solución basada en *smart contracts* desarrollada en este trabajo (ver Tabla 5.2).

Como se observa en la comparativa, la arquitectura propuesta reduce los principales cuellos de botella del mercado tradicional. La reducción de la barrera de entrada democratiza el acceso a la inversión inmobiliaria, mientras que la automatización y la inmediatez de las transacciones aportan un grado de liquidez sin precedentes a un activo históricamente ilíquido. De este modo, la plataforma no solo optimiza los procesos existentes, sino que habilita un nuevo modelo de mercado más ágil

Parámetro	Modelo Tradicional	Plataforma de Tokenización
Barrera de entrada	Generalmente alto (se requiere gran capital)	Muy bajo (fraccionamiento desde 1 <i>token</i>)
Tiempo de cierre	Semanas o meses (notaría, registro, intermediarios)	Segundos/Minutos
Disponibilidad	Horario comercial y dependencia de citas físicas	24/7, acceso global sin restricciones geográficas
Transparencia	Información fragmentada y de terceros	Registro público, inmutable y auditable <i>on-chain</i>
Liquidación de rentas	Procesos manuales, mensuales o trimestrales.	Automatizable mediante <i>smart contracts</i>

Tabla 5.2.: Comparativa de eficiencia: Modelo Tradicional vs. Plataforma de Tokenización

y accesible. El proyecto de la aplicación se puede consultar al completo en: <https://github.com/nicovives/TFM-TokenizacionRWA>.

6. Conclusiones

El desarrollo de este trabajo ha demostrado la viabilidad de integrar la tecnología *Blockchain* en el sector inmobiliario, planteando una solución real a problemas tradicionales como la falta de liquidez, las altas barreras de entrada y la opacidad administrativa. A través del diseño e implementación de una Aplicación Descentralizada (*DApp*), se ha logrado construir un ecosistema completo de tokenización de activos físicos.

Mediante el uso de *smart contracts* y el estándar ERC-20, se ha visto cómo se puede dividir el valor de un inmueble en *tokens* digitales, reduciendo drásticamente el capital mínimo necesario para participar en el mercado inmobiliario, abriendo la puerta a pequeños y medianos inversores. La separación de la lógica en dos capas (un contrato “fábrica” para el mercado general y contratos individuales para cada inmueble) ha demostrado ser un patrón de diseño eficiente. Además, junto con la interfaz desarrollada en React y conectada mediante ethers.js, se ha logrado una experiencia de usuario fluida y accesible para el público general, diferenciando claramente los roles de administrador e inversor. La integración del sistema de archivos IPFS a través de los *gateways* de Pinata ha permitido almacenar datos no estructurados en la Web3 y al registrar las URLs exactas de las escrituras legales y las fotografías de las viviendas en la *blockchain*, se garantiza que cualquier intento de manipulación de archivos altere el Content Identifier (CID), siendo rechazado de inmediato por la red.

La funcionalidad implementada para la compraventa de participaciones en tiempo real sienta las bases de un mercado secundario líquido. Además de los beneficios directos para el inversor, la arquitectura desarrollada pone de manifiesto una mejora en la eficiencia operativa del sector. Al minimizar la dependencia de intermediarios tradicionales —como entidades bancarias para la financiación, cámaras de compensación y ciertos procesos burocráticos— se reducen drásticamente los tiempos y los costes asociados a la transferencia de la propiedad. La lógica de los *smart contracts* garantiza resultados deterministas, reduciendo el riesgo de error humano y agilizando los intercambios en el mercado secundario.

Desde el punto de vista del rendimiento técnico, el análisis de eficiencia y de costes de *Gas* realizados durante la fase de pruebas respaldan la escalabilidad del sistema. El patrón *Factory* además de optimizar la lógica del código ha demostrado mantener los costes de despliegue y de transacciones dentro de un rango muy inferior al del mercado tradicional. Además, operando bajo el actual consenso *Proof-of-Stake* (PoS) de la red, la plataforma garantiza una huella energética mínima, juntando innovación tecnológica con la sostenibilidad, aspectos esenciales hoy en día.

Por otro lado, el registro de las transacciones en una red pública y descentralizada redefine el concepto de auditoría y trazabilidad. A diferencia de los registros de la propiedad convencionales, que a menudo pueden presentar procesos lentos u opacos, la

solución propuesta ofrece un libro general inmutable y auditable en tiempo real. Esto no solo genera una confianza sin precedentes, sino que también sirve como referencia sobre cómo los sistemas de registro de activos físicos pueden evolucionar hacia modelos Web3.

No obstante, en el ámbito regulatorio, la tokenización de activos del mundo real (RWA) plantea un cambio de paradigma que requiere una adaptación de la legislación vigente. La necesidad de clasificar estos activos como *Security Tokens* según el marco legal español (Jefatura del Estado, 2023) y la normativa europea MiCA (Parlamento Europeo y Consejo de la Unión Europea, 2023) indica que el siguiente gran reto es la creación de puentes legales que permitan a los *smart contracts* interactuar de forma vinculante con notarías y registros de la propiedad.

En definitiva, el auge de la tokenización de activos físicos no es un simple avance informático, sino una respuesta directa a las ineficiencias económicas heredadas de los sistemas financieros tradicionales. Al devolver al ciudadano la capacidad de acceder a activos refugio de forma directa, transparente y auditable, se reducen los efectos de la exclusión financiera. Este Trabajo de Fin de Máster no solo valida un marco tecnológico funcional, sino que materializa el concepto fundamental de la Web3: la transición hacia una economía verdaderamente descentralizada, eficiente y democratizada.

Líneas de trabajo futuro

El sistema desarrollado es plenamente funcional en la red de pruebas (Sepolia), pero el paso hacia un entorno de producción (Mainnet) abre diversas líneas de investigación y desarrollo futuro.

Para operar en el marco legal actual (como el reglamento MiCA en Europa), sería necesario integrar un sistema de Identidad Descentralizada (DID) o un proceso de verificación de identidad (KYC) que mediante *whitelists* autorice únicamente a carteras validadas a interactuar con los tokens. Actualmente, la revalorización de los inmuebles es gestionada manualmente por el administrador. Una futura integración podrían ser los oráculos descentralizados (como Chainlink) combinados con APIs de tasadoras inmobiliarias para automatizar el ajuste de precios basado en el mercado real. En el sistema desarrollado las transacciones y la tasación de las participaciones están ligadas al *token* nativo de la red (Ether). Sin embargo, en un escenario de tokenización real y comercial, la alta volatilidad de las criptomonedas tradicionales resulta incompatible con la naturaleza conservadora y estable del mercado inmobiliario. Como mejora se propone migrar el sistema de pagos para que opere exclusivamente con stablecoins (criptomonedas respaldadas por moneda fiduciaria, como USDC, USDT o EUROCC) o futuras Monedas Digitales de Bancos Centrales (CBDC). Esta transición permitiría a los inversores adquirir fracciones de vivienda utilizando representaciones digitales del Euro o el Dólar, separando el valor del inmueble de las fluctuaciones del mercado de criptomonedas, pero conservando la liquidación instantánea, la seguridad y la inmutabilidad que provee la *blockchain*. Queda además como posible ampliación la programación de una lógica de reparto de dividendos dentro del *smart contract*, permitiendo distribuir automáticamente los ingresos generados por el alquiler del inmueble físico entre los tenedores de los tokens, de manera proporcional a su participación.

Bibliografía

- Back, A. (2002). Hashcash - A Denial of Service Counter-Measure. Consultado el 9 de diciembre de 2025, desde <http://www.hashcash.org/papers/hashcash.pdf>
- Benet, J. (2014). IPFS - Content Addressed, Versioned, P2P File System [Foundational whitepaper for the InterPlanetary File System]. *arXiv preprint arXiv:1407.3561*. Consultado el 4 de marzo de 2026, desde <https://arxiv.org/abs/1407.3561>
- Bitcorati Inc. (2014, enero). *Interview with Vitalik Buterin* [Archivado del original el 17 de febrero de 2015]. Consultado el 15 de febrero de 2026, desde <https://web.archive.org/web/20150217161840/https://www.youtube.com/watch?v=EkeCapBOQQ0>
- Brickken. (2025). Digital Asset Protocol (DAP): Technical Documentation. Consultado el 7 de febrero de 2026, desde <https://docs.brickken.com>
- Buterin, V. (2014). Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. Consultado el 18 de enero de 2026, desde <https://ethereum.org/en/whitepaper/>
- Buterin, V., & Griffith, V. (2017). Casper the Friendly Finality Gadget. *arXiv preprint arXiv:1710.09437*. Consultado el 18 de enero de 2026, desde <https://arxiv.org/abs/1710.09437>
- Buterin, V., Hernandez, D., Kamphofner, T., Pham, K., Qiao, Z., Ryan, D., Sin, J., Wang, Y., & Zhang, Y. X. (2020). Combining GHOST and Casper. *arXiv preprint arXiv:2003.03052*. Consultado el 18 de enero de 2026, desde <https://arxiv.org/abs/2003.03052>
- Caballero, M. (2019). *Bitcoin. Blockchain y Tokenización para inquietos*. Bubok.
- Cgewecke. (2025). hardhat-gas-reporter: Gas Usage Analytics for Hardhat. Consultado el 12 de abril de 2026, desde <https://github.com/cgewecke/hardhat-gas-reporter>
- Chaum, D., Fiat, A., & Naor, M. (1990). Untraceable Electronic Cash. En S. Goldwasser (Ed.), *Advances in Cryptology — CRYPTO' 88* (pp. 319-327). Springer New York.
- CoinGecko. (2024). *CoinGecko API Documentation: The World's Most Accurate Cryptocurrency Data API*. Consultado el 15 de marzo de 2026, desde <https://www.coingecko.com/en/api/documentation>

- ConsenSys. (2024). *MetaMask Documentation: A crypto wallet and gateway to blockchain apps*. Consultado el 12 de marzo de 2026, desde <https://docs.metamask.io/>
- Dai, W. (1998). B-money. Consultado el 10 de diciembre de 2025, desde <https://nakamotoinstitute.org/library/b-money/>
- De Filippi, P., & Hassan, S. (2016). Blockchain technology as a regulatory technology: From code is law to law is code. *First Monday*, 21(12). <https://doi.org/10.5210/fm.v21i12.7113>
- ERC-3643 Association. (2025). *ERC-3643: The Standard for Real-World Asset Tokenization*. Consultado el 11 de febrero de 2026, desde <https://docs.erc3643.org/erc-3643>
- Ethereum Foundation. (2024a). *Design and UX: Best practices for building decentralized applications*. Consultado el 1 de abril de 2026, desde <https://ethereum.org/en/developers/docs/design-and-ux/>
- Ethereum Foundation. (2024b). *Ethereum Documentation: Networks and Testnets*. Consultado el 1 de abril de 2026, desde <https://ethereum.org/en/developers/docs/networks/>
- Ethereum Foundation. (2024c). *Gas and Fees in Ethereum*. Consultado el 15 de abril de 2026, desde <https://ethereum.org/en/developers/docs/gas/>
- Ethereum Foundation. (2024d). *Solidity Documentation: Contract ABI Specification*. Consultado el 5 de abril de 2026, desde <https://docs.soliditylang.org/en/latest/abi-spec.html>
- Ethereum Foundation. (2024e). *Solidity Documentation: Contracts - Events*. Consultado el 22 de marzo de 2026, desde <https://docs.soliditylang.org/en/v0.8.20/contracts.html#events>
- Ethereum Foundation. (2024f). *Solidity Documentation: Contracts (Creating Contracts and Modifiers)*. Consultado el 23 de marzo de 2026, desde <https://docs.soliditylang.org/en/v0.8.20/contracts.html>
- Ethereum Foundation. (2024g). *Solidity Documentation: Error handling: Assert, Require, Revert and Exceptions*. Consultado el 23 de marzo de 2026, desde <https://docs.soliditylang.org/en/v0.8.20/control-structures.html#error-handling-assert-require-revert-and-exceptions>
- Ethereum Foundation. (2024h). *Solidity Documentation: Types - Mapping Types*. Consultado el 23 de marzo de 2026, desde <https://docs.soliditylang.org/en/v0.8.20/types.html#mapping-types>
- Ethereum Foundation. (2024i). *Solidity Documentation: Unit Testing and Smart Contract Security*. Consultado el 14 de abril de 2026, desde <https://docs.soliditylang.org/en/v0.8.20/security-considerations.html>
-

- Ethereum Foundation. (2025a). Estándar de token ERC-20. Consultado el 2 de febrero de 2026, desde <https://ethereum.org/es/developers/docs/standards/tokens/erc-20/>
- Ethereum Foundation. (2025b). Estándar de token no fungible ERC-721. Consultado el 4 de febrero de 2026, desde <https://ethereum.org/es/developers/docs/standards/tokens/erc-721/>
- Ethereum Foundation. (2025c). Ethereum Virtual Machine (EVM). Consultado el 18 de enero de 2026, desde <https://ethereum.org/developers/docs/evm/>
- Ethereum Foundation. (2025d). La historia de Ethereum. Consultado el 17 de enero de 2026, desde <https://ethereum.org/es/ethereum-forks/>
- Ethereum Foundation. (2025e). Proof-of-stake (PoS). Consultado el 18 de enero de 2026, desde <https://ethereum.org/developers/docs/consensus-mechanisms/pos/>
- Etherscan. (2024). *Contract Verification: Verifying and Publishing Smart Contracts*. Consultado el 4 de abril de 2026, desde <https://docs.etherscan.io/v/sepolia-etherscan/>
- Fanning, K., & Centers, D. P. (2016). Blockchain and Its Coming Impact on Financial Services. *Journal of Corporate Accounting & Finance*, 27(5), 53-57. <https://doi.org/10.1002/jcaf.22179>
- Jefatura del Estado. (2023). Ley 6/2023 de los Mercados de Valores y de los Servicios de Inversión. Consultado el 9 de febrero de 2026, desde <https://www.boe.es/buscar/act.php?id=BOE-A-2023-7053>
- J.P. Morgan. (2024). *Introducing Kinexys by J.P. Morgan: The next chapter for our blockchain business unit* (inf. téc.). J.P. Morgan Payments. Consultado el 8 de febrero de 2026, desde <https://www.jpmorgan.com/insights/payments/blockchain-digital-assets/introducing-kinexys>
- Kaneko, Y. (2025). Comparing Electricity Consumption Per Use of Blockchain and Generative AI. *IEEE Access*, 13, 93775-93785. <https://doi.org/10.1109/ACCESS.2025.3573722>
- Lombrozo, E., Lau, J., & Wuille, P. (2015). *BIP 141: Segregated Witness (Consensus layer)* (Standard). Bitcoin Improvement Proposals. Consultado el 26 de diciembre de 2025, desde <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>
- Merkle, R. C. (1980). Protocols for Public Key Cryptosystems. *1980 IEEE Symposium on Security and Privacy*, 122.
- Meta Platforms, Inc. (2023). *React: The library for web and native user interfaces*. Consultado el 6 de marzo de 2026, desde <https://react.dev/learn>
-

- Moore, R. (2023). *Ethers.js: A complete and compact library for interacting with the Ethereum Blockchain and its ecosystem*. Consultado el 6 de marzo de 2026, desde <https://docs.ethers.org/>
- Mozilla Developer Network. (2024). *JavaScript Reference: Promise.all()*. Consultado el 1 de abril de 2026, desde https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all
- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Consultado el 8 de diciembre de 2025, desde <https://cdn.nakamotoinstitute.org/docs/bitcoin.pdf>
- Nomic Foundation. (2024). *Hardhat: The Ethereum development environment for professionals*. Consultado el 12 de abril de 2026, desde <https://hardhat.org/docs>
- OECD. (2020). *The Tokenisation of Assets and Potential Implications for Financial Markets* (inf. téc.). OECD Publishing. Paris.
- Ondo Finance. (2025). *Re: Ondo Finance - Roadmap for Tokenized Securities* (inf. téc.). U.S. Securities y Exchange Commission. Consultado el 8 de febrero de 2026, desde <https://www.sec.gov/files/ctf-written-input-ondo-finance-120425.pdf>
- OpenZeppelin. (2023). *OpenZeppelin Contracts: A library for secure smart contract development*. Consultado el 1 de marzo de 2026, desde <https://docs.openzeppelin.com/contracts/>
- OpenZeppelin. (2024a). *Access Control: Ownership and Role-Based Access Control (RBAC)*. Consultado el 15 de abril de 2026, desde <https://docs.openzeppelin.com/contracts/5.x/access-control>
- OpenZeppelin. (2024b). *ERC20 API Reference: __mint*. Consultado el 22 de marzo de 2026, desde https://docs.openzeppelin.com/contracts/4.x/api/token/erc20#ERC20-__mint-address-uint256-
- Parlamento Europeo y Consejo de la Unión Europea. (2023). Reglamento (UE) 2023/1114 relativo a los mercados de criptoactivos. Consultado el 9 de febrero de 2026, desde <https://www.boe.es/buscar/doc.php?id=DOUE-L-2023-80808>
- Pinata Technologies. (2024). *Pinata Documentation: IPFS Pinning Services*. Consultado el 20 de marzo de 2026, desde <https://docs.pinata.cloud/>
- Pitta, J. (1999). Requiem for a Bright Idea. Consultado el 27 de diciembre de 2025, desde <https://www.forbes.com/forbes/1999/1101/6411390a.html>
- Protocol Labs. (2024). *IPFS Documentation: Content Addressing and CIDs*. Consultado el 20 de marzo de 2026, desde <https://docs.ipfs.tech/concepts/content-addressing/>
-

- Ripple & Boston Consulting Group. (2025). Approaching the Tokenization Tipping Point. Consultado el 8 de febrero de 2026, desde <https://ripple.com/reports/approaching-tokenization-at-the-tipping-point.pdf>
- RWA.xyz. (2026). Real-World Asset Analytics Dashboard: Real Estate and Market Overview. Consultado el 21 de abril de 2026, desde <https://app.rwa.xyz/>
- Szabo, N. (1996). Smart Contracts: Building Blocks for Digital Markets. Consultado el 10 de diciembre de 2025, desde <https://nakamotoinstitute.org/library/smart-contracts-building-blocks-for-digital-markets/>
- Szabo, N. (2005). Bit Gold. Consultado el 10 de diciembre de 2025, desde <https://nakamotoinstitute.org/library/bit-gold/>
- Tokeny Solutions. (2023). *ERC3643: The Official Smart Contract Standard for Permissioned Tokens (T-REX Whitepaper v4)* (inf. téc.). Tokeny Solutions. Consultado el 10 de febrero de 2026, desde <https://tokeny.com/wp-content/uploads/2023/05/ERC3643-Whitepaper-T-REX-v4.pdf>
- Tutellus. (2025a). Cómo tokenizar un inmueble: convertir el ladrillo en un activo líquido, global y sin burocracia. Consultado el 6 de febrero de 2026, desde <https://criptoblog.tutellus.com/post/como-tokenizar-un-inmueble>
- Tutellus. (2025b). Tokenización de viviendas: la guía definitiva para invertir desde 100 €. Consultado el 6 de febrero de 2026, desde <https://criptoblog.tutellus.com/post/tokenizacion-de-vivienda>
- Tutellus. (2025c). Tokenizar inmuebles en España: la guía definitiva para inversores y propietarios. Consultado el 6 de febrero de 2026, desde <https://criptoblog.tutellus.com/post/tokenizar-inmuebles-en-espana>
- U.S. Securities and Exchange Commission. (2024). *Form D: Notice of Exempt Offering of Securities - BlackRock USD Institutional Digital Liquidity Fund Ltd.* (inf. téc. N.º CIK 0002013810). U.S. Securities y Exchange Commission. Consultado el 8 de febrero de 2026, desde https://www.sec.gov/Archives/edgar/data/2013810/000201439024000001/xslFormDX01/primary_doc.xml
- Vercel. (2024). *Vercel Documentation: The platform for frontend developers*. Consultado el 18 de marzo de 2026, desde <https://vercel.com/docs>
- Walker, G. (2015a). Block | A Container for Bitcoin Transactions. Consultado el 18 de diciembre de 2025, desde <https://learnmeabitcoin.com/technical/block/>
- Walker, G. (2015b). Elliptic Curve | Mathematics for Cryptographic Systems. Consultado el 18 de diciembre de 2025, desde <https://learnmeabitcoin.com/technical/cryptography/elliptic-curve/>
-

- Walker, G. (2015c). Hash Function | Fingerprints for Data. Consultado el 18 de diciembre de 2025, desde <https://learnmeabitcoin.com/technical/cryptography/hash-function/>
- Walker, G. (2025a). Hard Fork | A Change that is Incompatible with Old Software. Consultado el 18 de diciembre de 2025, desde <https://learnmeabitcoin.com/technical/blockchain/hard-fork/>
- Walker, G. (2025b). Soft Fork | A Change that is Compatible with Old Software. Consultado el 18 de diciembre de 2025, desde <https://learnmeabitcoin.com/technical/blockchain/soft-fork/>
- Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*. Consultado el 1 de marzo de 2026, desde <https://ethereum.github.io/yellowpaper/paper.pdf>
- Yaga, D., Mell, P., Roby, N., & Scarfone, K. (2018). *Blockchain technology overview* (NIST interagency/internal report N.º 8202). National Institute of Standards y Technology. <https://doi.org/10.6028/NIST.IR.8202>
- Zheng, G., Gao, L., Huang, L., & Guan, J. (2021). Operation Principles of Smart Contract. En *Ethereum Smart Contract Development in Solidity* (pp. 159-195). Springer Nature Singapore. https://doi.org/10.1007/978-981-15-6218-1_6
-

A. Anexo

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 // Importamos el estándar ERC-20 de OpenZeppelin
5 import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
6
7 contract ContratoInmueble is ERC20 {
8
9     // Variables públicas para guardar los datos del mundo real
10    string public direccionInmueble;
11    string public ipfsDocumentoUrl; // CID del PDF de la escritura
12    string public ipfsImagenUrl; // CID de la imagen del inmueble
13
14    /**
15     * @dev Constructor que se ejecuta UNA SOLA VEZ al crear el contrato
16     */
17    constructor(
18        string memory _nombre,           // Ej: "Piso Elche Altabix"
19        string memory _simbolo,          // Ej: "EAX"
20        uint256 _cantidadTotalTokens,    // Ej: 100000 (partes en las que
21        dividimos la casa)
22        string memory _direccion,        // Ej: "Calle Sixto Marco 1, Número
23        2, Planta 3, Puerta 4"
24        string memory _ipfsUrl,          // Ej: "ipfs://QmTuY..." (El hash del
25        PDF)
26        string memory _ipfsImagenUrl,    // Ej: "ipfs://QmTuY..." (El hash de
27        la imagen)
28        address _mercado                 // La dirección del contrato general
29        que gestionará las ventas
30    ) ERC20(_nombre, _simbolo) {
31
32        direccionInmueble = _direccion;
33        ipfsDocumentoUrl = _ipfsUrl;
34        ipfsImagenUrl = _ipfsImagenUrl;
35
36        // Creamos todos los tokens y se los damos al contrato general (
37        Mercado)
38        // Se multiplica por 10**decimals() porque blockchain no entiende de
39        decimales reales
40        _mint(_mercado, _cantidadTotalTokens * 10 ** decimals());
41    }
42 }
```

Código A.1: Contrato de una vivienda

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 // Importamos el contrato de la vivienda para poder crear tokens desde aquí
5 import "../ContratoInmueble.sol";
6 // Importamos la interfaz estándar ERC20 para poder interactuar con los
  tokens
7 import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
8
9 contract MercadoInmobiliario {
10
11     // El administrador será mi wallet (para que solo yo pueda subir PDFs/
      crear casas)
12     address public administrador;
13
14     // Creamos un Struct para guardar los datos comerciales de cada casa
15     struct DetallesInmueble {
16         address direccionContrato;
17         uint256 precioPorToken; // En Wei (la medida más pequeña de ETH)
18         uint256 tokensDisponibles; // Cuántos quedan en la tienda
19     }
20
21     // Un diccionario (mapping) que asocia el DNI del contrato con su ficha
      comercial
22     mapping(address => DetallesInmueble) public inmuebles;
23
24     // Una lista (array) con las direcciones de todas las casas para
      mostrarlas en la web
25     address[] public listaInmuebles;
26
27     // Eventos (Señales que el contrato envía a la web para actualizar la
      pantalla)
28     event InmuebleCreado(address tokenAddress, string nombre, uint256 precio)
      ;
29     event TokensComprados(address comprador, address token, uint256 cantidad)
      ;
30     event TokensVendidos(address vendedor, address token, uint256 cantidad);
31     event PrecioModificado(address tokenInmueble, uint256 precioAntiguo,
      uint256 precioNuevo);
32
33     constructor() {
34         administrador = msg.sender; // El que despliega el Mercado se
      convierte en el administrador
35     }
36
37     /// @dev FUNCIÓN 1: El Administrador crea una nueva vivienda
38     function crearInmueble(
39         string memory _nombre,
40         string memory _simbolo,
41         uint256 _cantidadTotal,
42         string memory _direccionFisica,
43         string memory _ipfsUrl,
44         string memory _ipfsImagenUrl,

```

```
45     uint256 _precioPorToken
46   ) public {
47     require(msg.sender == administrador, "Solo el administrador puede
        tokenizar casas");
48
49     // Despliegue del clon del contrato ContratoInmueble (El Patrón
        Factory)
50     ContratoInmueble nuevoInmueble = new ContratoInmueble(
51         _nombre, _simbolo, _cantidadTotal, _direccionFisica, _ipfsUrl,
        _ipfsImagenUrl, address(this)
52     );
53
54     // Obtención de su DNI (address) único
55     address direccionToken = address(nuevoInmueble);
56
57     // Registro en la base de datos de nuestra tienda
58     inmuebles[direccionToken] = DetallesInmueble({
59         direccionContrato: direccionToken,
60         precioPorToken: _precioPorToken,
61         tokensDisponibles: _cantidadTotal * 10 ** 18 // Ajuste por los 18
        decimales de blockchain
62     });
63
64     // Añadir a la lista
65     listaInmuebles.push(direccionToken);
66
67     // Aviso a la web
68     emit InmuebleCreado(direccionToken, _nombre, _precioPorToken);
69   }
70
71   /// @dev FUNCIÓN 2: Un inversor compra tokens pagando con criptomonedas
        nativas (ETH/MATIC)
72
73   function comprarTokens(address _tokenInmueble, uint256 _cantidadTokens)
        public payable {
74     DetallesInmueble storage inmueble = inmuebles[_tokenInmueble];
75     require(inmueble.direccionContrato != address(0), "El inmueble no
        existe");
76
77     uint256 cantidadTokensDec = _cantidadTokens * 10 ** 18;
78     require(inmueble.tokensDisponibles >= cantidadTokensDec, "No quedan
        suficientes tokens");
79
80     // Calculamos cuánto tiene que pagar en total
81     uint256 costeTotal = inmueble.precioPorToken * _cantidadTokens;
82     require(msg.value >= costeTotal, "No has enviado suficiente ETH para
        la compra");
83
84     // Actualizamos el inventario de la tienda
85     inmueble.tokensDisponibles -= cantidadTokensDec;
86
87     // Enviamos los tokens del Mercado a la wallet del comprador
88     IERC20(_tokenInmueble).transfer(msg.sender, cantidadTokensDec);
```

```

89         emit TokensComprados(msg.sender, _tokenInmueble, _cantidadTokens);
90     }
91
92
93     /// @dev FUNCIÓN 3: Un propietario vende tokens y recibe criptomonedas.
94     function venderTokens(address _tokenInmueble, uint256 _cantidadTokens)
95         public {
96         DetallesInmueble storage inmueble = inmuebles[_tokenInmueble];
97         require(inmueble.direccionContrato != address(0), "El inmueble no
98             existe");
99
100         uint256 cantidadTokensDec = _cantidadTokens * 10 ** 18;
101
102         // El Mercado coge los tokens de la wallet del usuario y los devuelve
103         // a la tienda
104         // (Para ello el usuario debe haber dado permiso 'approve' antes en
105         // la web)
106         IERC20(_tokenInmueble).transferFrom(msg.sender, address(this),
107             cantidadTokensDec);
108
109         // Sumamos los tokens al inventario
110         inmueble.tokensDisponibles += cantidadTokensDec;
111
112         // Le devolvemos su ETH al usuario
113         uint256 pagoTotal = inmueble.precioPorToken * _cantidadTokens;
114         (bool exito, ) = payable(msg.sender).call{value: pagoTotal}("");
115         require(exito, "Fallo al devolver el ETH");
116
117         emit TokensVendidos(msg.sender, _tokenInmueble, _cantidadTokens);
118     }
119
120     /// @dev FUNCIÓN 4: El administrador actualiza el precio del token según
121     // el mercado real
122     function modificarPrecio(address _tokenInmueble, uint256 _nuevoPrecio)
123         public {
124         require(msg.sender == administrador, "Solo el administrador puede
125             cambiar el precio");
126
127         DetallesInmueble storage inmueble = inmuebles[_tokenInmueble];
128         require(inmueble.direccionContrato != address(0), "El inmueble no
129             existe");
130
131         inmueble.precioPorToken = _nuevoPrecio;
132
133         emit PrecioModificado(_tokenInmueble, inmueble.precioPorToken,
134             _nuevoPrecio);
135     }
136 }

```

Código A.2: Contrato del mercado